



TRABAJO DE FINAL DE GRADO

GRADO EN INTELIGENCIA ROBÓTICA

Estudio e implementación del control de péndulo invertido sobre carro en un sistema de tiempo real

Estudiante:
Miguel Porcar Vicent

Tutor:
Ignacio Peñarrocha Alós

Fecha de presentación: Junio de 2026
Curso académico: 2025/2026

Agradecimientos:

Palabras clave:

CASTELLÓN, a XX de XXXXXXX de 2025

Todos los derechos reservados

ÍNDICE GENERAL

1	Introducción	7
1.1	Punto de partida	8
1.2	Motivaciones	10
1.3	Planificación	10
1.3.1	Objetivos	10
1.3.2	Tareas	11
1.3.3	Estimación de tiempos	12
1.4	Estimación de costes	13
2	Análisis	15
2.1	Requisitos funcionales	16
2.2	Requisitos no funcionales	17
2.3	Diseño del sistema	18
2.3.1	Introducción al modelado dinámico	18
2.3.2	Modelo dinámico del sistema	19
2.3.3	Estrategia de control Swing-up	21
2.3.4	Modelado en el espacio de estados	24
2.3.5	Control por realimentación del estado	26
2.3.6	Lógica de conmutación	27
2.4	Arquitectura del sistema	28
2.4.1	Arquitectura de la simulación	28
2.4.2	Arquitectura del sistema real	29
2.4.3	Especificación de componentes de hardware	30

3	Desarrollo	35
3.1	Trabajo de simulación	36
3.1.1	Modelado del sistema en Simulink	36
3.1.2	Algoritmo de control en Simulink	37
3.1.3	Comunicación con ROS y Visualización del Gemelo Digital	40
3.1.4	Simulaciones Recursivas	42
3.2	Programación del microcontrolador	44
3.2.1	Módulo eQEP (Lectura del Encoder)	47
3.2.2	Módulo ePWM1 (Actuación del Motor)	48
3.2.3	Protocolo de comunicación serie y telemetría (SCI)	51
3.2.4	Rutina de interrupción periódica y determinismo temporal	54
3.3	Compensación y caracterización de la zona muerta	55
3.3.1	Metodología de caracterización experimental	56
3.3.2	Influencia de la frecuencia de PWM y selección de diseño	57
3.4	Identificación de parámetros	58
3.4.1	Relación entre fuerza y voltaje	58
3.4.2	Masa y rozamiento del carro	61
3.4.3	Rozamiento del péndulo	64
3.4.4	Validación y contraste con los parámetros del fabricante	66
3.5	Implementación del algoritmo de control	67
3.5.1	Control de posición en <code>calcula_u()</code>	68
3.5.2	Control de energía (Swing-up) en <code>calcula_u()</code>	69
4	Resultados	71
4.1	Puesta a punto e integración del hardware	71
4.2	Control de posición	73
4.3	Control de energía (Swing-up)	76
4.4	Conmutación entre controles	76
4.5	Comparativa entre los resultados alcanzados y los objetivos iniciales	76
4.6	Evaluación de la planificación temporal y justificación de desviaciones	76
5	Conclusiones y posible trabajo futuro	77
	Bibliografía	79
6	Apéndices	81
A	Desarrollo matemático y modelado dinámico complementario	81
A.1	Desarrollo del Lagrangiano	81
A.2	Análisis de la dinámica eléctrica del actuador	82
A.3	Función de transferencia del carro	83
B	Soporte al trabajo de simulación	84
B.1	Dinámica en MATLAB Functions	84
B.2	Script control de energía	86
B.3	Configuración de la conexión MATLAB ROS	87
B.4	Desarrollo de simulaciones recursivas	89

C	Configuración y registros del microcontrolador (TMS320F280049C)	90
C.1	Tabla de Multiplexación Completa del TMS32F280049C	90
C.2	Registros del módulo eQEP	92
C.3	Registros del módulo ePWM	96
C.4	Tasa de baudios del SCI	101
C.5	Configuración de la ISR en detalle	103

1. Introducción

El objeto de este proyecto es el estudio y control de un péndulo invertido sobre un carro móvil. Aunque a primera vista pueda parecer un sistema mecánico simple, en el ámbito de la **Robótica** este dispositivo constituye uno de los problemas fundamentales de control y equilibrio dinámico.

La relevancia de este sistema reside en su transferencia directa a tecnologías críticas de la robótica moderna. El reto de mantener una barra inestable en equilibrio vertical es, en esencia, el mismo principio físico que permite a un **robot humanoide** caminar sin desplomarse, a un cohete mantenerse estable durante el despegue o a vehículos de transporte personal (como los *Segways*) desplazarse de forma autónoma. Controlar este sistema implica dominar la capacidad de un robot para reaccionar ante la gravedad y las perturbaciones externas en milisegundos.

En ingeniería, este reto se utiliza para evaluar la eficacia de un sistema de control. El objetivo es que un microcontrolador (cerebro) interprete la inclinación de la barra mediante sensores (ojos) y actúe sobre un motor (músculos) con una velocidad y precisión tales que desafíen la inestabilidad natural del sistema.

Finalmente, en el marco de la robótica, este proyecto se sitúa en la denominada **capa reactiva**. Mientras que gran parte de la robótica actual se enfoca en procesos cognitivos de alto nivel —como la navegación mediante SLAM—, este sistema aborda el control a bajo nivel, equiparable a los **reflejos motores** de un ser vivo. No se trata de "pensar" una trayectoria a largo plazo, sino de reaccionar en milisegundos para corregir desviaciones imperceptibles. Esta capacidad de respuesta inmediata es la que garantiza que el robot sea capaz de interactuar con el mundo físico de forma segura, sirviendo de base fundamental sobre la que se construyen las capas de inteligencia superior.

1.1 Punto de partida

El trabajo se realiza sobre un equipo educativo clásico: el **Digital Pendulum 33-005** de la marca *Feedback Instruments Ltd.* Este sistema se divide en dos partes: el controlador 33-201 (la caja con la electrónica) y la unidad mecánica 33-200 (el raíl con el carro y la barra).

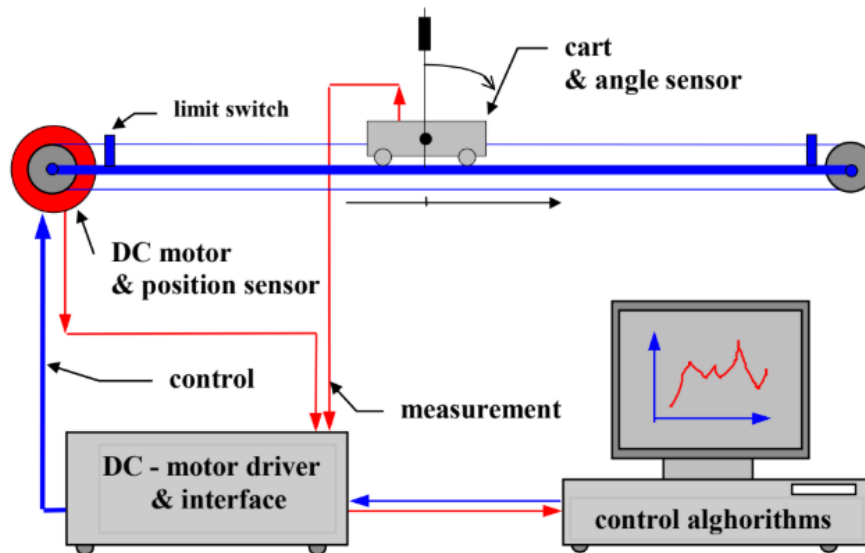


Figura 1.1: Digital Pendulum 33-005, descripción esquemática [1]

En la Figura 1.1 se detalla la disposición de la unidad mecánica 33-200. El sistema consta de un raíl sobre el que se desplaza horizontalmente un carro metálico. Este movimiento es accionado por un motor de corriente continua situado en un extremo, que transmite la fuerza mediante una correa dentada. En los extremos del raíl se sitúan los finales de carrera, que actúan como interruptores de seguridad para detener el motor antes de que el carro colisione con la estructura. Sobre el carro se monta el eje del péndulo, cuya rotación libre de 360 grados es captada por un sensor óptico (encoder) situado en su base.

Este equipo lleva en los laboratorios de la universidad más de 25 años. Debido a este largo periodo de tiempo, el punto de partida del proyecto se caracteriza por una obsolescencia tecnológica total:

- **Electrónica antigua:** Según la documentación del fabricante, este sistema fue diseñado para operar con **MATLAB 5** y **Windows 95**, utilizando protocolos de comunicación y tarjetas de expansión (bus ISA/PCI) que ya no existen en los ordenadores actuales [2]. Esto lo convertía en una "caja negra" cerrada e incompatible con las herramientas de programación modernas, lo que justificaba su sustitución completa por un sistema de control digital actual.

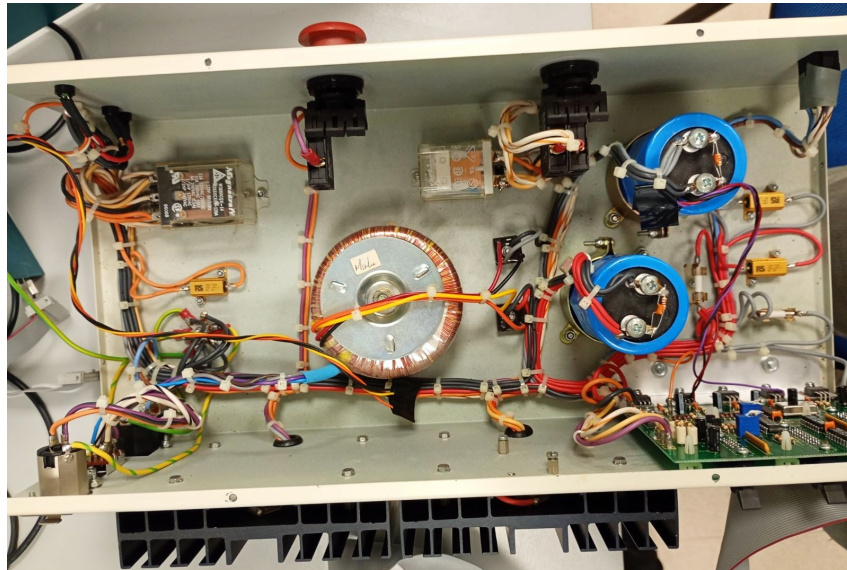
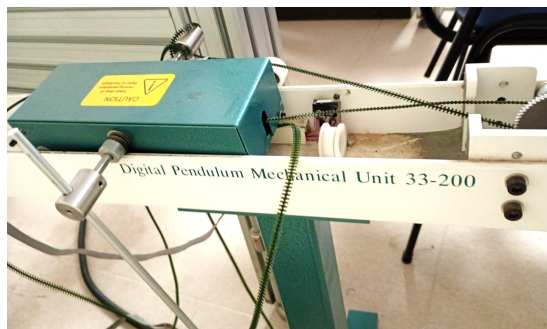
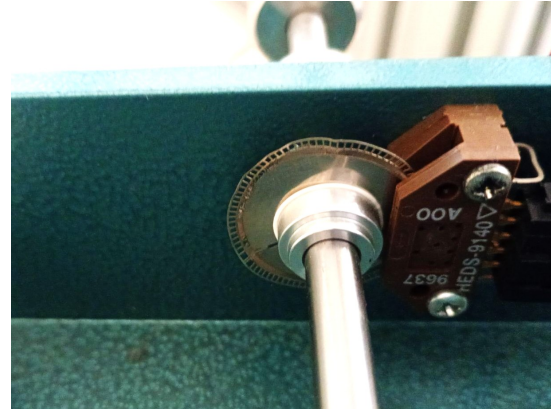


Figura 1.2: Controlador digital original

- **Falta de mantenimiento:** La unidad mecánica presentaba suciedad acumulada en los raíles y grasa reseca en los engranajes, lo que impedía que el carro se moviera con suavidad, complicando enormemente la tarea de equilibrio. Además de tener un desgaste notable en la correa dentada (transmisión) y el disco óptico del encoder.



(a) Unidad mecánica con falta de mantenimiento



(b) Disco óptico doblado y con pérdida de pulsos

Figura 1.3: Estado inicial de la unidad mecánica

Luego el punto de partida es el **rescate de la unidad mecánica**, aprovechando aquellos elementos que aún conservaban su utilidad, como el motor. No obstante, ha sido necesario intervenir en otros componentes que, debido a su naturaleza como piezas de desgaste, presentaban falta de mantenimiento. Por otro lado, para solucionar el problema del control digital obsoleto, se decidió **sustituir la electrónica antigua** por tecnología actual (microcontrolador de la familia *C2000 de Texas Instruments* y drivers modernos). Esta elección no solo permite que la unidad vuelva a ser plenamente funcional, sino que además la convierte en una plataforma de **código abierto**, facilitando que futuros usuarios puedan entender, modificar y mejorar el sistema sin las restricciones del hardware propietario original.

1.2 Motivaciones

Las razones que impulsan el desarrollo de este proyecto combinan el interés personal por ciertos conocimientos adquiridos en el grado y la oportunidad de brindar un servicio a la universidad:

- **Personal y Vocacional:** Mi interés principal dentro del *Grado en Inteligencia Robótica* reside en el control automático, la programación de sistemas empotrados en tiempo real y la electrónica. Este proyecto me permite profundizar en la asignatura de *Sistemas Automáticos Avanzados*, enfrentándome al reto de diseñar algoritmos de control a bajo nivel con restricciones críticas de tiempo y hardware que imponen los sistemas físicos reales.
- **Curiosidad Científica:** Me motiva entender el funcionamiento interno de las máquinas. El control automático me brinda la oportunidad perfecta para aplicar de forma práctica la física y las matemáticas que subyacen a los sistemas robóticos capaces de equilibrarse y moverse con precisión.
- **Profesional:** El uso de la familia de microcontroladores *C2000 de Texas Instruments* constituye una motivación clave. Superar las capas de abstracción de *Arduino IDE* para trabajar directamente sobre el registro y el hardware de los *C2000* me permite entender el control de bajo nivel de forma real. Esta capacidad para gestionar sistemas complejos con estándares industriales constituye una ventaja competitiva.
- **Académica:** Durante este último curso, he disfrutado de la *Beca de Colaboración del Ministerio* en el *Departamento de Ingeniería de Sistemas Industriales y Diseño*. Parte de mi labor ha consistido en poner en marcha esta infraestructura, por lo que este TFG es la culminación del trabajo. Dejar una unidad funcional y actualizada que sirva como banco de pruebas para que futuros estudiantes e investigadores puedan seguir evolucionando algoritmos y técnicas de control avanzado en la universidad sería un gran honor.

1.3 Planificación

Dentro de este apartado se aborda de forma lógica y ordenada los objetivos generales del proyecto y las tareas asociadas para lograr su consecución. Además se expone el *Diagrama de Gantt* para tener una visión general de la planificación de tiempos por cada tarea.

1.3.1 Objetivos

- **O1:** Obtención del modelo matemático del sistema de péndulo invertido sobre carro.
- **O2:** Simulación en *Matlab/Simulink* para la validación preliminar de los controladores e integración con *ROS*.
- **O3:** Diseño y evaluación en simulación diferentes estrategias de control.
- **O4:** Caracterización y puesta a punto el hardware de la planta real, incluyendo la etapa de potencia y la lectura de sensores.
- **O5:** Desarrollo el firmware de bajo nivel en el microcontrolador, garantizando el control en tiempo real.
- **O6:** Implementación, ajuste y validación experimental de los algoritmos de control sobre la unidad mecánica real.
- **O7:** Documentación técnica del trabajo realizado y los resultados obtenidos.

1.3.2 Tareas

Cada tarea y subtarea tiene un objetivo asociado para facilitar el desarrollo del proyecto en pequeños retos dentro de un plazo. De este modo la organización es mayor y el enfoque del trabajo es claro.

T1. Modelado matemático y físico (O1)

- T1.1 Obtención de las ecuaciones de movimiento mediante Euler-Lagrange.
- T1.2 Identificación de parámetros cinemáticos y dinámicos (masas, longitudes, inercias).
- T1.3 Caracterización de las no linealidades: fricción estática y dinámica en el raíl y zona muerta del motor.

T2. Entorno de simulación e integración con ROS (O2)

- T2.1 Implementación del modelo dinámico en bloques de *Simulink*.
- T2.2 Configuración del nodo de comunicación entre *Matlab* y el ecosistema *ROS* para el intercambio de datos.
- T2.3 Desarrollo de un entorno visual para la monitorización de la simulación.

T3. Diseño de estrategias de control en simulación (O3)

- T3.1 Diseño del control de posición mediante regulador cuadrático lineal (LQR).
- T3.2 Desarrollo del algoritmo de *Swing-up* basado en el control de energía.

T4. Preparación del hardware y electrónica (O4)

- T4.1 Revisión mecánica y mantenimiento de la unidad *Feedback 33-200* (limpieza, engrasado y ajuste de tensión).
- T4.2 Integración del driver de potencia y diseño del conexionado con los encoders ópticos.
- T4.3 Implementación de sistemas de seguridad físicos (finales de carrera) y lógicos.

T5. Desarrollo del firmware de bajo nivel (O5)

- T5.1 Configuración de periféricos de captura (eQEP) y generación de señales PWM en el microcontrolador.
- T5.2 Programación del bucle de control con una rutina de interrupción periódica.
- T5.3 Implementación del protocolo de comunicación entre el microcontrolador y el ordenador para obtener la telemetría.

T6. Validación experimental en el sistema real (O6)

- T6.1 Ejecución y ajuste fino de los controladores (*tuning*) sobre la planta física.
- T6.2 Comparativa de rendimiento entre los resultados obtenidos en el laboratorio y la simulación previa.

T7. Documentación y cierre (O7)

- T7.1 Elaboración de la memoria técnica del proyecto.
- T7.2 Preparación de la defensa pública y material visual de apoyo.

1.3.3 Estimación de tiempos

Para garantizar la viabilidad del proyecto dentro de un marco temporal de **300 horas totales** y con la actividad presencial estructurada en jornadas de **6 horas diarias** (en horario de 08:00 a 14:00 de lunes a viernes). Se ha estructurado la planificación temporal en un cómputo de 30 horas semanales, fijando la duración total del cronograma en un periodo de **10 semanas**.

En el Cuadro 1.1 se detalla la asignación horaria prevista para cada una de las tareas principales y sus objetivos asociados. Dado el desafío técnico que representa la programación a bajo nivel sobre el microcontrolador, se ha otorgado una prioridad destacada al desarrollo del firmware (T5), reservando un total de 70 horas para su correcta ejecución y depuración. Por otro lado, se reservan 50 horas para la redacción final de la memoria técnica y la preparación de la defensa (T7). No obstante, la labor de documentación se plantea como una actividad transversal y continua a lo largo del proyecto.

Cuadro 1.1: Desglose de horas estimadas por tarea principal.

Tarea	Descripción	Objetivo	Horas Estimadas
T1	Modelado matemático y físico	O1	30 h
T2	Entorno de simulación e integración con ROS	O2	40 h
T3	Diseño de estrategias de control en simulación	O3	30 h
T4	Preparación del hardware y electrónica	O4	30 h
T5	Desarrollo del firmware de bajo nivel	O5	70 h
T6	Validación experimental en el sistema real	O6	50 h
T7	Documentación técnica y cierre	O7	50 h
Total estimado:			300 h

Se ha elaborado el diagrama de Gantt de la Figura 1.4. Como se aprecia en el cronograma, se ha diseñado un solapamiento estratégico entre las tareas de modelado y simulación virtual (T1, T2 y T3) y la fase de preparación del hardware (T4). Permitiendo optimizar las horas de laboratorio, garantizando una interacción directa y continuada con la planta física desde etapas tempranas. A su vez, se observa claramente que el grueso central del proyecto lo ocupa el desarrollo del firmware (T5), dedicando 4 semanas.

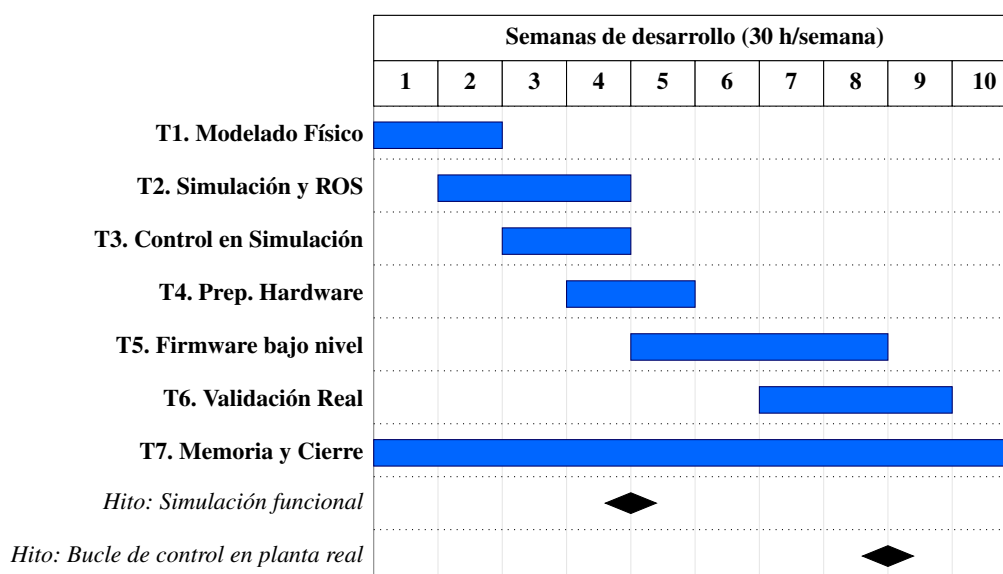


Figura 1.4: Diagrama de Gantt de la planificación prevista para el proyecto

1.4 Estimación de costes

2. Análisis

En este capítulo se realiza un estudio detallado de los pilares que sustentan el desarrollo del sistema de control para el péndulo invertido. El análisis se articula en tres ejes fundamentales:

- **Análisis de requisitos:** Donde se formalizan las capacidades operativas (funcionales) y las restricciones técnicas, de seguridad y rendimiento (no funcionales) que debe satisfacer el sistema.
- **Diseño del sistema:** Se describe la estrategia lógica y matemática adoptada, detallando el modelo dinámico, las leyes de control híbridas y la lógica de conmutación necesaria para el *swing-up* y la estabilización.
- **Arquitectura del sistema:** Se define la infraestructura tanto de simulación como de implementación real, especificando el flujo de datos, la integración con ROS y la selección de los componentes de hardware que garantizan la viabilidad del proyecto.

2.1 Requisitos funcionales

Los requisitos funcionales describen las acciones, cálculos y manipulaciones de datos específicos que el sistema debe realizar para cumplir su propósito. Para el control del péndulo invertido sobre carro se definen mediante sus entradas, comportamiento y salidas:

■ RF1. Modelado y simulación de la dinámica:

- *Entradas:* Parámetros físicos del sistema (masas, inercias, coeficientes de fricción) y señal de control simulada.
- *Comportamiento:* Resolución matemática de las Ecuaciones Diferenciales Ordinarias (EDO) que rigen la dinámica no lineal del mecanismo en el entorno MATLAB/Simulink.
- *Salidas:* Estado cinemático simulado del sistema (posiciones y velocidades articulares).

■ RF2. Visualización y telemetría en ROS:

- *Entradas:* Estado cinemático del sistema (proveniente de la simulación o del hardware real) y archivo de descripción geométrica URDF.
- *Comportamiento:* Traducción y publicación de los datos mediante el puente de comunicación de ROS Toolbox para actualizar el árbol de transformadas (TF) del robot.
- *Salidas:* Visualización 3D actualizada en tiempo real mediante la interfaz RViz (tópico `/joint_states`).

■ RF3. Control de inyección de energía (Swing-up):

- *Entradas:* Posición y velocidad angular actual del péndulo $(q, \dot{q})$.
- *Comportamiento:* Cálculo y ejecución de una ley de control basada en energía para balancear el péndulo desde su posición estable ($q = 0$ rad) hasta el punto de linealización del sistema ($q = \pi$ rad).
- *Salidas:* Acción de control (u) .

■ RF4. Control de estabilización (Realimentación del estado):

- *Entradas:* Vector de estado del sistema $(x = [x, \dot{x}, \theta, \dot{\theta}]^T \equiv [q, \dot{q}]^T)$.
- *Comportamiento:* Estabilización del sistema sobre el punto de linealización utilizando control por realimentación del estado.
- *Salidas:* Acción de control (u) .

■ RF5. Adquisición y decodificación de señales:

- *Entradas:* Señales eléctricas de pulsos en cuadratura (A y B) provenientes de los encoders del motor y del péndulo.
- *Comportamiento:* Procesamiento y conteo de los pulsos digitales mediante el hardware específico.
- *Salidas:* Valores digitales discretos de posición y velocidad interpretables por la ley de control.

2.2 Requisitos no funcionales

Los requisitos no funcionales imponen condiciones y restricciones a la implementación del diseño:

- **RNF1. Restricciones de rendimiento (Determinismo y tiempo real):** El bucle de control debe ejecutarse dentro de una rutina de interrupción periódica en un microcontrolador, garantizando un periodo de muestreo estable y un *jitter* despreciable para la viabilidad del control discreto.
- **RNF2. Restricciones de fiabilidad (Robustez):** El sistema debe rechazar perturbaciones externas moderadas y ser tolerante a las discrepancias producidas por dinámicas no modeladas (fricciones no lineales y holguras mecánicas).
- **RNF3. Restricciones de seguridad (Gestión de fallos):**
 - *Seguridad mecánica:* El software debe implementar una parada de emergencia si las lecturas de posición indican que el carro excede los límites físicos del riel. En caso de fallo en el software de parada se debe disponer de finales de carrera para cortar la alimentación del actuador.
 - *Seguridad eléctrica:* Se debe garantizar el aislamiento galvánico entre la etapa de control (3.3V) y la etapa de potencia (24V).
- **RNF4. Condiciones de implementación (Abstracción HAL):** El código desarrollado debe hacer uso de la capa de abstracción de hardware (HAL) ofrecida por el fabricante del microcontrolador, asegurando la portabilidad, estandarización y eficiencia del firmware.

2.3 Diseño del sistema

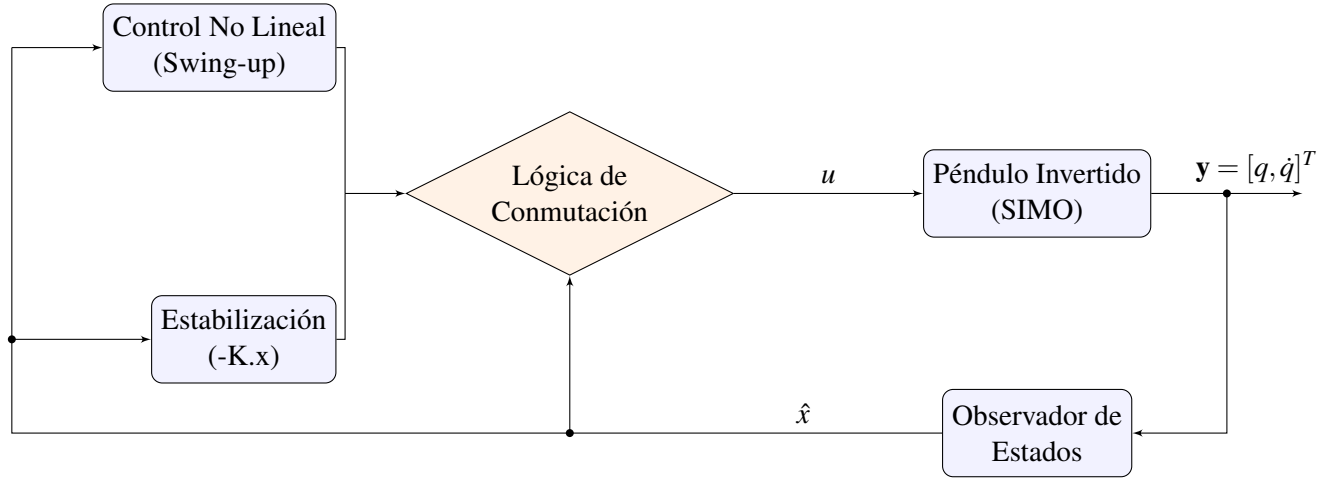


Figura 2.1: Arquitectura de control

La arquitectura de control propuesta en la Figura 2.1 se aplica sobre un sistema subactuado tipo SIMO (Single Input Multiple Output). Para la etapa de realimentación, se implementa un observador del estado por inversión. Mediante este enfoque, los estados del sistema se obtienen exclusivamente a partir de las mediciones de salida, dado que, en el caso concreto de sistemas robóticos, el estado coincide con las salidas cinemáticas $(q, \dot{q})$.

2.3.1 Introducción al modelado dinámico

A diferencia de la cinemática, que estudia el movimiento de un mecanismo sin considerar las fuerzas y pares que lo producen, el modelado dinámico describe explícitamente la relación entre la fuerza aplicada y el movimiento resultante [3]. Disponer de las ecuaciones de movimiento exactas es un requisito estricto y fundamental tanto para la simulación del sistema como para el diseño de controladores basados en modelo.

Para determinar la dinámica del péndulo invertido sobre carro, se recurre a las ecuaciones de Euler-Lagrange. Este enfoque energético se basa en la obtención del Lagrangiano del sistema (L), definido como la diferencia entre la energía cinética (E_c) y la energía potencial (E_p):

$$L = E_c - E_p \quad (2.1)$$

Las ecuaciones diferenciales no lineales resultantes pueden reescribirse y compactarse de forma elegante en la formulación matricial clásica para sistemas robóticos [4]:

$$M(q)\ddot{q} + C(q, \dot{q})\dot{q} + G(q) = Bu \quad (2.2)$$

Para el sistema en concreto, las dimensiones de cada matriz son:

- $M(q) \in \mathbb{R}^{2 \times 2}$ matriz de inercia.
- $C(q, \dot{q}) \in \mathbb{R}^{2 \times 2}$ matriz de fuerzas centrípetas y de Coriolis. Incluye también el rozamiento viscoso.
- $G(q) \in \mathbb{R}^{2 \times 1}$ vector de fuerzas gravitacionales.
- $B \in \mathbb{R}^{2 \times 1}$ vector de entrada, mapea la acción de control u .

Este enfoque matricial no solo facilita el análisis teórico, sino que permite realizar correcciones en la simulación del sistema muy fácilmente, ya que los parámetros que lo caracterizan (masas, longitudes, inercias, etc.) se encapsulan dentro de las propias matrices.

2.3.2 Modelo dinámico del sistema

El primer paso para el modelado dinámico consiste en definir un sistema de referencia en el plano bidimensional (x,y) . Esto permite localizar la posición del centro de masas (CM) del péndulo en función de las coordenadas generalizadas del sistema: la posición lineal del carro (x) y la posición angular del péndulo (θ) .

Considerando l como la distancia desde el eje de rotación hasta el centro de masas del péndulo, sus coordenadas son:

$$\begin{cases} x_{cm} = x + l \sin(\theta) \\ y_{cm} = -l \cos(\theta) \end{cases} \quad (2.3)$$

Esta elección de coordenadas establece de forma implícita el marco de trabajo del controlador. Se define el péndulo colgando hacia abajo como el origen angular ($\theta = 0$ rad) y de acuerdo con la geometría de la Ecuación 2.3, se asume una convención de signos donde el incremento de θ es en sentido antihorario.

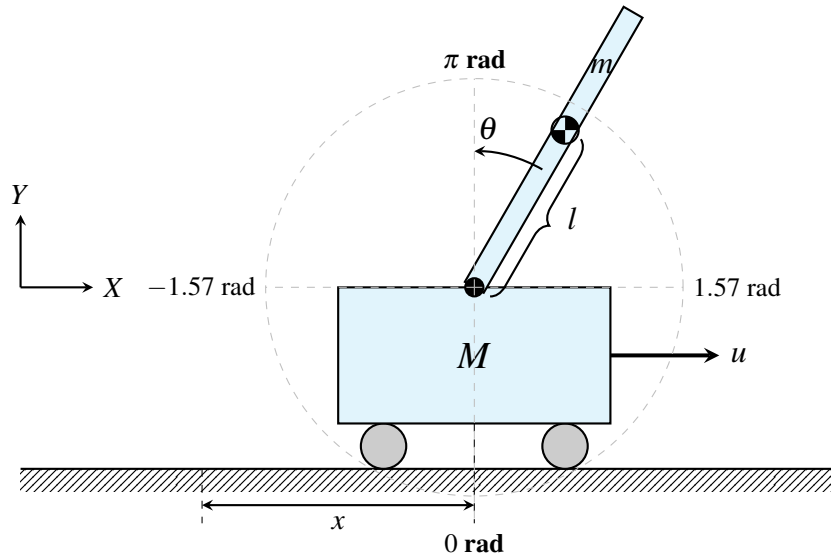


Figura 2.2: Esquema cinemático, definición del sistema de coordenadas

Una vez establecida la convención de signos del sistema y tener una visión esquemática ofrecida en la Figura 2.2, es momento de desarrollar la ecuación 2.1 hasta obtener la forma matricial compacta de la Ecuación 2.2. La energía potencial y cinética del sistema en concreto se puede representar como [5]:

$$\begin{cases} E_c = \frac{1}{2}(M+m)\dot{x}^2 + m\dot{x}\dot{\theta}l \cos(\theta) + \frac{1}{2}ml^2\dot{\theta}^2 \\ E_p = -mgl \cos(\theta) \end{cases} \quad (2.4)$$

A partir de formular la energía cinética y potencial, se desarrolla el Lagrangiano (Anexo A.1) y se calculan sus derivadas parciales hasta obtener las siguientes ecuaciones de movimiento [5]:

$$\begin{cases} (M+m)\ddot{x} + ml\ddot{\theta} \cos(\theta) - ml\dot{\theta}^2 \sin(\theta) = u \\ ml\ddot{x} \cos(\theta) + ml^2\ddot{\theta} + mgl \sin(\theta) = 0 \end{cases} \quad (2.5)$$

Antes de compactar las ecuaciones en la forma matricial de la Ecuación 2.2 es importante añadir los términos de rozamiento viscoso para el riel y el péndulo.

$$\begin{cases} (M+m)\ddot{x} + ml\ddot{\theta}\cos(\theta) + c_x\dot{x} - ml\dot{\theta}^2\sin(\theta) = u \\ ml\ddot{x}\cos(\theta) + ml^2\ddot{\theta} + c_\theta\dot{\theta} + mgl\sin(\theta) = 0 \end{cases} \quad (2.6)$$

Finalmente se compacta la expresión en forma matricial como indica la Ecuación 2.2. La caracterización del sistema que posteriormente se implementará en simulación es:

$$\begin{bmatrix} M+m & ml\cos\theta \\ ml\cos\theta & I+ml^2 \end{bmatrix} \begin{bmatrix} \ddot{x} \\ \ddot{\theta} \end{bmatrix} + \begin{bmatrix} c_x & -ml\dot{\theta}\sin\theta \\ 0 & c_\theta \end{bmatrix} \begin{bmatrix} \dot{x} \\ \dot{\theta} \end{bmatrix} + \begin{bmatrix} 0 \\ mgl\sin\theta \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \end{bmatrix} u \quad (2.7)$$

Si se analiza la Ecuación 2.7 se deducen tres características clave del sistema:

- **Sistema subactuado:** La acción de control u solo permite empujar el carro de forma horizontal, no hay control directo en el posicionado del péndulo.
- **Acoplamiento de términos:** El movimiento del carro afecta a la posición del péndulo y a su vez el movimiento del péndulo actúa como una perturbación en el posicionado del carro.
- **Sistema inestable:** Si se analiza el sistema en $\theta = \pi \text{ rad}$ presenta polos con parte real positiva debido al término $mgl\sin\theta$. Por tanto, se requiere aplicar acciones de control en bucle cerrado para poder permanecer en esta posición.

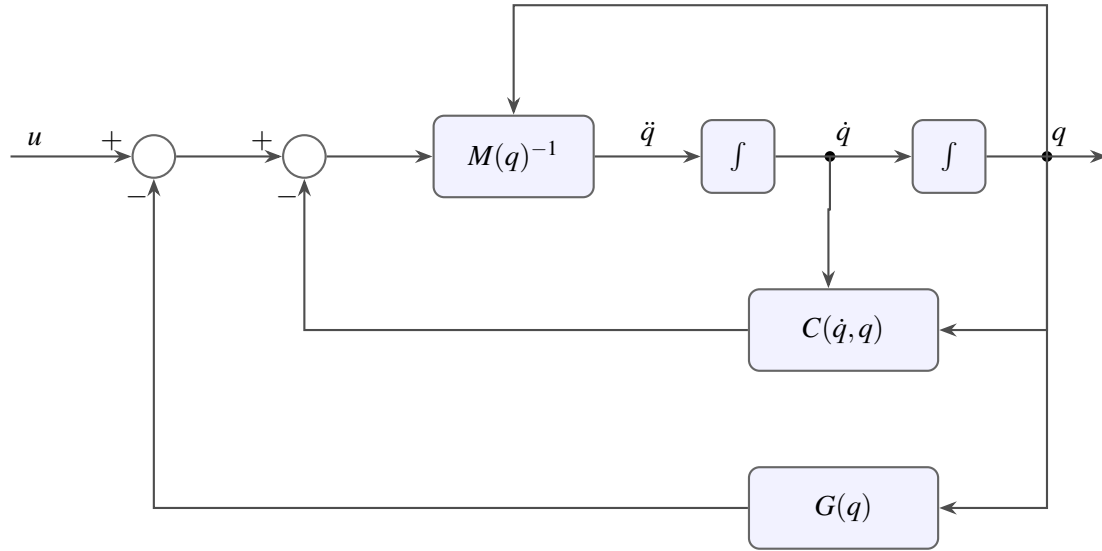


Figura 2.3: Diagrama de bloques del modelado

Finalmente, en la Figura 2.3 se muestra la representación del modelado dinámico del sistema con diagramas de bloques. Es una forma interesante de representación, ya que la traslación a *Simulink* es directa, sirviendo como preámbulo para la implementación de la simulación en este entorno.

2.3.3 Estrategia de control Swing-up

La primera ley de control se centra en el levantamiento del péndulo desde su posición estable ($\theta = 0 \text{ rad}$) hasta su posición inestable y punto en el que se conmuta al control de posición ($\theta = \pi \text{ rad}$).

La idea intuitiva tras las matemáticas es simple, consiste en balancear el péndulo inyectando energía a favor del movimiento. El objetivo es llegar al punto de estabilización con pequeñas acciones de control que producen oscilaciones de amplitud creciente en el sistema.

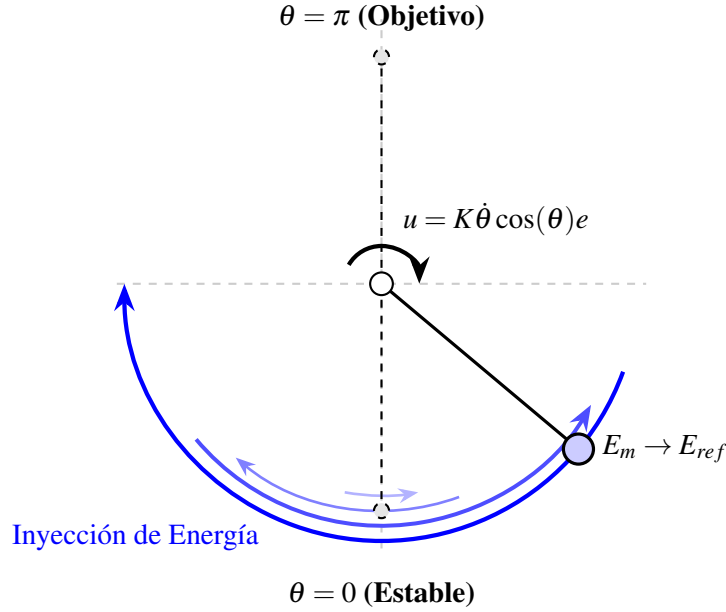


Figura 2.4: Esquema del control por balanceo (swing-up) basado en energía.

Para formalizar la ley de control esquematizada en la Figura 2.4, se define la energía del péndulo libre que viene dada por la suma de la energía potencial y la cinética [6]:

$$E_m = E_c + E_p = \frac{1}{2} J \dot{\theta}^2 - mgl \cos(\theta) \quad (2.8)$$

En el caso de la Ecuación 2.8 se define la máxima energía potencial cuando $\theta = \pi \text{ rad}$ de forma que el sistema de referencia del control coincide con el del modelado. Una vez se formaliza el modelo energético, se determina la referencia energética que el control por balanceo debe conseguir [6]:

$$E_{ref} = mgl \quad (2.9)$$

Por otro lado, el error de seguimiento para calcular la acción de control u viene dado por la diferencia entre la energía actual del péndulo (E_m , Ecuación 2.8) y la energía de referencia (E_{ref} , Ecuación 2.9) [6]:

$$e = E_m - E_{ref} \quad (2.10)$$

Finalmente se define la ley de control que permite balancear el péndulo simple hasta su posición inestable, donde la acción de control u es el par aplicado en el eje de rotación [6]:

$$u = K \dot{\theta} \cos(\theta) e \quad (2.11)$$

En función del valor de la ganancia K se conseguirá el objetivo en diferente número de oscilaciones. Valores pequeños de la ganancia requieren de más iteraciones para llegar hasta el punto de estabilización.

Sin embargo, la ley de control definida en la Ecuación 2.11 no es directamente transferible al péndulo invertido sobre carro. Los motivos son los siguientes:

- **Acción de control:** En la ley de control propuesta en 2.11 el actuador se encuentra en el eje del péndulo. El sistema definido en 2.7 es subactuado y solo se tiene control sobre la fuerza de empuje del carro. Para aplicar la estrategia de energía, es necesario convertir la acción de control virtual (el par necesario en el eje) a una aceleración real del carro a través del acoplamiento dinámico del sistema.
- **Límites físicos del riel:** Se hace indispensable, por tanto, un control multiobjetivo que regule la energía del péndulo garantizando simultáneamente la estabilidad de la posición del carro dentro del espacio de trabajo del riel.

Para abordar las limitaciones mencionadas, se recurre a la técnica de **Linearización por Realimentación Parcial (PFL)**[7]. Permite transformar linealizar la dinámica de un subconjunto del sistema mediante un cambio de variables y una ley de control no lineal, siempre que exista un acoplamiento inercial suficiente entre las coordenadas actuadas y las pasivas [7].

En este trabajo se aplica la variante conocida como **linealización colocalizada** (*Collocated Linearization*), la cual consiste en linealizar el grado de libertad activo del sistema, es decir, aquel donde se ubica físicamente la acción de control [7]. El objetivo es diseñar una ley de control $u(N)$ que cancele las no linealidades de la dinámica del carro para que este se comporte, desde el punto de vista del controlador de alto nivel, como un doble integrador respecto a una aceleración deseada $\ddot{x}_{des}(\frac{m}{s^2})$.

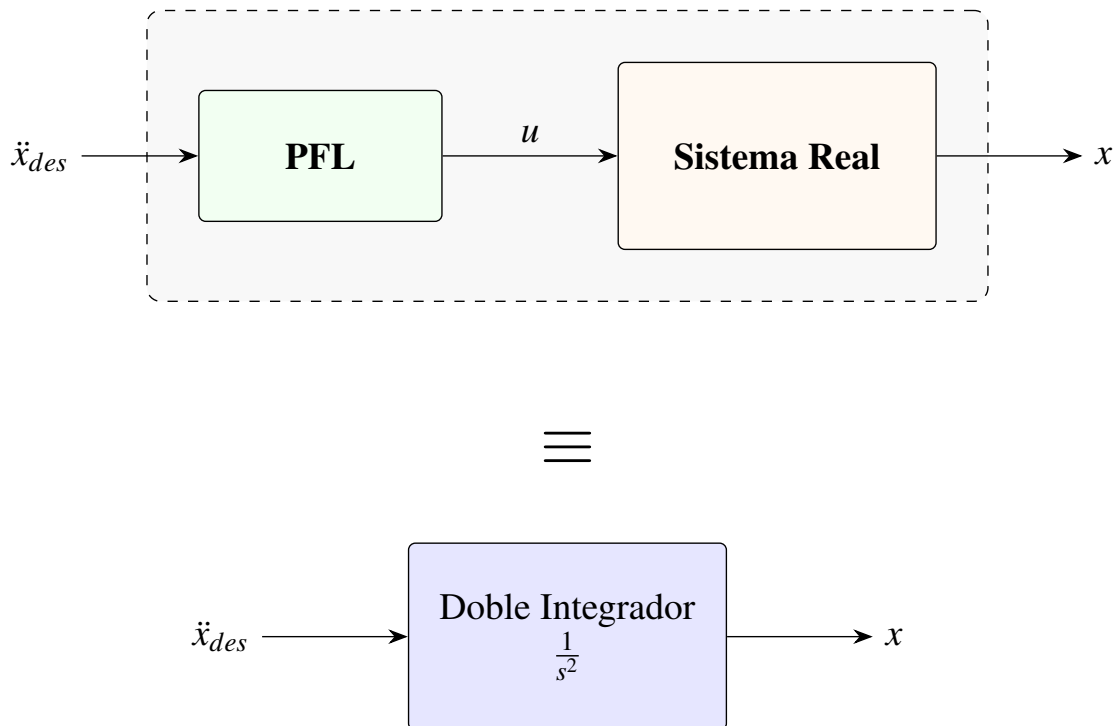


Figura 2.5: Concepto de equivalencia de la Linealización Colocalizada.

Partimos de las ecuaciones de movimiento del sistema que incluyen los términos de fricción del carro (c_x) y del péndulo (c_θ):

$$\begin{cases} (M+m)\ddot{x} + ml\ddot{\theta}\cos(\theta) + c_x\dot{x} - ml\dot{\theta}^2\sin(\theta) = u \\ ml\ddot{x}\cos(\theta) + J\ddot{\theta} + c_\theta\dot{\theta} + mgl\sin(\theta) = 0 \end{cases} \quad (2.12)$$

Donde $J = I + ml^2$ es el momento de inercia del péndulo respecto al eje de rotación. Para eliminar $\ddot{\theta}$ de la ecuación de control, se despeja dicha variable de la dinámica del péndulo [5]:

$$\ddot{\theta} = \frac{1}{J} (-mgl\sin\theta - ml\cos\theta\ddot{x} - c_\theta\dot{\theta}) \quad (2.13)$$

Al imponer la condición de linealización $\ddot{x} = \ddot{x}_{des}$ y sustituir la expresión 2.13 en la ecuación del carro, se obtiene la fuerza u necesaria para anular la dinámica no lineal del subsistema actuado [5]:

$$u = (M+m)\ddot{x}_{des} + ml\cos\theta \left[\frac{1}{J} (-mgl\sin\theta - ml\cos\theta\ddot{x}_{des} - c_\theta\dot{\theta}) \right] - ml\dot{\theta}^2\sin\theta + c_x\dot{x} \quad (2.14)$$

Como se observa, la fuerza física aplicada (u) compensa dinámicamente la masa total y los efectos inerciales del péndulo. Esta arquitectura permite definir la aceleración deseada $\ddot{x}_{des}$ como el punto de encuentro donde convergen los dos objetivos de control del sistema [5]:

$$\ddot{x}_{des} = \underbrace{K_e\dot{\theta}\cos\theta(E - E_{ref})}_{\text{Control de Energía (Swing-up)}} - \underbrace{(K_px + K_d\dot{x})}_{\text{Control de Posición (PD)}} \quad (2.15)$$

La ley de control resultante en la ecuación 2.15 simboliza un compromiso entre la inyección de energía necesaria para el balanceo y la regulación de la posición del carro sobre el riel [5]. Se debe ajustar el peso de cada ganancia para obtener un equilibrio óptimo entre ambos controles.

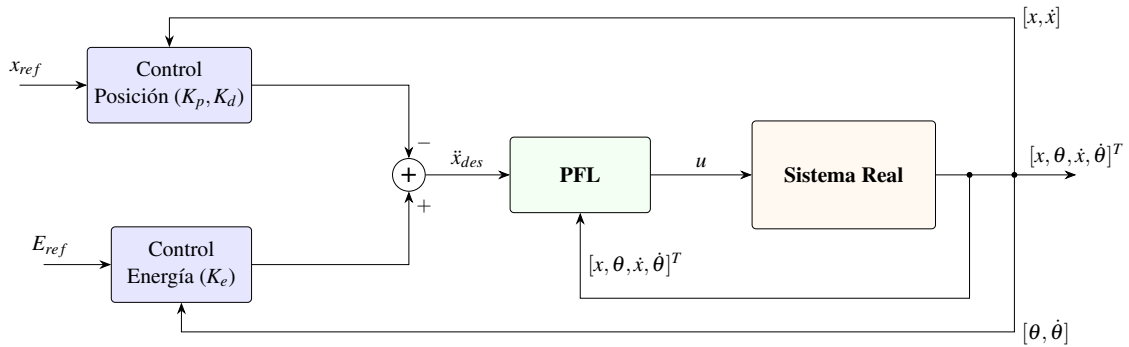


Figura 2.6: Diagrama de bloques de la estrategia de Swing-up con linealización parcial (PFL)

2.3.4 Modelado en el espacio de estados

Una vez que la estrategia de *swing-up* ha llevado al péndulo a su posición vertical inestable ($\theta \approx \pi \text{ rad}$), se requiere mantener el péndulo equilibrado en su punto superior mientras se regula la posición del carro en un punto de referencia (típicamente el centro del riel, $x = 0$). Para lograr el cometido se opta por aplicar un control por realimentación del estado. Un control de este tipo, requiere una representación en el espacio de estados del modelo dinámico linealizado en el punto de equilibrio. Todos los sistemas bajo este marco se definen como [8]:

$$\begin{cases} \dot{x}(t) = Ax(t) + Bu(t) \\ y(t) = Cx(t) + Du(t) \end{cases} \quad (2.16)$$

Donde el vector de estados $\mathbf{x}(t) = [x, \theta, \dot{x}, \dot{\theta}]^T \equiv [q, \dot{q}]^T$ define completamente la condición del sistema en cada instante. Los componentes de la representación se describen a continuación [8]:

- **Matriz de estado** ($A \in \mathbb{R}^{4 \times 4}$): Caracteriza la dinámica interna del sistema y cómo evolucionan los estados (posiciones y velocidades) entre sí cuando no hay acción de control. Sus autovalores son los polos del sistema e indican si es estable.
- **Matriz de entrada** ($B \in \mathbb{R}^{4 \times 1}$): Define cómo la fuerza aplicada al carro (u) afecta a la aceleración de cada uno de los estados.
- **Matriz de salida** ($C \in \mathbb{R}^{4 \times 4}$): Relaciona los estados internos con las variables medidas. Dado que en este proyecto se dispone de sensores para todas las variables, C se simplifica a la matriz identidad I .
- **Matriz de transmisión directa** (D): Representa el acoplamiento directo entre la entrada y la salida. Para este caso D es cero, ya que la acción de control actúa sobre la dinámica del sistema primero.

Para sistematizar la transición entre el modelo físico no lineal y el diseño del controlador de estabilidad, se parte de la ecuación 2.2. Al aplicar una linealización por Jacobiano alrededor del punto de equilibrio inestable $x = [0, \pi, 0, 0]^T$, las matrices de la dinámica se transforman en matrices de coeficientes constantes evaluadas en dicho punto [8]:

- **Inercia:** $\bar{M} = M(\bar{q})$
- **Amortiguamiento:** $\bar{C} = C(\bar{q}, 0)$
- **Rigidez (Gravedad):** $\bar{G} = \left. \frac{\partial G(q)}{\partial q} \right|_{q=\bar{q}}$
- **Entrada:** $\bar{B} = B(\bar{q})$

Para el caso en particular del péndulo invertido sobre carro, el sistema linealizado en dicho punto es el siguiente:

$$\underbrace{\begin{bmatrix} M+m & -ml \\ -ml & I+ml^2 \end{bmatrix}}_{\bar{M}} \underbrace{\begin{bmatrix} \ddot{x} \\ \ddot{\theta} \end{bmatrix}} + \underbrace{\begin{bmatrix} c_x & 0 \\ 0 & c_\theta \end{bmatrix}}_{\bar{C}} \underbrace{\begin{bmatrix} \dot{x} \\ \dot{\theta} \end{bmatrix}} + \underbrace{\begin{bmatrix} 0 \\ -mgl \end{bmatrix}}_{\bar{G}} = \underbrace{\begin{bmatrix} 1 \\ 0 \end{bmatrix}}_{\bar{B}} u \quad (2.17)$$

El sistema descrito en la ecuación 2.17 es bajo el que se diseñará el control de posición. Contra más lejos esté el péndulo de su vertical menos efectivo será el control, ya que las diferencias del

sistema real respecto del lineal empiezan a acentuarse, este es uno de los motivos por el que se hace necesaria la conmutación entre control de posición y swing-up.

Definiendo el vector de estados como $\mathbf{x} = [q, \dot{q}]^T$, el sistema puede reescribirse en la representación compacta de espacio de estados mediante la siguiente expresión [8]:

$$\begin{bmatrix} \dot{q} \\ \ddot{q} \end{bmatrix} = \underbrace{\begin{bmatrix} \mathbf{0} & I \\ -\bar{M}^{-1}\bar{G} & -\bar{M}^{-1}\bar{C} \end{bmatrix}}_A \begin{bmatrix} q \\ \dot{q} \end{bmatrix} + \underbrace{\begin{bmatrix} \mathbf{0} \\ \bar{M}^{-1}\bar{B} \end{bmatrix}}_B u \quad (2.18)$$

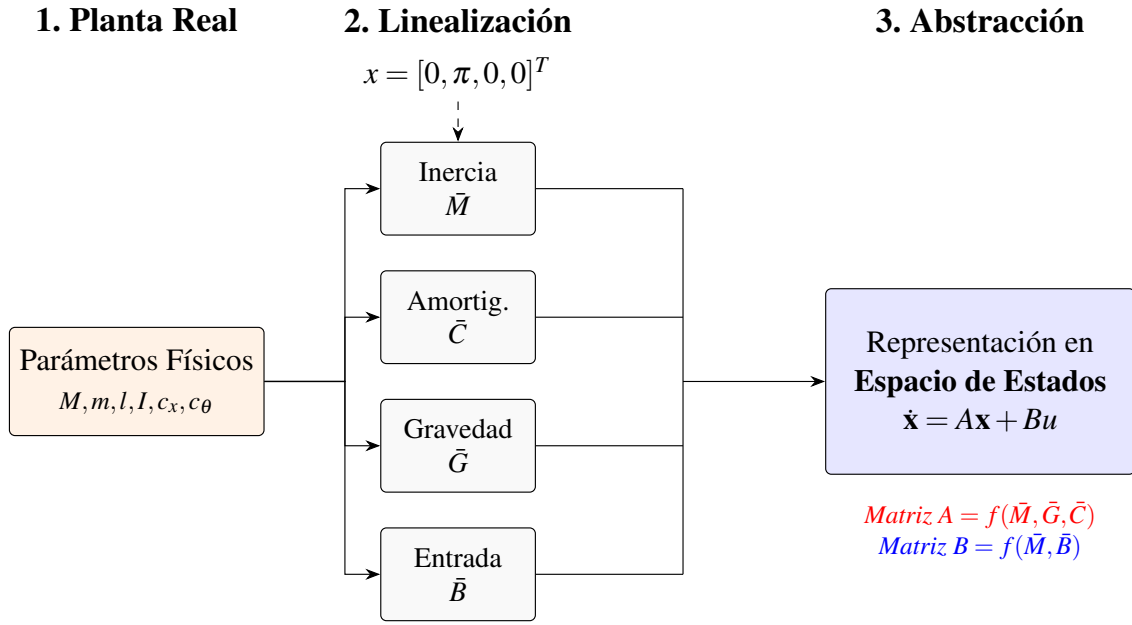


Figura 2.7: Flujo de trabajo para el modelado

Las principales ventajas de aplicar esta técnica para la obtención de la representación del sistema en el espacio de estados son:

- **Abstracción y generalización del modelado:** La técnica es escalable a cualquier sistema que cumpla la dinámica definida en la ecuación 2.2. Permite centrarse en el diseño del controlador en lugar de obtener de forma manual las matrices A y B para cada sistema concreto.
- **Eficiencia en la implementación:** La transición a un marco matricial facilita enormemente la implementación en lenguajes de programación de alto nivel, como *MATLAB* o *Python* (mediante la librería *NumPy*). El programador simplemente define las matrices ($\bar{M}$, $\bar{C}$, $\bar{G}$ y $\bar{B}$) luego el propio lenguaje obtiene en la salida la representación en espacio de estados (A y B).

2.3.5 Control por realimentación del estado

Tras definir en la anterior sección el *sistema lineal invariante (LTI)* que define la dinámica del sistema en el punto de equilibrio $x = [0, \pi, 0, 0]^T$, se requiere diseñar una ley de control que permita estabilizar el sistema en esta región.

El **control por realimentación del estado** para este caso asume que todos los estados del sistema ($\mathbf{x}$) están disponibles para su medición. Bajo esta premisa, la acción de control $u(t)$ se calcula como una combinación lineal de dichos estados multiplicados por un vector de ganancias K , sumado opcionalmente a un término de referencia $r(t)$ escalado por una ganancia K_r [9]:

$$u(t) = K_r r(t) - K \mathbf{x}(t) \quad (2.19)$$

Desglosando el vector de ganancias $K = [K_1, K_2, K_3, K_4]$, la fuerza aplicada al carro se define de forma extendida como:

$$u = K_r r - (K_1 x + K_2 \theta + K_3 \dot{x} + K_4 \dot{\theta}) \quad (2.20)$$

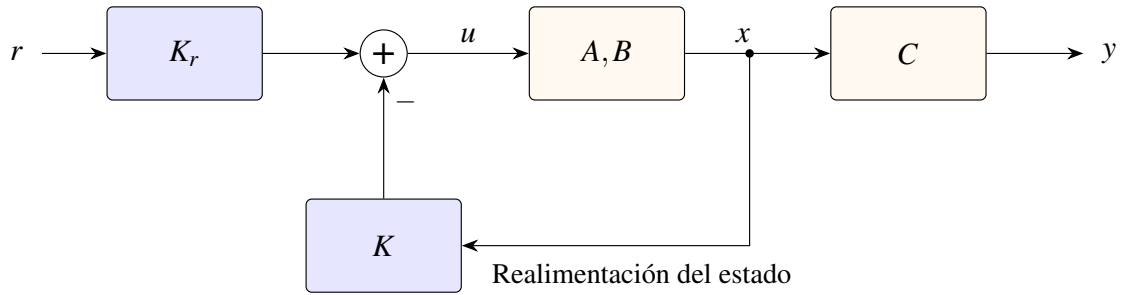


Figura 2.8: Diagrama de bloques de la ley de control por realimentación del estado

La figura 2.8 muestra como el controlador obtiene la acción de control a aplicar en cada instante de tiempo en función de los estados y la referencia. Por otro lado, en comparación con la ecuación 2.16, el sistema en bucle cerrado representado en espacio de estados es [9]:

$$\begin{cases} \dot{x}(t) = Ax(t) + B(K_r \cdot r(t) - K \cdot x(t)) = (A - BK)x(t) + BK_r r(t) \\ y(t) = Cx(t) + Du(t) \end{cases} \quad (2.21)$$

Bajo esta configuración, la dinámica del sistema en bucle cerrado queda gobernada por la expresión matricial $(A - BK)$. Por tanto, el diseño del controlador se reduce a determinar los valores del vector de ganancias $K = [k_1, k_2, k_3, k_4]$ que garanticen que todos los autovalores resultantes posean parte real negativa, dotando así al sistema de una respuesta temporal específica. Para abordar este problema, se consideran habitualmente dos metodologías:

- **Asignación de polos:** Consiste en definir arbitrariamente la ubicación de los autovalores de la matriz $(A - BK)$ en el plano complejo. Se calculan las ganancias de K para que el sistema responda con una dinámica concreta (amortiguamiento y tiempo de establecimiento) [9].

- **Regulador Cuadrático Lineal (LQR):** Esta metodología aborda el diseño como un problema de optimización. En lugar de elegir la ubicación de los polos, el diseñador define qué es más costoso: que el sistema se aleje del equilibrio o que el actuador trabaje en exceso [9]. El algoritmo calcula de forma automática la matriz de ganancias K que minimiza el siguiente índice de coste cuadrático:

$$J = \int_0^{\infty} (\mathbf{x}^T Q \mathbf{x} + u^T R u) dt \quad (2.22)$$

El comportamiento del controlador se sintoniza a través de dos matrices de ponderación:

- **Matriz Q (4×4):** Penaliza la desviación de los estados. Valores elevados en la diagonal de Q indican que se prioriza la precisión y la rapidez en la estabilización de los estados $(x, \theta, \dot{x}, \dot{\theta})$.
- **Matriz R (1×1):** Penaliza el esfuerzo de control $u(t)$. Un valor de R elevado suaviza la respuesta del actuador.

Es importante notar que la respuesta del sistema no depende de los valores absolutos de las matrices, sino del peso relativo entre ellas (Q frente a R).

2.3.6 Lógica de conmutación

Finalmente, la lógica de conmutación atiende a un umbral angular. Este bloque de decisión monitoriza de forma continua la posición angular estimada del péndulo ($\hat{\theta}$). Si el péndulo ingresa dentro de una vecindad acotada en torno a la vertical absoluta (*el punto de linealización del sistema*), la lógica conmuta al control por realimentación del estado. En caso contrario, el sistema mantiene activo el controlador de swing-up para elevar el mecanismo. Este comportamiento híbrido se formaliza en el siguiente pseudocódigo:

Algoritmo 1 Lógica de Conmutación

Require: Vector de estados estimado $\hat{x} = [\hat{x}_c, \hat{x}_c, \hat{\theta}, \hat{\theta}]^T$, umbral angular θ_{th}

Ensure: Señal de control u a aplicar a la planta

```

1: loop
2:    $\hat{\theta} \leftarrow \hat{x}(3)$                                 ▷ Obtener posición angular actual
3:   if  $|\hat{\theta}| \leq \theta_{th}$  then
4:      $u \leftarrow -K\hat{x}$                                 ▷ Control de posición
5:   else
6:      $u \leftarrow f_{swing}(\hat{x})$                         ▷ Control de energía
7:   end if
8:
9: end loop

```

2.4 Arquitectura del sistema

Para definir la arquitectura del sistema se ha hecho una división entre la parte de simulación y la implementación en un sistema real.

La arquitectura del sistema de simulación permite obtener una vista general del flujo de trabajo para el modelado y la validación antes de la implementación real. Por otro lado, la arquitectura del sistema real se centra en los periféricos involucrados en el control así como el proceso de discretización que se debe aplicar para controlar un sistema real utilizando un microcontrolador.

2.4.1 Arquitectura de la simulación

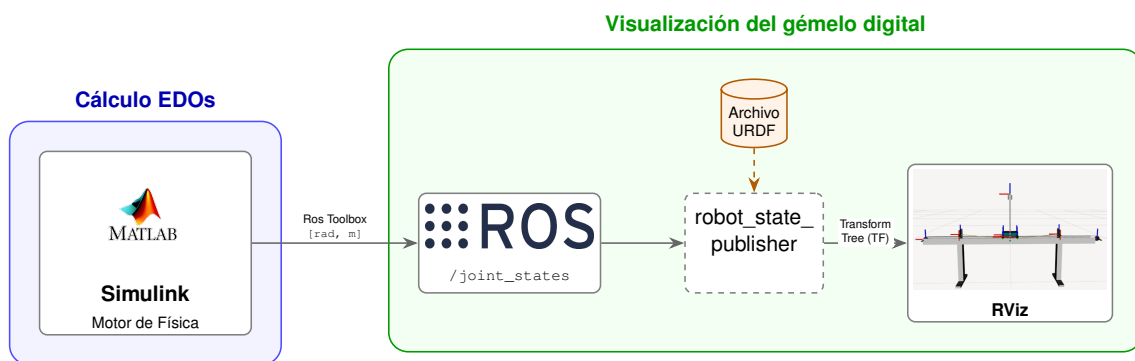


Figura 2.9: Arquitectura de la simulación

La Figura 2.9 detalla la arquitectura de software diseñada para la simulación del sistema. A continuación se desglosa cada elemento del esquema para conocer su función y como se conecta con el resto de la arquitectura:

- **Cálculo de EDOs (Motor de Física):** Implementado en el entorno MATLAB/Simulink, este bloque se encarga de resolver las Ecuaciones Diferenciales Ordinarias (EDO) que describen la dinámica del sistema. Simulink actúa como el motor de física, calculando en cada paso de tiempo los estados del robot (posiciones en radianes y metros). Estos datos se transmiten al ecosistema ROS mediante ROS Toolbox, publicando la información en el tópico `/joint_states`.
- **Visualización, gemelo digital:** Este bloque recibe la información cinemática para representar el comportamiento del sistema en tiempo real. Está totalmente integrado en el entorno de ROS y se compone de varias piezas:
 - **Archivo URDF (Unified Robot Description Format):** Es un archivo XML que describe como es el robot o mecanismo que se quiere simular. Se especifican las visuales, las restricciones cinemáticas y los marcos de referencia de las articulaciones.
 - **Robot State Publisher:** Es el nodo de ROS que permite representar el movimiento del robot. Se suscribe al tópico `/joint_states` y actualiza el árbol de transformadas de nuestro archivo URDF.
 - **RViz:** La última pieza de la arquitectura software, es la interfaz que permite visualizar de forma gráfica e intuitiva todo el flujo de datos como un modelo 3D en movimiento.

2.4.2 Arquitectura del sistema real

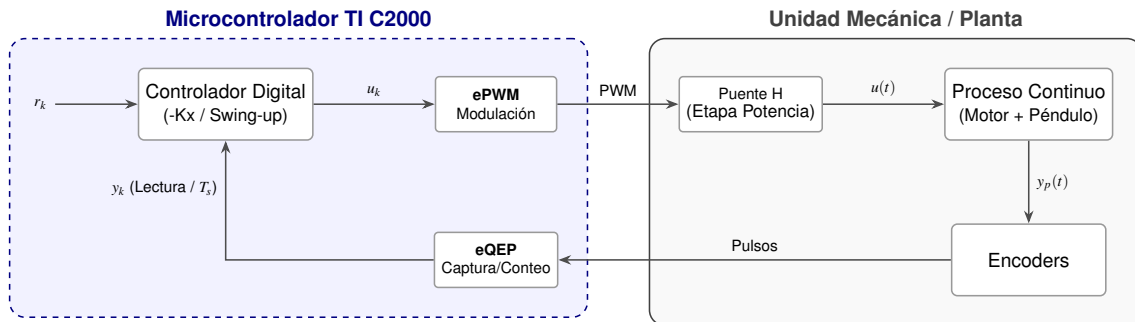


Figura 2.10: Arquitectura del sistema real

La Figura 2.10 presenta de forma esquemática la implementación del bucle de control en el microcontrolador.

Esta estructura permite observar la interacción entre el entorno digital (discreto) y el proceso físico (continuo). El núcleo del sistema es el **Controlador Digital**. Este procesador se encarga de ejecutar el algoritmo de control que procesa la señal de error entre la referencia (r_k) y la realimentación (y_k).

El concepto fundamental es el **periodo de muestreo** (T). Este parámetro determina la periodicidad de la rutina de control, permitiendo sincronizar las etapas de medida, cálculo y actuación sobre la planta de manera precisa. Durante el desarrollo del proyecto se determinará el periodo de muestreo atendiendo a factores como la dinámica del sistema en bucle cerrado o la frecuencia de la señal PWM.

Finalmente la interacción entre el mundo digital y el mundo físico es posible mediante el uso de **periféricos** que permiten medir el estado del sistema o modificarlo. Diferenciamos en dos clases de periféricos:

- **Actuadores:** La señal de control digital calculada (u_k) se traduce en señales lógicas de dirección y una señal modulada por ancho de pulso (**PWM**) generada por el periférico **ePWM**. Tanto el estado de las señales digitales como el ciclo de trabajo (*duty cycle*) del PWM se mantienen constantes durante el intervalo de muestreo (T_s). Estas señales excitan un puente en H (driver), el cual conmuta el voltaje de alimentación para inyectar una corriente en el motor. De este modo, se transforma la decisión del algoritmo en el par mecánico necesario para accionar el proceso continuo.
- **Sensores:** La respuesta física del sistema ($y_p(t)$) es captada por encoders ópticos incrementales. A diferencia de un sensor analógico convencional, estos dispositivos generan una secuencia de pulsos basada en eventos de movimiento (detección de flancos). El periférico **eQEP** del microcontrolador actúa como una unidad de conteo especializada que registra estos eventos de forma asíncrona. No obstante, para el bucle de control, la señal se digitaliza de forma efectiva (y_k) al leer el estado de dichos contadores en intervalos de tiempo estrictamente periódicos (T_s). Este muestreo periódico es fundamental para el cálculo de variables derivadas, como la velocidad, mediante la diferencia de posición en cada intervalo.

2.4.3 Especificación de componentes de hardware

Para la implementación física del sistema de control diseñado, se han seleccionado componentes que garantizan un compromiso óptimo entre precisión de lectura, capacidad de procesamiento y tiempo de respuesta. En la Tabla 2.1 se resumen los elementos principales del prototipo.

Cuadro 2.1: Resumen de componentes del sistema de hardware.

Componente	Modelo	Función Principal
Microcontrolador	TMS320F280049C	Procesamiento de señales y ejecución del control
Driver de motor	AQMH2407ND	Interfaz de potencia (Puente H con aislamiento)
Encoder Motor	HEDS-5140-I (2048 PPR)	Medición de posición lineal (x)
Encoder Péndulo	HEDS-5140-A06 (2000 PPR)	Medición de ángulo vertical (θ)
Actuador	Crouzet 82 760 0 (24V)	Generación de fuerza sobre el carro (u)

Unidad de Procesamiento (Microcontrolador)

Para la implementación del algoritmo de control se ha seleccionado el **TMS320F280049C**, un microcontrolador de tiempo real de la familia **C2000** de **Texas Instruments**. Este dispositivo está específicamente diseñado para aplicaciones de control, destacando su núcleo **C28x** a **100 MHz** que integra una unidad de punto flotante (**FPU**) y una unidad de aceleración trigonométrica (**TMU**) [10]. Ambos aceleradores son fundamentales para procesar la aritmética del controlador y las funciones trigonométricas del modelo.

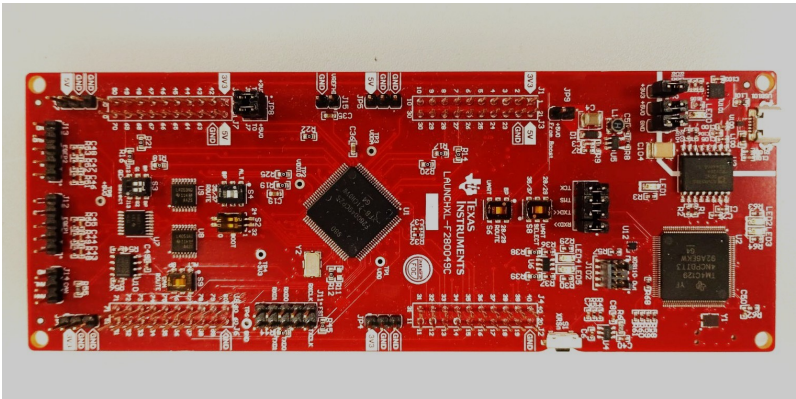


Figura 2.11: TMS320F280049C en formato *Launchpad*

Asimismo, el chip dispone de periféricos de control avanzado que descargan de tareas críticas a la CPU, los más destacados en el proyecto son:

- **eQEP (Enhanced Quadrature Encoder Pulse):** Dos módulos dedicados para la decodificación de señales en cuadratura de los encoders, permitiendo una lectura de posición y ángulo sin consumo de ciclos de reloj [10].
- **ePWM (Enhanced Pulse Width Modulator):** El periférico permite una resolución de **16 bits** en la generación del ciclo de trabajo (*duty cycle*) y hasta **32 bits** si se utiliza *High Resolution PWM*, garantizando un control fino sobre el motor DC [10].

El desarrollo software se apoya en el ecosistema oficial de Texas Instruments. Como entorno de desarrollo integrado (IDE) se emplea **Code Composer Studio (CCS)** [11], permite una depuración avanzada y optimización específica para la arquitectura C28x. Además, se utiliza la **SDK C2000Ware**, que proporciona una capa de abstracción de hardware (HAL) facilitando la programación y la interpretabilidad del código [12].

Etapas de Potencia y Actuación

La acción de control $u(t)$ se traduce en una tensión aplicada al motor mediante el driver de motores de corriente continua **AQMH2407ND**. Implementa un **Puente H** de doble canal basado en tecnología **MOSFET** [13], lo que le otorga una eficiencia energética superior y una disipación térmica significativamente menor en comparación con drivers basados en transistores **BJT**, como el **L298N** [14].

El dispositivo es capaz de suministrar una corriente continua de hasta **7 A** por canal (con picos de hasta 50 A) [13]. El control se gestiona mediante señales PWM provenientes del microcontrolador, permitiendo la regulación de la tensión media aplicada al motor.

Un aspecto fundamental en el diseño de este driver es la integración de **optoacopladores EL357N-G** [15], los cuales proporcionan un **aislamiento galvánico** entre la etapa de control y la de potencia. Esta característica es esencial en la implementación de sistemas de control, ya que ofrece:

- **Mitigación de ruido:** Evita que el ruido electromagnético inducido por la conmutación de las bobinas del motor afecte a las señales de medición.
- **Protección del hardware:** Asegura la integridad del microcontrolador frente a posibles picos de tensión o corrientes de retorno (*back-EMF*) provenientes de la etapa de potencia.

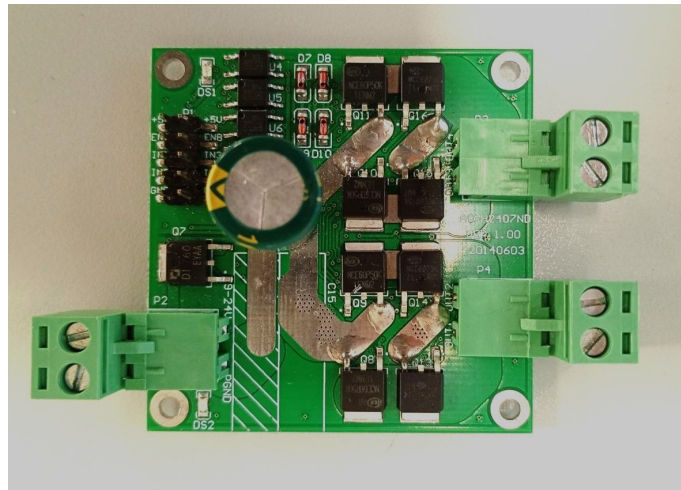


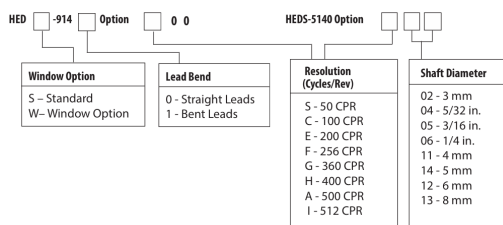
Figura 2.12: Driver AQMH2407ND

Sensores (Encoders)

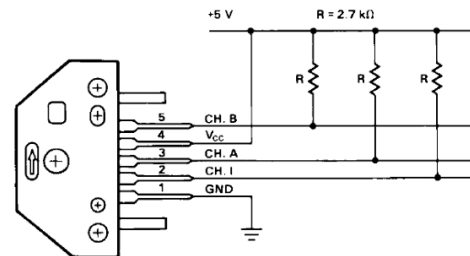
Para el caso concreto de péndulo invertido sobre carro, se requieren dos encoders (uno por grado de libertad), estos sensores son vitales para obtener el estado del sistema en cada instante de tiempo y calcular una acción de control acorde.

Concretamente, se ha utilizado un sistema de detección óptica que se compone de un módulo detector de la serie **HEDS-9140** y un disco óptico de la serie **HEDS-5140**. A continuación se describe la ubicación y resolución de los dos encoders del sistema:

- **Posición del carro (x):** Se utiliza un encoder acoplado al eje del motor de continua que proporciona una resolución de **2048 pulsos por revolución (PPR)** tras el procesado en cuadratura.
- **Ángulo del péndulo (θ):** Acoplado en el eje del péndulo, ofrece 500 ciclos por revolución (CPR), traduciendo en una resolución de **2000 PPR** en cuadratura.



(a) Especificaciones módulo de detección y disco óptico según modelo específico



(b) Descripción del módulo detector y mapeo de señales

Figura 2.13: Detalles técnicos de los componentes de detección óptica de la serie HEDS [16]

La Figura 2.13a permite identificar con precisión la configuración de cada sensor atendiendo a su resolución y dimensiones mecánicas:

- **Encoder del motor:** Para alcanzar los 2048 PPR en cuadratura, se utiliza un disco óptico de 512 CPR. Siguiendo la codificación de la figura, este corresponde al modelo **HEDS-5140-I**.
- **Encoder del péndulo:** Este eje utiliza el modelo **HEDS-5140-A06**. El código **A** identifica la resolución de 500 CPR (2000 PPR en cuadratura), mientras que el sufijo **06** indica un diámetro interno del disco, garantizando un ajuste mecánico con el eje sin holguras.

Por último, el módulo detector **HEDS-9140** (Figura 2.13b) es el encargado de transformar el paso de las ranuras del disco en señales TTL. La disposición de sus fotodetectores internos permite que el periférico **eQEP** del microcontrolador interprete el desfase entre los canales A y B para determinar la dirección del movimiento.

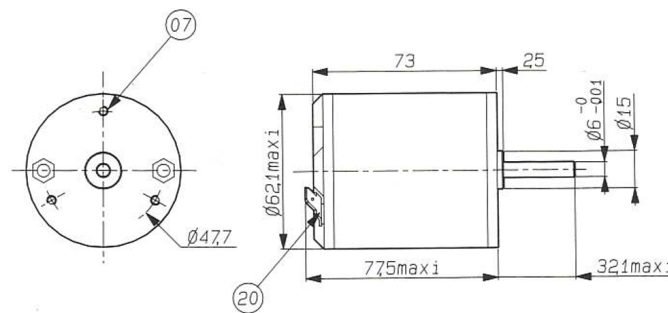
Actuador

El movimiento del carro se realiza mediante un motor de corriente continua de imanes permanentes del fabricante **Crouzet**, modelo **82 760 0**.

Cuadro 2.2: Parámetros técnicos del motor Crouzet 82 760 0 (24V) [17]

Parámetro	Valor	Unidad
Velocidad nominal	2300	rpm
Par nominal	76	mN·m
Potencia útil nominal	18	W
Corriente nominal	1,3	A
Resistencia (R)	5,5	Ω
Inductancia (L)	7,2	mH
Constante de par (K_t)	0,068	N·m/A
Inercia (J_m)	270	g·cm ²
Eficiencia (η)	57	%

Los datos del fabricante expuestos en la tabla 2.2 son vitales para caracterizar actuador y estimar la relación entre el voltaje aplicado en sus bornes y la fuerza horizontal que empuja el carro.



(a) Dimensiones del actuador [17]



(b) Motor instalado en la unidad mecánica del laboratorio

Figura 2.14: Detalles del actuador Crouzet 82 760 0: diseño técnico y montaje real.

El sistema utiliza una transmisión por correa dentada (Figura 2.14b) para convertir el movimiento rotativo del motor en el desplazamiento lineal del carro, minimizando las holguras mecánicas.

3. Desarrollo

Este capítulo constituye el núcleo de este trabajo, abarcando desde la validación en entornos virtuales hasta el despliegue definitivo de las leyes de control sobre la plataforma física. Adoptando el paradigma del *Diseño Basado en Modelos* (MBD), el desarrollo se organiza en un flujo de trabajo que unifica la simulación numérica, la programación a bajo nivel del hardware y la identificación experimental de la dinámica real del sistema.

Para guiar el desarrollo de forma estructurada, el contenido de este capítulo se organiza en cinco bloques fundamentales:

- **Trabajo de simulación:** Modelado matemático inicial en *MATLAB/Simulink* y diseño de las estrategias de control (posición, algoritmo de levantamiento o *swing-up* y lógica de conmutación). Asimismo, se detalla la integración de *ROS* dentro de la simulación.
- **Programación del microcontrolador:** Configuración de bajo nivel y arquitectura del firmware sobre el microcontrolador *TMS320F280049C*. Se detalla la puesta en marcha de los módulos periféricos esenciales: *eQEP* para la lectura encoders, *ePWM* para la actuación del motor, y el periférico *SCI* para el envío de la telemetría por el puerto serie.
- **Caracterización y compensación de la zona muerta:** Estudio experimental enfocado en conocer los efectos no lineales de la fricción estática del motor.
- **Identificación paramétrica experimental:** Metodología aplicada para aislar y calcular las constantes físicas reales de la planta de laboratorio y poder ajustar las simulaciones.
- **Implementación del algoritmo de control:** Integración y despliegue final de las leyes de control en el microcontrolador

Por último, cabe destacar que, si bien a lo largo de este documento se incorporan bloques de código específicos para describir detalles de implementación, el grueso del desarrollo se encuentra completamente disponible en el **repositorio oficial del proyecto** [18].

3.1 Trabajo de simulación

3.1.1 Modelado del sistema en Simulink

El punto de partida consistió en trasladar el modelo representado en la Ecuación 2.7 hacia un entorno de simulación numérica en *MATLAB/Simulink*. Para una primera aproximación, se emplearon los parámetros de masas, longitudes, inercias y rozamientos proporcionados en la documentación original del fabricante, los cuales se detallan en la Tabla 3.1. No obstante, debido a que la unidad mecánica cuenta con más de 25 años de antigüedad, posteriormente se llevó a cabo la identificación experimental de sus parámetros reales.

Cuadro 3.1: Parámetros proporcionados por el fabricante [19]

Parámetro	Símbolo	Valor	Unidades
Masa del carro	M	1.12	kg
Masa del péndulo	m	0.12	kg
Distancia al centro de masas	l	0.305	m
Inercia del péndulo	J_p	0.0136	$\text{kg} \cdot \text{m}^2$
Fricción del carro	c_x	0.05	$\text{N} \cdot \text{s}/\text{m}$
Fricción del péndulo	c_θ	0.0005	$\text{N} \cdot \text{m} \cdot \text{s}/\text{rad}$

Este enfoque metodológico responde firmemente al paradigma del *Diseño Basado en Modelos* (MBD). Disponer de un modelo matemático bien calibrado para asegurar una transición eficiente entre la teoría y la práctica. Al emular con precisión el comportamiento de la planta real, el entorno virtual permite verificar de manera exhaustiva la estabilidad y robustez de los algoritmos diseñados antes de su implementación en el hardware, mitigando por completo el riesgo de daños mecánicos o eléctricos por pérdidas de control.

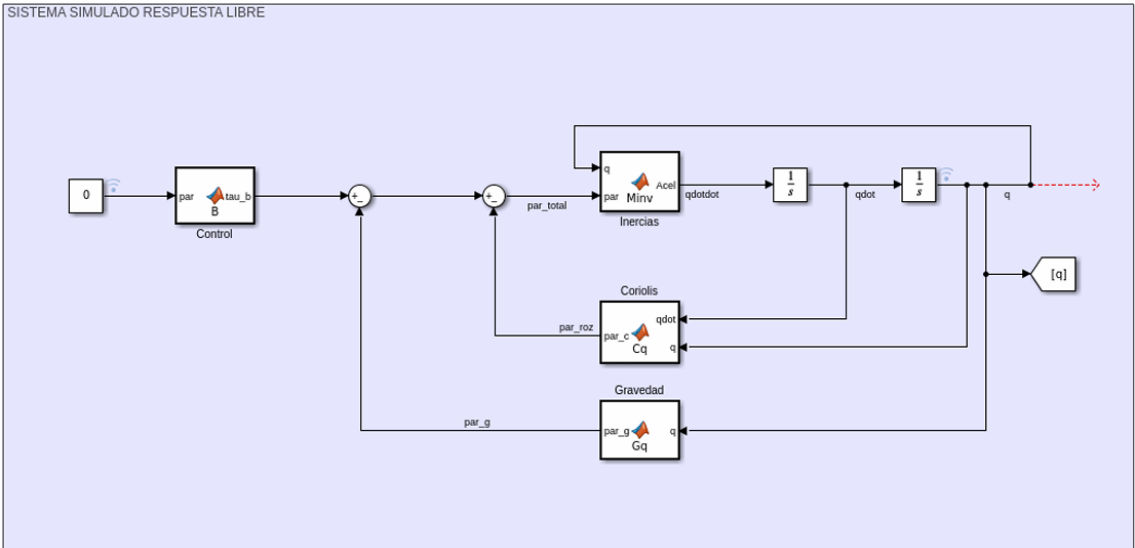


Figura 3.1: Diagrama de bloques del modelo dinámico implementado en *Simulink*

La Figura 3.1 detalla la arquitectura implementada en *Simulink*, la cual constituye la traslación directa de las ecuaciones dinámicas del sistema al entorno computacional, en el Anexo B.1 se detalla el contenido de cada bloque. Esta estructura materializa de forma práctica el álgebra de bloques conceptual introducida previamente en la Figura 2.3, sirviendo como la plataforma sobre la cual se ejecutan los algoritmos de control por realimentación del estado y por energía.

3.1.2 Algoritmo de control en Simulink

Control de posición

Una vez modelada la dinámica del sistema de péndulo invertido sobre carro e implementada en *Simulink*, el siguiente paso fue implementar un control de posición por realimentación del estado para mantener el sistema estable con el péndulo invertido ($\theta = \pi$ rad).

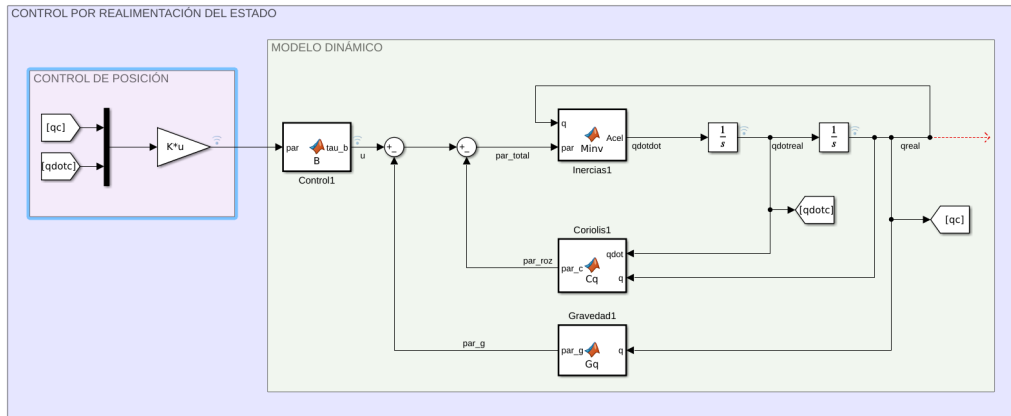


Figura 3.2: Diagrama de bloques con control de posición por realimentación del estado

Como se ilustra en la Figura 3.2, la arquitectura implementada establece una dinámica en bucle cerrado donde el bloque del controlador procesa de forma continua los vectores de estado medidos (q y $\dot{q}$). A partir de esta información, el algoritmo computa la acción de control óptima, denotada como la fuerza de empuje u . El fundamento matemático detrás de esta técnica fue abordado en el desarrollo teórico de la Sección 2.3.5.

La ley de control se reduce conceptualmente a una combinación lineal de los estados multiplicados por un vector de ganancias K , expresado formalmente como $u = K(\text{ref} - x)$.

Consecuentemente, como se aprecia en el detalle del diagrama de bloques, esta ley de control se materializa en *Simulink* mediante un único bloque de ganancia matricial (denotado en la simulación como $K*u$).

Para concluir, la obtención numérica de los vectores de ganancia K dentro del entorno de *MATLAB*, se optó por las dos metodologías clásicas de control propuestas en la Sección 2.3.5:

- **Regulador Lineal Cuadrático (LQR):** se definieron las matrices de ponderación Q y R para sintonizar el comando `lqr()` [20].
- **Asignación de polos:** Se fija la dinámica en bucle cerrado del sistema con el comando `place()` [21].

Comando `lqr()` y comando `place()` para la obtención de K

```
%% OBTENCION DE K = [K1, K2, K3 K4]
% 1. Regulador Lineal Cuadrático (LQR)
Q = diag([1, 1, 1, 1]);
R = 0.1;
[K_lqr, S, E] = lqr(A, B, Q, R);

% 2. Asignación de Polos (Pole Placement)
polos = [-2.5, -3, -3.5, -4];
K_plc = place(A, B, polos);
```

Levantamiento de péndulo

El siguiente hito en el trabajo, fue conseguir levantar el péndulo con la técnica de *Swing-Up* abordada de forma teórica en la Sección 2.3.3. Para conseguirlo se utilizó la misma metodología que para el control de posición del péndulo, se tiene un conjunto de bloques que representan la dinámica del sistema y un bloque para el controlador.

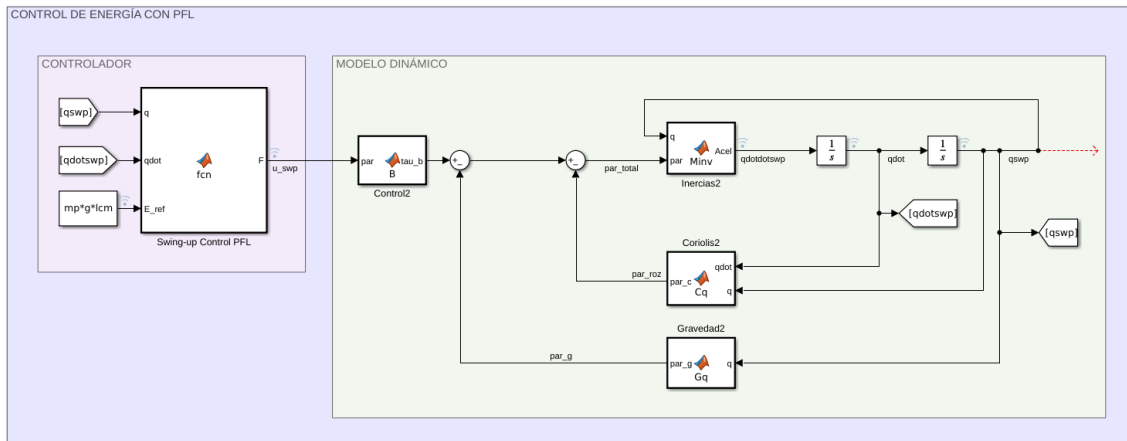


Figura 3.3: Diagrama de bloques con control de energía para el levantamiento del péndulo

Atendiendo a la Figura 3.3 el bloque de simulación tiene dos subconjuntos claramente diferenciados:

- **Control de energía (Zona Izquierda):** Se utiliza una *MATLAB Function* para integrar de forma compacta la técnica de Swing-Up con PFL, tiene como entradas los estados del sistema ($q, \dot{q}$) y la energía de referencia (E_{ref}), como salida proporciona una acción de control (u) en forma de fuerza de empuje horizontal. Para conocer la implementación del controlador en más detalle consulte el Anexo B.2.
- **Modelo Dinámico (Zona Derecha):** Representa la planta del sistema con los parámetros que la caracterizan. Los detalles de la implementación y el bloque de código de cada función se abordan en la Sección 3.1.1 y el Anexo B.1.

Conmutación entre controladores

Una vez implementados ambos controladores por separado, para finalizar con la implementación en *Simulink* se aborda la lógica de conmutación explicada en la Sección 2.3.6.

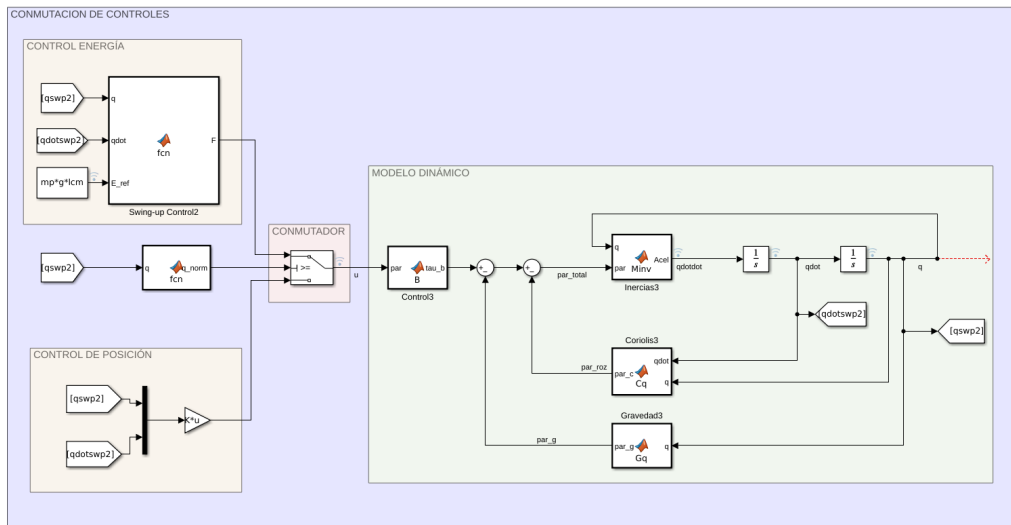


Figura 3.4: Diagrama de bloques conmutación entre controles

La arquitectura implementada en *Simulink* (Figura 3.4) representa el hito central del desarrollo de la simulación. Su diseño permite emular el comportamiento físico del péndulo e integrar de forma segura las estrategias de control antes de su transferencia al hardware real. Para facilitar su comprensión, el diagrama se divide en tres bloques funcionales:

- **Modelo Dinámico de la Planta (Zona Derecha):** Tal y como se comentó durante el capítulo representa el comportamiento mecánico del sistema.
- **Algoritmos de Control (Zona Izquierda):** Resuelven el desafío de control mediante dos estrategias diferenciadas según el estado del péndulo:
 - **Control de Energía (Swing-up):** Actúa cuando el péndulo está hacia abajo, balanceándolo de un lado a otro para inyectarle la energía necesaria para elevarlo.
 - **Control de Posición:** Actúa mediante realimentación de estados para mantener el péndulo en equilibrio estricto una vez ha alcanzado la posición vertical inestable.
- **Lógica de Conmutación (Zona Central):** Es el componente que resuelve la transición automática entre ambos controladores. El bloque q_{norm} normaliza el ángulo del péndulo entre $[-\pi, \pi]$; si este se encuentra lejos de la vertical, el interruptor (Switch) selecciona el control de energía. En el instante en que el péndulo entra en la zona de equilibrio (supera cierto umbral angular), el dispositivo conmuta automáticamente al control de posición.

Tras la integración de la lógica de conmutación de ambos controles, la parte de cálculo numérico para las *EDOs* queda resuelta, se pueden ajustar controladores y observar el comportamiento del sistema únicamente con esta implementación. No obstante, atendiendo a la Figura 2.9 sobre la *Arquitectura de la Simulación* falta implementar la conexión con el entorno de *ROS* para la visualización del gemelo digital en *RViz*.

3.1.3 Comunicación con ROS y Visualización del Gemelo Digital

Una vez resuelto el cálculo dinámico y el control en Simulink, el siguiente hito fundamental consistió en establecer la infraestructura de comunicación con el ecosistema de ROS. Esta integración permite materializar el concepto de gemelo digital, facilitando la visualización tridimensional del sistema en tiempo real.

Para articular esta conexión se empleó la herramienta *ROS Toolbox*, estructurando el flujo de información bajo el paradigma de publicador/suscriptor:

- **Lógica de Publicación (Nodo Publicador):** Como se ilustra en la Figura 3.5, el nodo desarrollado en Simulink toma de forma continua el vector con las posiciones x y θ ($[q]$) y el tiempo de simulación para estructurar el envío de datos mediante un bloque de conversión (MATLAB to ROS). Esta información se encapsula en la estructura de un mensaje dedicado a articulaciones (`sensor_msgs/JointState`) y el bloque Publish la difunde activamente en la red bajo el tópico `/joint_states`.

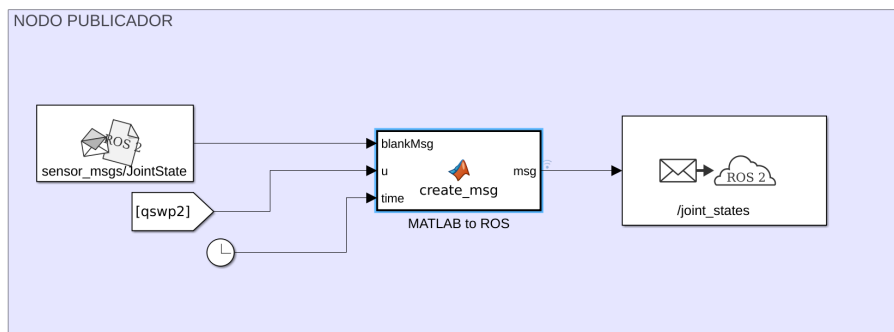


Figura 3.5: Nodo publicador en Simulink

- **Modelado y Visualización en RViz:** Se diseñó un archivo *URDF* que define la geometría y las articulaciones mecánicas del carro y el péndulo. La herramienta de visualización *RViz* se suscribe al tópico generado por Simulink, traduciendo instantáneamente cada mensaje numérico recibido en movimiento visual sobre el modelo 3D.

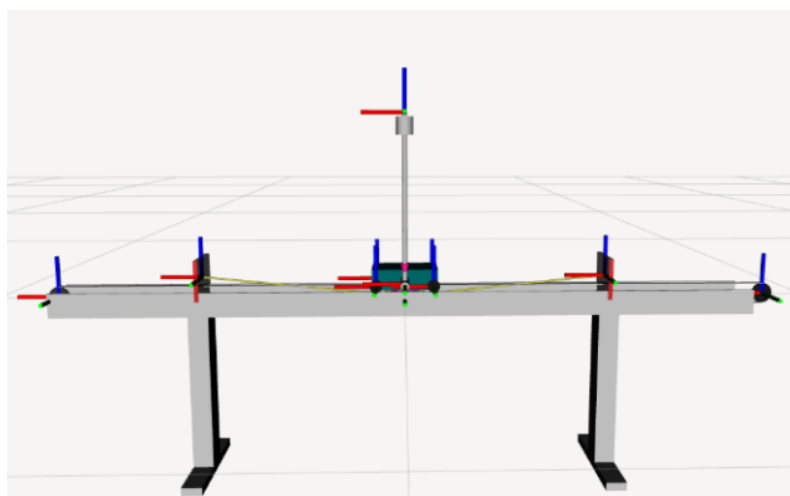


Figura 3.6: Visualización del modelo 3D en RViz

Modularidad del sistema y desviaciones en la planificación

Hubo una desviación respecto de la planificación inicial dada la modularidad y el alto nivel de abstracción en el ecosistema de ROS 2. Como extensión al entorno de simulación, se implementó una infraestructura paralela orientada a la monitorización de la planta física real en el mismo entorno tridimensional.

Para lograrlo, se desarrolló un nodo que actúa como intermediario entre el hardware (microcontrolador) y el ecosistema de ROS 2. El nodo, desde el punto de vista de la comunicación, realiza las mismas funciones que el implementado en Simulink:

- **Obtención de los datos:** Lectura de la telemetría en formato (CSV) que el microcontrolador transmite a través del puerto serie (SCI/UART) a 115200 bps.
- **Encapsulado en mensajes:** El nodo procesa la cadena de caracteres, extrae las posiciones cinemáticas reales (x y θ) y las encapsula exactamente en la misma estructura de mensaje estándar `sensor_msgs/JointState`.

Al inyectar estos datos en el mismo tópico (`/joint_states`), la herramienta *RViz* responde de manera idéntica sin requerir ninguna modificación interna. Esta característica dota al proyecto de una gran versatilidad, permitiendo conmutar de forma transparente entre la simulación y la monitorización del comportamiento físico real.

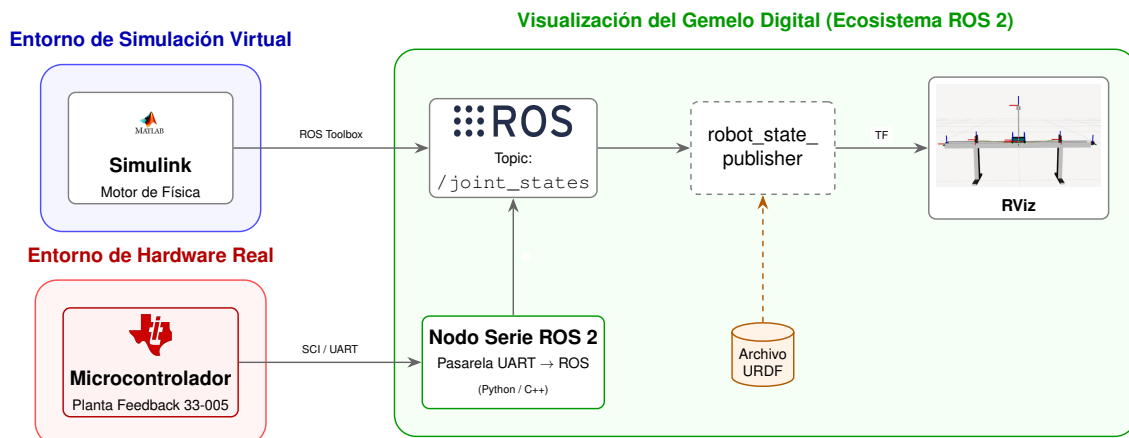


Figura 3.7: Representación de la idea de modularidad y abstracción en el ecosistema de ROS 2

Problemas derivados

Durante la integración de esta arquitectura, surgieron dos problemas técnicos que requirieron desviaciones de la configuración por defecto para estabilizar la plataforma de simulación:

- **Incompatibilidad en el Middleware (DDS):** Inicialmente se presentaron cortes intermitentes en la comunicación y pérdida de paquetes entre plataformas. El problema se resolvió modificando el estándar de comunicación de red hacia *Cyclone DDS*. Suele ser un problema muy común en la distribución *Humble* que se empleó para el proyecto.
- **Sincronización del Solver de Simulink:** El motor de cálculo numérico de Simulink tendía a desfasarse de la frecuencia de actualización que exige ROS para la publicación de sus tópicos. Para solucionarlo, se configuró el *solver* en un paso de tiempo fijo discreto.

La configuración de *Simulink* así como la conversión de un vector numérico en un mensaje estándar de ROS, se encuentra en el Anexo B.3. Por otro lado, la estructura completa del paquete de ROS 2 y sus *launch files* se encuentran detallados en el `README.md` del **repositorio oficial del proyecto** [18].

3.1.4 Simulaciones Recursivas

Para finalizar el bloque de simulación, se desarrolló una arquitectura alternativa basada en la programación estructurada de un bucle iterativo, prescindiendo de dependencias externas de entornos de simulación comerciales. El desarrollo de este entorno de simulación "en código" responde a dos objetivos fundamentales:

- **Portabilidad:** El código fuente es reproducible y ejecutable en cualquier plataforma. Aunque se implementó inicialmente en MATLAB para mantener la integración con el resto del proyecto, esta estructura facilita su futura migración a nodos de ROS en lenguajes como C++ o Python.
- **Transición natural hacia la implementación real:** La estructura recursiva del código emula fielmente el bucle de control de un microcontrolador real (que trabaja a intervalos de tiempo fijos), simplificando el paso del software al hardware definitivo.

Para lograr esto de forma autónoma, se sustituyó el motor de física comercial por un algoritmo de integración numérica elemental (método de Euler de primer orden), los valores de los estados ($x_k = [x, \theta, \dot{x}, \dot{q}]^T$) en la siguiente iteración del bucle vienen dados por la expresión recursiva:

$$\mathbf{x}_{k+1} = \mathbf{x}_k + T_s \cdot f(\mathbf{x}_k, u_k) \quad (3.1)$$

Esta ecuación gobierna el comportamiento del simulador en cada paso de tiempo y se compone de los siguientes elementos:

- $\mathbf{x}_{k+1}$: Representa el **estado futuro** del sistema (posiciones y velocidades que tendrá el mecanismo en el siguiente instante).
- $\mathbf{x}_k$: Es el **estado actual** conocido en la iteración presente.
- T_s : Es el **periodo de muestreo**, es decir, el pequeño intervalo de tiempo que transcurre entre cada iteración del bucle.
- $f(\mathbf{x}_k, u_k)$: Es la función que describe la **dinámica física del sistema**. Combina el estado actual $\mathbf{x}_k$ con la acción de control en ese momento (u_k) para devolver las velocidades y aceleraciones instantáneas del mecanismo.

Mientras que las velocidades se obtienen de forma directa, el cálculo de las aceleraciones dentro de la función $f(\mathbf{x}_k, u_k)$ requiere resolver algebraicamente las ecuaciones que definen la dinámica del sistema abordadas en la Sección 2.3.2. El desarrollo matemático completo para resolver este sistema dinámico y su traducción algorítmica al código de simulación se detallan en el Apéndice B.4. Asimismo, el entorno de simulación completo junto con diversos ejemplos de funcionamiento —tales como el control de posición, la maniobra de levantamiento o *swing-up* y la lógica de conmutación entre controladores— se encuentra disponible para su ejecución en el **repositorio oficial del proyecto** [18].

Problemas Encontrados y Compromiso del Periodo de Muestreo

El principal reto técnico durante el desarrollo de este entorno simulado fue la aparición de **inestabilidades numéricas artificiales**. El éxito de la simulación depende críticamente de la elección del periodo de muestreo (T_s).

Debido a que el método de Euler asume de forma simplificada que las aceleraciones del sistema se mantienen constantes durante todo el intervalo T_s , se detectó que valores de tiempo excesivamente grandes entre iteraciones acumulaban un error de truncamiento que no correspondía con la realidad física.

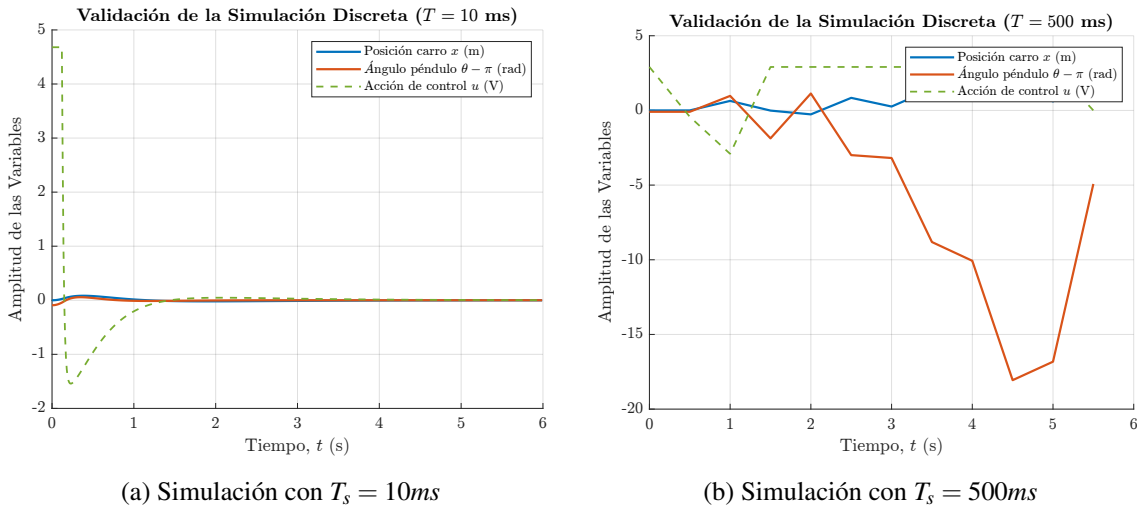


Figura 3.8: Dependencia de T_s en la simulación con expresiones recursivas

Para concluir, la Figura 3.8 ilustra el impacto crítico del periodo de muestreo al aplicar el mismo control de posición por realimentación del estado. En ella se observa cómo un intervalo de 10 ms produce una respuesta estable y fiel al comportamiento real del sistema, mientras que elevar dicho periodo a 500 ms degrada por completo la convergencia numérica, provocando una respuesta oscilatoria e inestable que distorsiona la física del mecanismo.

3.2 Programación del microcontrolador

En esta sección se detalla el desarrollo e implementación del firmware encargado de ejecutar las tareas de control en tiempo real. En primer lugar, se describe de forma breve el ecosistema de desarrollo empleado, la asignación física de los pines (PINOUT) y la forma general de configurar los periféricos involucrados.

Posteriormente, se desglosa con mayor detalle la configuración a bajo nivel de los periféricos críticos de actuación (ePWM) y captura de datos (eQEP), para finalizar con el diseño del canal de telemetría (SCI) y la sincronización de la interrupción periódica del sistema (rutina de control).

Por último, el firmware completo se encuentra estructurado como un proyecto listo para ser importado, compilado y cargado en el microcontrolador. Asimismo, la versión definitiva del sistema, junto con los diversos códigos y programas de prueba desarrollados durante las fases experimentales intermedias (como los ensayos de la zona muerta o el ajuste individual de controladores), están disponibles de forma organizada en el directorio `firmware_C2000` del **repositorio oficial del proyecto** [18].

Ecosistema de desarrollo

El desarrollo del firmware, como se comentó en el Capítulo 2, se apoya en el entorno de desarrollo integrado (IDE) oficial de Texas Instruments, **Code Composer Studio (CCS)** [11]. Este entorno optimiza la compilación para la arquitectura C28x y proporciona herramientas avanzadas de depuración (*debug*). Gracias a ellas, es posible ejecutar el código línea por línea, así como inspeccionar y monitorizar el valor de los registros del sistema en tiempo real durante la ejecución del programa.

Además para programar el microcontrolador **TMS320F280049C** en el IDE, se tiene la SDK **C2000Ware** que ofrece tres metodologías de acceso a los periféricos y registros de memoria, cada una con un compromiso (*trade-off*) diferente entre nivel de abstracción, legibilidad y control sobre la programación: el acceso directo por dirección, el mapeo mediante estructuras de campos de bits (*Bit-fields*) y el uso de la librería de funciones *DriverLib* [22].

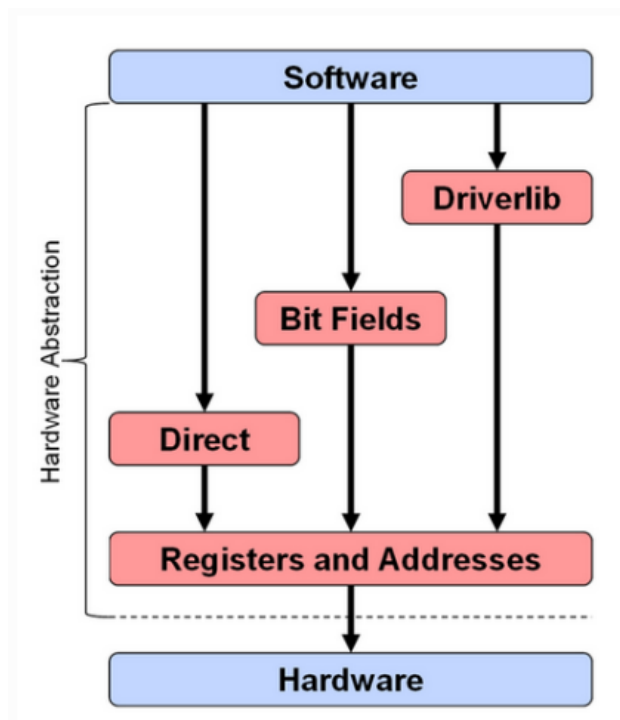


Figura 3.9: Niveles de abstracción para programar el microcontrolador [22]

El **Acceso Directo** ofrece un control absoluto pero con una dificultad de desarrollo muy alta y propensión a errores, al obligar a calcular manualmente máscaras binarias en hexadecimal. En el extremo opuesto, la librería **DriverLib** reduce la dificultad al mínimo mediante funciones abstractas de alto nivel, pero resta control directo al desarrollador sobre la configuración real de los registros del chip.

Por este motivo, se ha seleccionado el enfoque intermedio de **Campos de bits (Bit-fields)**. Esta metodología permite mapear los registros físicos del *datasheet* como estructuras de datos nativas en C. De este modo, se justifica su uso al garantizar un **control directo e inmediato sobre el hardware** bajo una dificultad moderada.

Cuadro 3.2: Comparativa de metodologías

Metodología	Dificultad	Control del Hardware	Ejemplo de Sintaxis
Acceso Directo	Alta	Absoluto (Manual)	*CMPR1 = 0x1234;
Bit-fields	Media	Alto (Directo)	EPwm1Regs.CMPA.bit.CMPA = valor;
DriverLib	Baja	Bajo (Abstracto)	EPWM_setCounterCompareValue()

La principal ventaja práctica de esta elección se manifiesta durante las etapas de depuración. Al haber programado utilizando los nombres nativos de los registros, el mapa de variables que despliega el IDE es plenamente inteligible y mantiene una trazabilidad directa con el código fuente; un beneficio que se perdería drásticamente si se empleara la abstracción de *DriverLib*.

Como se muestra en la Figura 3.10, durante la depuración del programa `control.c`, el entorno de desarrollo permite inspeccionar y monitorizar en tiempo real el valor exacto de los bits y registros críticos del sistema.

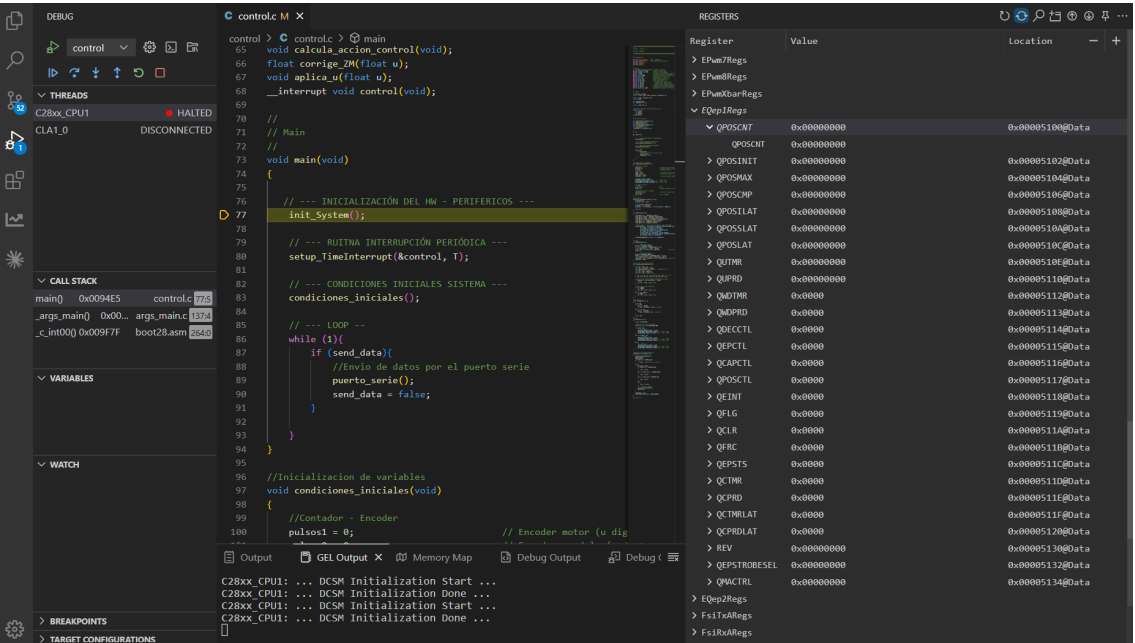


Figura 3.10: Depuración de código con CCS

Mapeo de periféricos y asignación de pines (PINOUT)

Antes de proceder con el desarrollo software, se define la interfaz física del sistema (*PINOUT*). El microcontrolador **TMS320F280049C** cuenta con un mecanismo interno de multiplexado para optimizar el uso de sus pines físicos, es decir, un mismo pin físico puede enrutarse internamente hacia módulos digitales independientes, entradas analógicas o funciones de propósito general (GPIO). La tabla completa de multiplexado del chip se proporciona en el Anexo C.1. Para la realización de este proyecto se emplea la tarjeta de desarrollo oficial **LAUNCHXL-F280049C**. La distribución espacial y función de los pines se ilustra en la Figura 3.11.

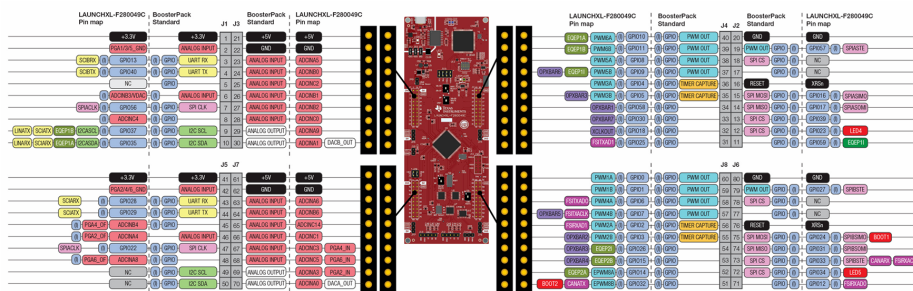


Figura 3.11: Mapa de pines y asignación de periféricos de la tarjeta LAUNCHXL-F280049C.

A partir de la distribución espacial provista por el fabricante y considerando los requerimientos operativos (actuación del motor, lectura de encoders y comunicación externa), se seleccionan los pines atendiendo a funcionalidad y localización en la tarjeta de desarrollo. En la Tabla 3.3 se consolida el mapeo definitivo de pines

Cuadro 3.3: Mapeo definitivo de entradas y salidas

Señal Física	Conexión LaunchPad	GPIO	Mux Value	Descripción Técnica / Propósito
EPWM1A	Pin 80 (J8)	GPIO0	1 (<i>ePWM</i>)	Señal de modulación de potencia (<i>ENA</i> del puente H).
MOTOR_IN1	Pin 79 (J8)	GPIO1	0 (<i>GPIO</i>)	Control de dirección del motor (<i>IN1</i> del puente H).
MOTOR_IN2	Pin 78 (J8)	GPIO6	0 (<i>GPIO</i>)	Control de dirección del motor (<i>IN2</i> del puente H).
EQEP1A	Pin 40 (J4)	GPIO10	5 (<i>eQEP</i>)	Entrada del Canal A para el encoder del carro.
EQEP1B	Pin 39 (J4)	GPIO11	5 (<i>eQEP</i>)	Entrada del Canal B para el encoder del carro.
EQEP2A	Pin 72 (J8)	GPIO14	10 (<i>eQEP</i>)	Entrada del Canal A para el encoder del péndulo.
EQEP2B	Pin 73 (J8)	GPIO15	10 (<i>eQEP</i>)	Entrada del Canal B para el encoder del péndulo.
SCIRXDA	Enrutamiento interno ¹	GPIO28	1 (<i>SCI</i>)	Entrada de recepción de comandos desde el PC.
SCITXDA	Enrutamiento interno ¹	GPIO29	1 (<i>SCI</i>)	Salida de transmisión de telemetría hacia el PC.

¹ Las señales del módulo SCI (UART) se encuentran interconectadas por hardware en el circuito de la placa LAUNCHXL-F280049C hacia el depurador XDS110. Esto permite la comunicación bidireccional con el PC a través del puerto JTAG con un único cable USB.

Configuración general de periféricos

La utilidad de este apartado radica en exponer el procedimiento que se ha diseñado en el firmware para la correcta habilitación, inicialización y puesta en marcha de los periféricos esenciales del sistema. Con el fin de estructurar el código de manera eficiente, el proceso de inicialización de cada módulo consta de tres fases fundamentales:

- **Habilitación del reloj del periférico:** Se activa la señal de reloj específica a través de los registros de control del sistema (`CpuSysRegs.PCLKCR2` para el módulo *ePWM1* y `CpuSysRegs.PCLKCR4` para los módulos *eQEP*). Debido a que estos registros están protegidos, se requiere el uso de las instrucciones de ensamblador `EALLOW` y `EDIS` para su modificación segura.
- **Configuración de registros internos:** Ajuste de los registros del módulo de acuerdo con las especificaciones del fabricante para obtener el comportamiento deseado.
- **Activación final del módulo:** Modificación del bit de habilitación global por hardware (como el bit `QEPEN` en el caso de la lógica *eQEP*) para indicar a la CPU que el periférico se encuentra plenamente operativo.

Con el propósito de garantizar la modularidad y legibilidad del software, la totalidad de este proceso de configuración general se ha encapsulado en el archivo de código fuente `periféricos.c`. La implementación íntegra del firmware desarrollado y la estructura de directorios del proyecto se encuentran accesibles de forma pública en el **repositorio oficial del proyecto** [18]. Dada la relevancia de estos submódulos, en los siguientes apartados se analiza con detalle su implementación y funcionamiento interno.

3.2.1 Módulo eQEP (Lectura del Encoder)

La lectura de los encoders ópticos se delega en una máquina de estados integrada en el hardware del periférico *eQEP*. Como se ilustra en la Figura 3.12, las señales físicas de los canales *QEPA* y *QEPB* presentan un desfase eléctrico de 90° . Al activar el modo de decodificación en cuadratura, el hardware detecta cada flanco de subida y bajada, lo que permite transitar por cuatro combinaciones binarias distintas (00, 10, 11, 01) por cada periodo de la señal.

Esta decodificación no solo cuadruplica ($\times 4$) la resolución del encoder, sino que permite discernir instantáneamente el sentido de giro (horario o antihorario) según la secuencia de transición de los estados.

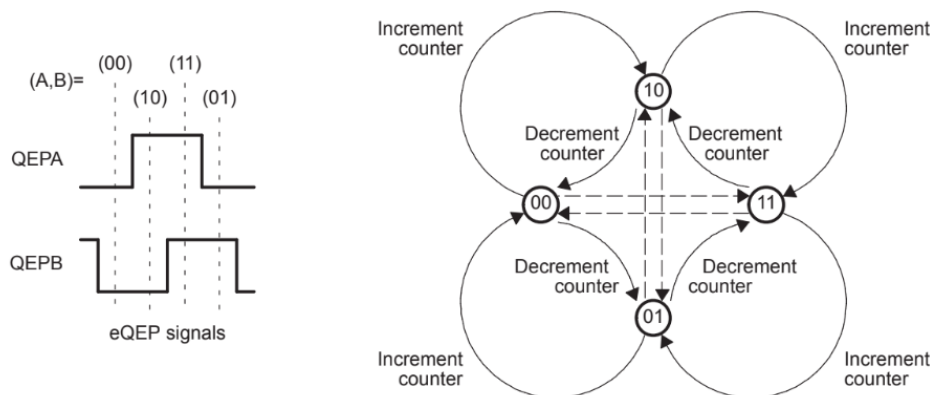


Figura 3.12: Máquina de estados, decodificación en cuadratura [23]

En la Tabla 3.4 se detalla la configuración final asignada a los campos de bits de este módulo para conseguir la decodificación en cuadratura. Si se desea conocer en detalle el mapa de memoria y configuraciones alternativas consulte el Anexo C.2.

Cuadro 3.4: Configuración final de los registros del módulo eQEP

Registro / Campo	Valor Asignado	Justificación Técnica / Propósito
QDECCTL.bit.QSRC	0	Modo cuadratura (resolución $\times 4$).
QEPCTL.bit.FREE_SOFT	2	El contador ignora los puntos de parada del <i>debugger</i> .
QPOSMAX	0xFFFFFFFF	Límite máximo de conteo (rango completo de 32 bits).
QPOSINIT	0	Posición de inicialización.
QPOSCNT	0	Reset del contador de posición al arrancar.
QEPCTL.bit.QPEN	1	Habilitación del periférico.

La asignación de un rango de 32 bits mediante QPOSMAX garantiza que el sistema pueda registrar el desplazamiento total del carro y la posición del péndulo sin riesgo de desbordamiento (*overflow*). Además, la configuración actual del campo FREE_SOFT está pensada para las fases de depuración: al indicarle al periférico que ignore los puntos de parada del entorno de desarrollo, se evita que el contador de posición se congele o pierda la sincronía con el estado físico real del sistema cuando se pausa manualmente la ejecución del programa.

3.2.2 Módulo ePWM1 (Actuación del Motor)

Para comprender el funcionamiento del periférico *ePWM*, es necesario analizar tanto su infraestructura interna como el comportamiento de su temporizador principal. En la Figura 3.13 se presenta el diagrama de bloques general del módulo. Como se observa, el periférico no es un simple temporizador, sino un ecosistema compuesto por varios submódulos especializados.

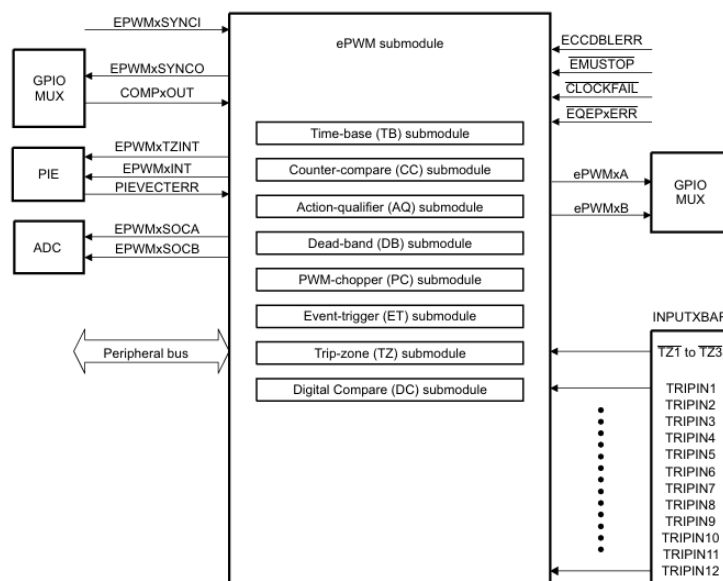


Figura 3.13: Diagrama de bloques y submódulos del periférico ePWM [23].

El comportamiento de la señal de modulación se rige principalmente por la acción combinada de sus tres primeros submódulos:

- **Time-base (TB) submodule**
- **Counter-compare (CC) submodule**
- **Action-qualifier (AC) submodule**

El **Submódulo Time-base** determina la simetría, la resolución y la frecuencia de la señal PWM. Como ilustra la Figura 3.14, se ha seleccionado el modo simétrico triangular o conteo hacia arriba y hacia abajo (*Up-Down count*). En este modo, el contador dinámico TBCTR incrementa su valor hasta alcanzar el límite cargado en el registro de periodo TBPRD y luego desciende simétricamente hasta cero, duplicando la duración del periodo de la señal PWM respecto a los modos unidireccionales.

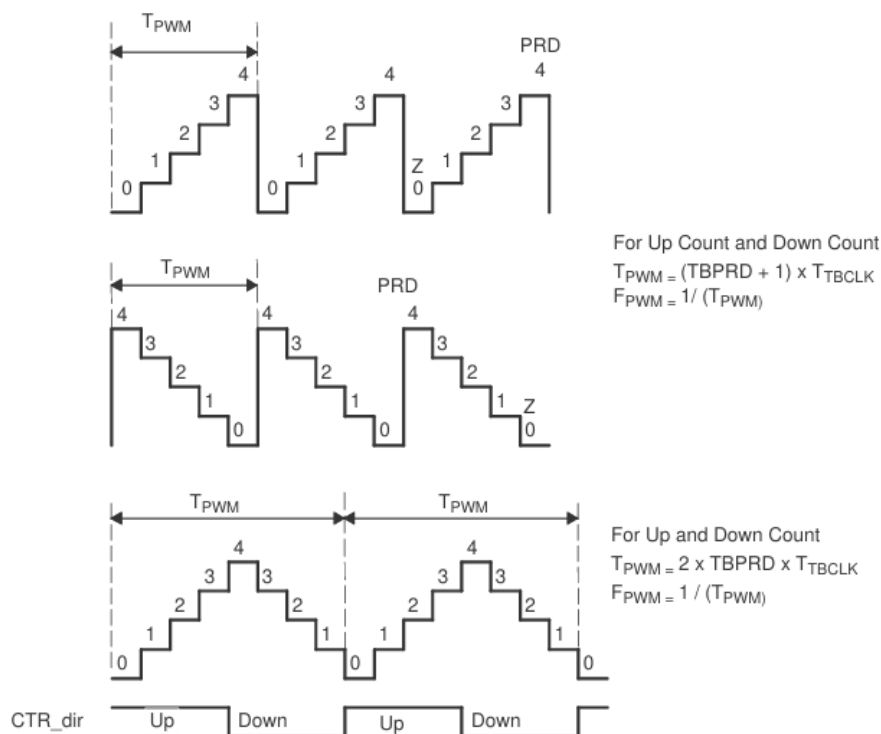


Figura 3.14: Modos de conteo del submódulo Time-base (TB) y ecuaciones de periodo [23].

A partir de la rampa triangular establecida, el **Submódulo de Comparación (Counter-compare)** supervisa en tiempo real el valor instantáneo del contador TBCTR y lo contrasta con el registro CMPA.

La generación real de la onda cuadrada se delega posteriormente en el **Submódulo Calificador de Acción (Action-qualifier)**. Cuando TBCTR intersecta el valor de CMPA, el periférico altera instantáneamente el estado del pin físico sin intervención de la CPU, forzando un nivel lógico bajo (LOW) durante la fase de subida (*Up-count*) y un nivel alto (HIGH) durante la fase de bajada (*Down-count*).

Debido a esta combinación de eventos implementada en el firmware, el ciclo de trabajo (*duty cycle*) de la señal resultante guarda una relación lineal y directamente proporcional con el valor cargado en CMPA:

- **CMPA = 0:** Las intersecciones ocurren en el origen de la rampa. Al iniciar la pendiente ascendente, el evento de subida (CAU) fuerza de inmediato la señal a nivel bajo, manteniéndola de forma continua en 0 V, lo que equivale a un **ciclo de trabajo del 0 %** (motor completamente detenido).
- **CMPA = TBPRD / 2:** Las intersecciones ocurren exactamente en el punto medio de ambas pendientes, generando una onda perfectamente simétrica con un **ciclo de trabajo del 50 %** (el motor recibe la mitad de la tensión eficaz).
- **CMPA = TBPRD:** El umbral de comparación se sitúa en el vértice superior de la rampa triangular, maximizando el tiempo en que la señal permanece en nivel lógico alto. Esto se traduce en un **ciclo de trabajo del 100 %** (el motor opera a su máxima velocidad de giro al recibir el valor eficaz completo de la fuente).

En la Tabla 3.5 se sintetiza la configuración definitiva cargada en los campos de bits de la instancia EPwm1Regs para establecer una frecuencia de conmutación de 1 kHz atendiendo a criterios de zona muerta en el actuador y no linealidades debidas a la fricción estática que se abordan en la Sección 3.3.

Cuadro 3.5: Configuración final de los registros de control del módulo ePWM1.

Registro / Campo	Valor Asignado	Justificación Técnica / Propósito
TBPRD	5000	Resolución para modular el ciclo de trabajo
TBCTR	0	Inicialización y limpieza preventiva del contador de tiempo base.
TBCTL.bit.CTRMODE	2	Configuración de conteo triangular simétrico.
TBCTL.bit.HSPCLKDIV	5	Pre-escalador de reloj, alta velocidad $TBCLK = SYSCLK / (HSPCLKDIV \cdot CLKDIV)$.
TBCTL.bit.CLKDIV	1	Pre-escalador de reloj secundario.
CMPA.bit.CMPA	0	Ciclo de trabajo (<i>duty cycle</i>) inicial en un estado seguro del 0 %.
AQCTLA.bit.CAU	1	Fuerza la señal a 0 V al cruzar el umbral CMPA en la fase de subida.
AQCTLA.bit.CAD	2	Fuerza la señal a 3.3 V al cruzar el umbral CMPA en la fase de bajada.

3.2.3 Protocolo de comunicación serie y telemetría (SCI)

La integración de un canal de comunicación serie responde a la necesidad de adquirir y monitorizar desde el PC en tiempo real la telemetría del sistema de péndulo invertido sobre carro. Disponer de un flujo constante de datos es un requisito fundamental para validar el correcto desempeño del bucle de control, ya que permite monitorizar las variables críticas del sistema (x, θ, u).

Con el propósito de garantizar la interoperabilidad con las herramientas de análisis de datos (como *MATLAB* o *Python*) y agilizar la posterior generación de gráficas de rendimiento, la trama de datos se estructura y transmite en formato *CSV* (*Comma-Separated Values*). A continuación se describen los conceptos elementales de la comunicación serie implementada.

Arquitectura general del módulo SCI

El periférico *SCI* (*Serial Communications Interface*) integrado en el **TMS320F280049C** opera como un módulo de comunicación asíncrona de dos canales (*UART*). Para comprender su funcionamiento de forma general, la estructura interna del periférico se puede simplificar en el diagrama de bloques presentado en la Figura 3.15.

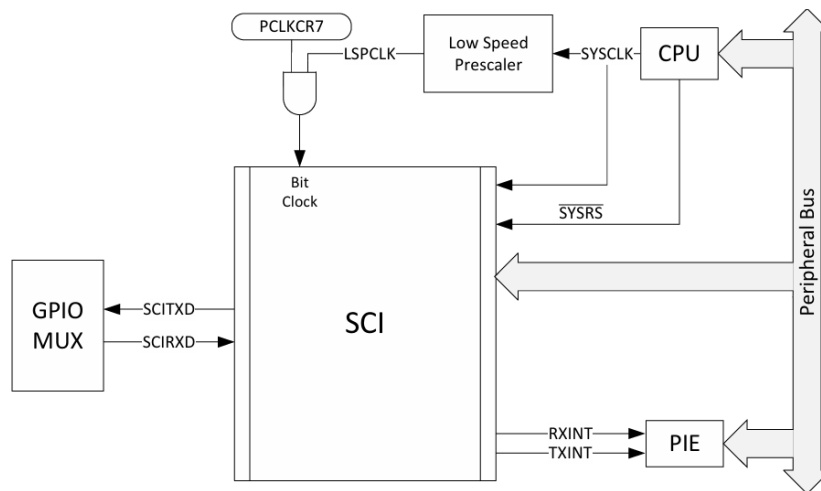


Figura 3.15: Diagrama de bloques y conexiones principales del periférico SCI [23]

Como se observa en el esquema, el funcionamiento del módulo con el resto del microcontrolador se rige bajo cuatro ejes principales:

- **Conexión con la CPU y Buses:** El núcleo del procesador (*CPU*) se comunica de forma directa con los registros internos del módulo *SCI* a través del bus periférico (*Peripheral Bus*).
- **Sincronización de Reloj (*Bit Clock*):** El módulo de comunicaciones depende de la señal de reloj *LSPCLK* (*Low Speed Peripheral Clock*). El acceso a esta señal está condicionado por una compuerta lógica controlada por el registro de control *PCLKCR7*, se debe configurar para habilitar el periférico.
- **Enrutamiento Externo (*GPIO MUX*):** Las señales del periférico (*SCITXD* para transmitir y *SCIRXD* para recibir) pasan primero por el multiplexor de pines (*GPIO MUX*), que se encarga de dirigir físicamente estas líneas hacia los pines externos de la placa.
- **Gestor de Interrupciones (*PIE*):** Para evitar que la CPU consuma ciclos de trabajo revisando continuamente el envío de una trama, el módulo *SCI* cuenta con líneas de aviso directo hacia el periférico de interrupciones (*PIE*) mediante las señales *RXINT* y *TXINT*, alertando al procesador únicamente cuando ocurre un evento en el canal de comunicaciones.

Tasa de baudios (Baudrate)

La tasa de baudios o *baudrate* define la velocidad a la que se transmite la información a través del puerto serie, expresada en bits por segundo (bps). Para la comunicación de la telemetría en este proyecto, se ha seleccionado una velocidad estándar y alta de **115200 bps**. Esta elección responde a un compromiso: es lo suficientemente rápida como para transmitir todas las variables de estado del péndulo sin acumular retrasos en la rutina de control, pero no lo bastante alta como para sufrir pérdidas durante la transmisión.

Para que el módulo *SCI* interno funcione a esta velocidad exacta, el hardware necesita dividir la frecuencia de su reloj de entrada. Como se observaba en el diagrama general, el periférico se alimenta de la señal de baja velocidad *LSPCLK*, la cual trabaja a una frecuencia fija de 25 MHz (derivada de los 100MHz del reloj principal del sistema o *SYSCLK*) [23].

La configuración de esta velocidad en el firmware se realiza dividiendo el valor de configuración en dos registros de 8 bits (*SCIHBAUD* y *SCILBAUD*) [23]. Para obtener la tasa de baudios deseada se asignan los siguientes valores:

- `SciaRegs.SCIHBAUD.all = 0x0000;` — Configura la parte alta del divisor a cero.
- `SciaRegs.SCILBAUD.all = 0x001A;` — Carga el valor hexadecimal 0x1A (26 en sistema decimal) en la parte baja del divisor.

La combinación de estos registros genera un factor de división por hardware que adapta los 25MHz del reloj interno para aproximar la comunicación lo máximo posible a los 115200bps teóricos. El desglose detallado del cálculo se detalla paso a paso en el Anexo C.4.

Formato de la trama

La comunicación asíncrona requiere que el emisor y el receptor acuerden de antemano la estructura bit a bit de la palabra que viaja por la línea.

En el firmware desarrollado, la definición la trama se gestiona a través del registro de control *SCICCR*. Al cargar el valor hexadecimal 0x0007 (equivalente al binario 00000111b), se configuran la trama con:

- 1 bit de parada.
- 8 bits de datos.
- Sin bit de paridad.

Bits	Bit Name	Designation	Functions
2-0	SCICHAR	SCICCR.2:0	Select the character (data) length (one to eight bits).
5	PARITYENA (ENABLE)	SCICCR.5	Enables the parity function if set to 1, or disables the parity function if cleared to 0.
6	PARITY (EVEN/ODD)	SCICCR.6	If parity is enabled, selects odd parity if cleared to 0; even parity if set to 1.
7	STOPBITS	SCICCR.7	Determines the number of stop bits transmitted—one stop bit if cleared to 0 or two stop bits if set to 1.

Figura 3.16: Formato de la trama según el registro *SCICCR* [23]

Implementación del envío de datos

Una vez establecida la configuración de bajo nivel del periférico, es necesario abordar la lógica de software encargada de explotar el puerto serie. La función `puerto_serie()`, detallada en el Bloque de Código 3.1, constituye la interfaz de alto nivel desarrollada para empaquetar y transferir las variables dinámicas del sistema desde el microcontrolador hacia el ordenador en tiempo real.

```

1  void puerto_serie(void)
2  {
3      float_parts p_x      = desglosar_float(x);
4      float_parts p_theta  = desglosar_float(theta);
5      float_parts p_u      = desglosar_float(u);
6
7      // Formato CSV: x, theta, u
8      sprintf(txBuffer, "%c%d.%04d,%c%d.%04d,%c%d.%04d\r\n",
9              p_x.signo,    p_x.entero,    p_x.decimal,
10             p_theta.signo, p_theta.entero, p_theta.decimal,
11             p_u.signo,    p_u.entero,    p_u.decimal);
12
13     transmitSCIAMessage((unsigned char *)txBuffer);
14 }

```

Listing 3.1: Envío de la telemetría vía puerto serie en formato CSV.

El funcionamiento interno de esta rutina se estructura bajo una secuencia de tres etapas orientadas a optimizar el uso de la CPU y estandarizar la salida:

1. **Descomposición de variables:** Las variables fundamentales (la posición del carro x , el ángulo de la varilla θ y la acción de control calculada u) se gestionan internamente en formato de punto flotante (`float`). Debido a que el uso nativo de operaciones flotantes con `%f` dentro de `sprintf` consume un número excesivo de ciclos de reloj, se invoca inicialmente la rutina auxiliar `desglosar_float()`. Esta extrae de forma aislada el signo (carácter '+' o '-'), la parte entera y los primeros cuatro dígitos de la parte decimal, empaquetándolos como enteros independientes en la estructura personalizada `float_parts`.
2. **Formato CSV:** Con los datos ya divididos en formato de caracteres y enteros simples, la función estándar `sprintf` concatena los campos de manera eficiente dentro del búfer global de transmisión `txBuffer`. El uso de los formateadores `%c` y `%d` garantiza un coste de cómputo mínimo. Además, las variables se separan estrictamente por comas (,) y la línea se remata mediante los caracteres de escape de retorno de carro y salto de línea (`\r\n`). Esto genera una hilera de texto plano bajo el estándar CSV.
3. **Transmisión:** Finalmente, la cadena de caracteres alojada en `txBuffer` se transfiere a la función de bajo nivel `transmitSCIAMessage()`. Esta rutina recorre el vector y deposita los caracteres directamente en el búfer del periférico *SCI-A* para su posterior envío físico por la línea de transmisión.

Como resultado directo de este hito de software, se logró capturar de forma transparente la dinámica real del sistema y almacenarla en archivos CSV. Esta disponibilidad de los datos no solo facilitó la posterior supervisión del sistema, sino que constituyó la herramienta analítica esencial para abordar con éxito las etapas de calibración de sensores, la caracterización del actuador así como la identificación experimental de los parámetros del sistema.

3.2.4 Rutina de interrupción periódica y determinismo temporal

En el diseño de sistemas de control digital, el cumplimiento estricto del periodo de muestreo (T) es fundamental. La ley de control asume que los estados se miden y las acciones de control se actualizan a intervalos de tiempo perfectamente uniformes y constantes. En este proyecto, se ha establecido un periodo de muestreo de 10 ms (100Hz).

Si el cálculo de la acción de control se ejecutara dentro de un bucle convencional `while(1)` mediante retardos por software (*delays*), cualquier proceso secundario —como el formateo de la telemetría analizado en el apartado anterior— alteraría sutilmente el tiempo de ciclo. Esta fluctuación temporal (*jitter*), degrada el comportamiento del controlador y puede provocar la inestabilidad del péndulo.

Para garantizar el **determinismo temporal**, el sistema delega la sincronización en un temporizador hardware (*timer*) y en el sistema de interrupciones del microcontrolador. Es decir, cuando el temporizador llega a 10 ms genera una "alerta" en el sistema que produce la ejecución de la rutina de control.

Configuración del timer

Se desarrolló la rutina de configuración `setup_TimeInterrupt()`. Esta función actúa como una capa de abstracción que vincula el temporizador de hardware con el algoritmo de control. La función se invoca de manera limpia desde el archivo principal `control.c` mediante la siguiente instrucción:

```
1 // Asociar la rutina "control" al periodo T
2 setup_TimeInterrupt(&control, T);
```

Listing 3.2: Configuración del timer desde el programa principal, más información en el Anexo C.5.

Al recibir como argumentos la dirección de memoria de la función de control (`&control`) y el periodo de muestreo deseado (T), el hardware coordina un canal de comunicación directo con la CPU. El flujo simplificado de esta señal de alerta se ilustra en el esquema de la Figura 3.17.

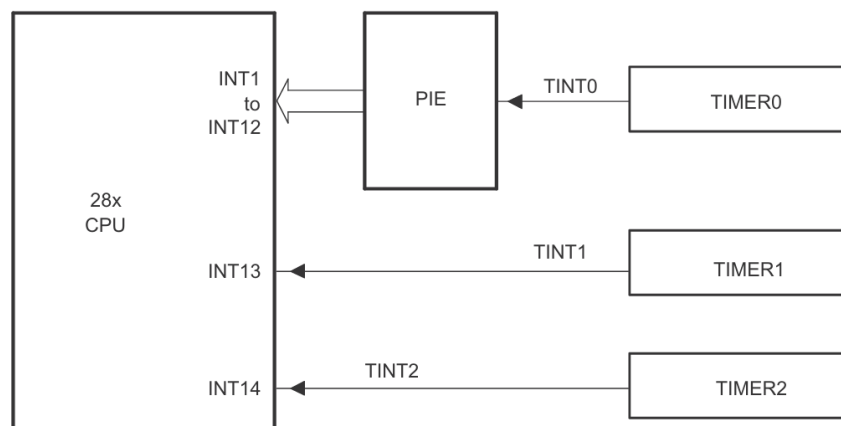


Figura 3.17: Rutas de las señales de interrupción asociadas a los TIMERS [23].

El `TIMER0` genera de forma autónoma la señal de aviso `TINT0` cada vez que expiran los 10 ms. Esta alerta es recogida por el periférico de interrupciones (*PIE*), el cual se encarga de canalizarla y enviarla hacia la línea prioritaria `INT1` de la CPU.

Como resultado, cada vez que el temporizador indica que han transcurrido los 10 ms, la CPU pausa la ejecución de cualquier tarea secundaria (como la preparación de las tramas serie), ejecuta de manera estrictamente prioritaria el algoritmo de control y regresa a sus funciones de fondo sin haber introducido retrasos en el bucle de control.

3.3 Compensación y caracterización de la zona muerta

Una vez finalizada la programación del microcontrolador, uno de los problemas durante el desarrollo de este proyecto fue la presencia de la *zona muerta* (*dead-band*) en el motor DC. Manifestándose como un intervalo de voltajes cercanos a cero en los cuales el motor es incapaz de vencer su propia fricción estática y la inercia inicial, permaneciendo completamente en reposo.

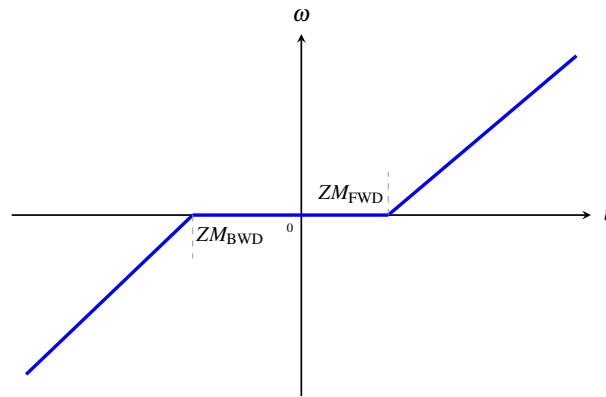


Figura 3.18: Representación conceptual de la zona muerta

En la Figura 3.18 se ilustra de manera conceptual esta no-linealidad, donde se aprecia la banda muerta central en la relación entre el voltaje aplicado (u) y la velocidad angular resultante (ω).

El Bloque de Código 3.3 implementa la corrección de la zona muerta por software. La señal "salta" la banda de no-linealidad del actuador, garantizando que el puente en H conmute desde el primer instante un voltaje efectivo capaz de mover el motor.

```

1
2 // Correccion de la zona muerta
3 float corrige_ZM(float u)
4 {
5     float uzmm;
6     if (u < 0) {
7         uzmm = u - ZM_BWD; // ZM hacia atras
8         if (uzmm < -VCC) {
9             uzmm = -VCC; // Saturation
10        }
11    }
12    else if (u > 0) {
13        uzmm = u + ZM_FWD; // ZM hacia delante
14        if (uzmm > VCC) {
15            uzmm = VCC; //Saturation
16        }
17    }
18    else {
19        uzmm = 0.0f;
20    }
21    return uzmm;
22 }
```

Listing 3.3: Compensación de la zona muerta por código

Destaca la necesidad de diferenciar analíticamente entre un umbral hacia adelante (ZM_{FWD}) y otro hacia atrás (ZM_{BWD}) responde a las asimetría de la fricción estática en el motor real.

3.3.1 Metodología de caracterización experimental

Para determinar con precisión los valores numéricos de los umbrales ZM_FWD y ZM_BWD , se diseñó un experimento en forma de máquina de estados finita (Código 3.4). El experimento se fundamenta en un ensayo que sigue la secuencia temporal descrita a continuación:

- **Incremento escalonado:** Partiendo del reposo absoluto ($u = 0$), la acción de control inyectada al actuador se incrementa de manera progresiva en escalones discretos de 10 unidades digitales de PWM.
- **Ventana de estabilización:** Cada escalón de tensión se mantiene constante durante un intervalo estricto de 0.5 s.
- **Inversión de marcha (Asimetría):** Tras finalizar el experimento en el sentido de giro positivo, el proceso se replica de forma idéntica inyectando valores decrecientes y negativos. De este modo, se evalúa el comportamiento en sentido inverso.

```

1 __interrupt void ident_zona_muerta(void) {
2     // --- 1. Lectura encoder, obtencion velocidad angular ---
3     mide_encoder();
4
5     // --- 2. Experimento zona muerta ---
6     switch(estado_test) {
7         case 0: //Adelante
8             u = u + 10;
9             if (u > 2500) { //50% del ciclo de trabajo
10                u = 0;
11                estado_test = 1;
12            } break;
13         case 1: //Detener motor por completo
14             u = 0; // Motor apagado
15             if (w == 0.0f) {
16                 estado_test = 2;
17             } break;
18         case 2: //Atras
19             u = u - 10;
20             if (u < -2500) { //50% del ciclo de trabajo
21                 u = 0;
22                 estado_test = 3;
23             }
24             break;
25         case 3: //Detener motor por completo
26             u = 0; // Motor apagado
27             if (w == 0.0f) {
28                 estado_test = 0; // Reinicia el ciclo continuo
29             } break;
30     }
31
32     // --- 3. Driver, puente H ---
33     ataca_motor(u);
34 }

```

Listing 3.4: Identificación de la zona muerta por código

3.3.2 Influencia de la frecuencia de PWM y selección de diseño

Por cuestiones físicas intrínsecas al bobinado del motor DC, la frecuencia de la señal PWM modifica el umbral de la zona muerta. Un análisis detallado de la constante de tiempo eléctrica del motor ($\tau_e = L/R$) revela un límite teórico de conmutación de 153.84Hz para que la corriente se establezca plenamente en cada ciclo, tal y como se justifica analíticamente en el Anexo A.2.

Para contrastar este principio y seleccionar la mejor configuración para el bucle de control, se ejecutó el protocolo de caracterización barriendo cinco frecuencias diferentes mediante la manipulación de los registros de pre-escalado del periférico ePWM1 (ver Anexo C.3). Las curvas experimentales resultantes se exponen en la Figura 3.19.

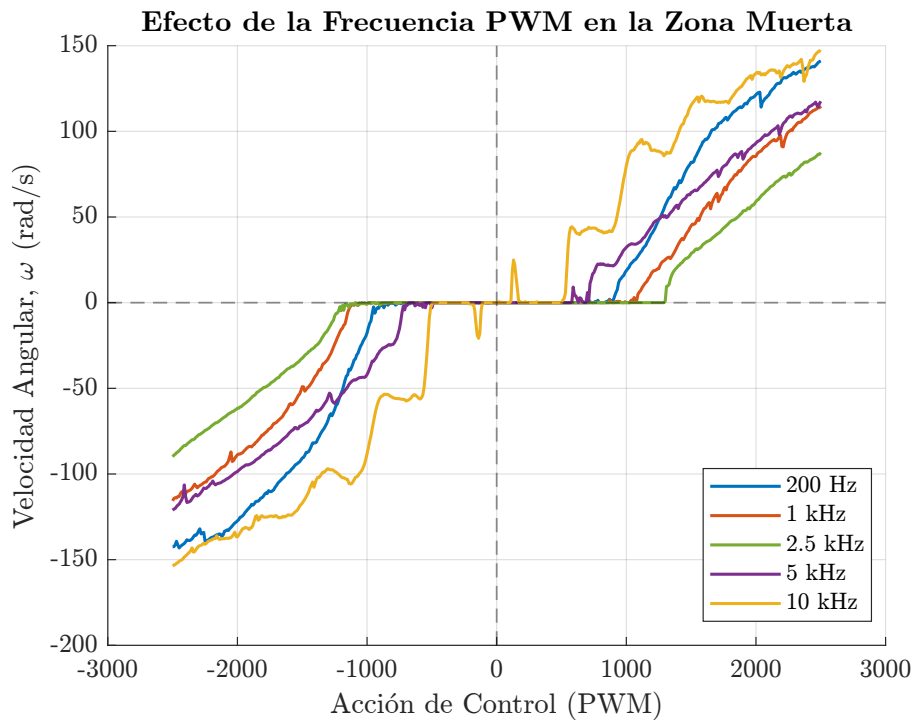


Figura 3.19: Caracterización empírica de la zona muerta del motor en función de la frecuencia del PWM.

Se determinó que la frecuencia óptima de operación para el proyecto es de **1 kHz**. Esta decisión de diseño representa un punto de equilibrio técnico justificado por tres criterios:

1. **Optimización de la linealidad frente a zona muerta:** Si bien la curva de 200Hz registra la menor zona muerta física, la respuesta a 1 kHz exhibe el segundo mejor umbral de arranque y ofrece una pendiente mucho más lineal.
2. **Limitación tecnológica del Driver (Optoacopladores):** Aunque las respuestas teóricas a 5kHz y 10kHz mitigarían por completo el ruido acústico y aparentemente presentan menor zona muerta, la gráfica desvela que su comportamiento es incorrecto. Esto es debido a que el *driver* tiene un límite de conmutación de 5kHz [13], provocando una distorsión en la transferencia de voltaje eficaz en bornes del motor.
3. **Frecuencia del lazo de control:** Una frecuencia de señal de 1 kHz resulta exactamente 10 veces superior a la tasa de muestreo del lazo digital (100Hz). Este margen asegura que la dinámica transitoria del actuador se establezca mucho antes de que el algoritmo de control en la CPU calcule e inyecte la siguiente acción de control.

3.4 Identificación de parámetros

Tal como se adelantó en el capítulo introductorio, la plataforma experimental constituye un sistema antiguo cuyo desgaste físico y uso prolongado distan de las condiciones ideales de fábrica. Luego, resulta indispensable identificar de manera empírica ciertos parámetros de la planta para otorgar validez al modelo dinámico y garantizar el ajuste de las simulaciones.

3.4.1 Relación entre fuerza y voltaje

El motivo de esta sección dentro del capítulo es que la acción de control calculada en el microcontrolador se traduce en una tensión (u) aplicada al motor DC. No obstante, las ecuaciones que gobiernan la dinámica 2.6 requieren como entrada una fuerza horizontal neta (F) expresada en Newtons.

Modelo teórico a partir de parámetros del fabricante

A bajas revoluciones (donde la fuerza electromotriz inversa es despreciable), la corriente que circula por el motor está gobernada por la ley de Ohm en función de su resistencia interna (R). A partir de la constante de par del motor (K_t), el par desarrollado (τ) se define teóricamente como:

$$\tau = K_t \cdot I = \frac{K_t}{R} \cdot u \quad (3.2)$$

Como la transmisión es un sistema de correa dentada y polea de radio efectivo r , la relación par-fuerza (F) responde a:

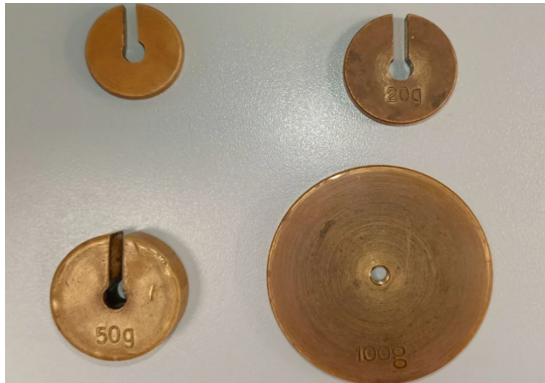
$$F = \frac{\tau}{r} = \frac{K_t}{r \cdot R} \cdot u \quad (3.3)$$

Sustituyendo con los parámetros del fabricante de la Tabla 2.2 ($K_t = 0.068 \text{ Nm/A}$, $R = 5.5 \Omega$ y un radio medido de la polea de $r = 0.025 \text{ m}$), se obtiene la siguiente relación:

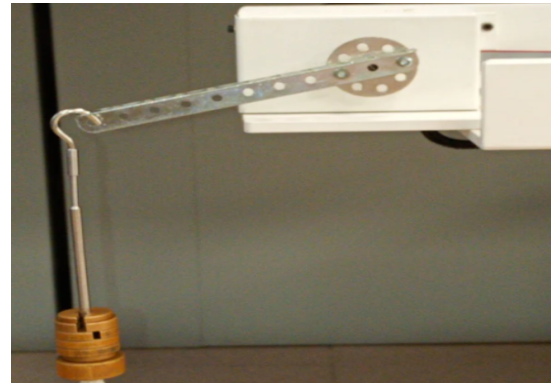
$$F_{\text{teórica}} = \frac{0.068}{0.025 \cdot 5.5} \cdot u \approx 0.5 \cdot u \quad (3.4)$$

Diseño del experimento

Con el propósito de caracterizar la ganancia real del actuador, se diseñó un banco de pruebas mecánico. El montaje consiste en acoplar rígidamente al eje del motor una palanca de longitud conocida ($L = 0.114 \text{ m}$), en cuyo extremo opuesto se suspenden de forma secuencial una serie de masas calibradas ($m \in \{10, 20, 50, 100\} \text{ g}$).



(a) Pesos calibrados para el experimento



(b) Montaje del experimento sobre el eje del motor

Figura 3.20: Banco de pruebas del ensayo experimental

El experimento se abordó implementando un algoritmo de control de posición del motor en el **TMS32F280049C**, orientado a contrarrestar el par gravitatorio inducido por la carga y mantener la posición angular fija ($\theta_{ref} = 0\text{rad}$). Se realizaron dos ensayos distintos:

1. **Control de posición PI:** El propósito de la componente integral consiste en garantizar un error de posición nulo ($e_p = 0$). Dado que el controlador contrarresta el par gravitatorio y estabiliza la posición de la palanca en $\theta = 0\text{rad}$, el par estático generado por la masa suspendida se calcula como:

$$\tau = \tau_g = m \cdot g \cdot l \quad (3.5)$$

De este modo, la acción de control en voltios (u) solicitada por el algoritmo iguala exactamente el voltaje necesario para equilibrar dicho par en el equilibrio.

2. **Control de posición P:** Este enfoque se utilizó para observar la desviación angular del sistema ante incrementos de carga, aislando posibles efectos de acumulación de la acción integral. Debido a que la palanca experimenta una inclinación estacionaria ($\theta \neq 0$), el par efectivo sobre el eje del actuador se ve atenuado y queda gobernado por la expresión:

$$\tau_{\text{ext}} = m \cdot g \cdot l \cdot \cos(\theta) \quad (3.6)$$

Deducción de la ganancia real

Tras correlacionar mediante regresión lineal los valores de voltaje medidos frente al par gravitatorio ejercido por las masas calibradas, se obtuvieron las siguientes relaciones entre par (τ) y voltaje (u):

$$\begin{cases} \tau_{\text{PI}} = 0.015 \cdot u \\ \tau_{\text{P}} = 0.016 \cdot u \end{cases} \quad (3.7)$$

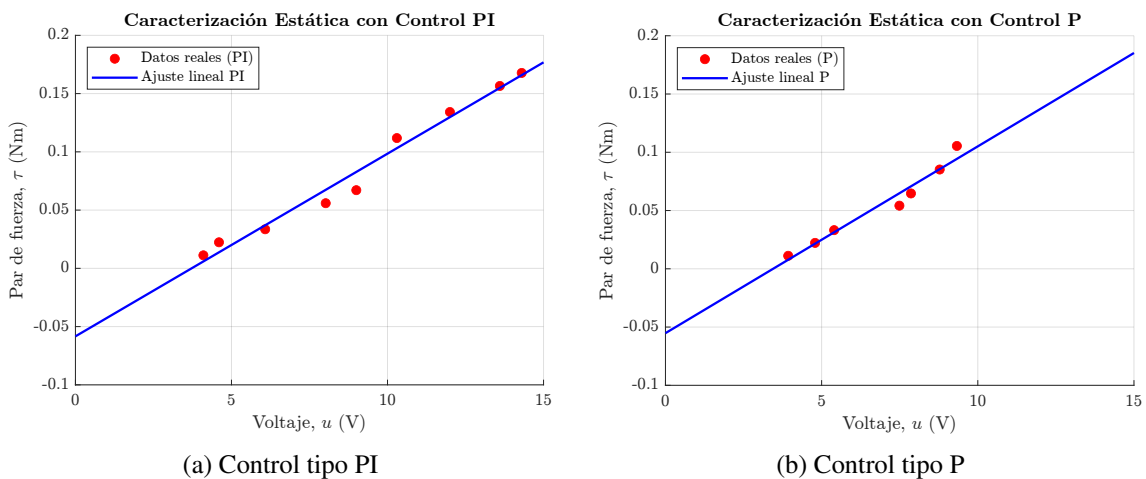


Figura 3.21: Ajuste de los modelos lineales sobre los datos experimentales

Como se puede observar en la Figura 3.21, el ajuste de los modelos lineales obtenidos mediante regresión a los datos experimentales es bastante satisfactorio en ambos ensayos. Por tanto, la relación par-voltaje deducida es plenamente válida y útil para ser integrada en la conversión de unidades del modelado (Newton) a unidades del microcontrolador (Voltios).

Tomando como referencia el modelo obtenido del control P ($\tau = 0.016 \cdot u$), se aplica dicha ganancia sobre el radio de la polea real del carro ($r = 0.025$ m) para obtener la transformación a fuerza lineal:

$$F = \frac{\tau}{r} = \frac{0.016}{0.025} \cdot u = 0.64 \cdot u \quad (3.8)$$

Este resultado concluye que, bajo condiciones de operación en la plataforma física, **por cada voltio aplicado al actuador se genera una fuerza horizontal efectiva de 0.64 N sobre el carro.** Luego la ecuación utilizada para modelar el sistema y ajustar controladores se actualiza a:

$$\begin{bmatrix} M + m & ml \cos \theta \\ ml \cos \theta & I + ml^2 \end{bmatrix} \begin{bmatrix} \ddot{x} \\ \ddot{\theta} \end{bmatrix} + \begin{bmatrix} c_x & -ml\dot{\theta} \sin \theta \\ 0 & c_\theta \end{bmatrix} \begin{bmatrix} \dot{x} \\ \dot{\theta} \end{bmatrix} + \begin{bmatrix} 0 \\ mgl \sin \theta \end{bmatrix} = \begin{bmatrix} 0.64 \\ 0 \end{bmatrix} u \quad (3.9)$$

Comparación con el modelo teórico

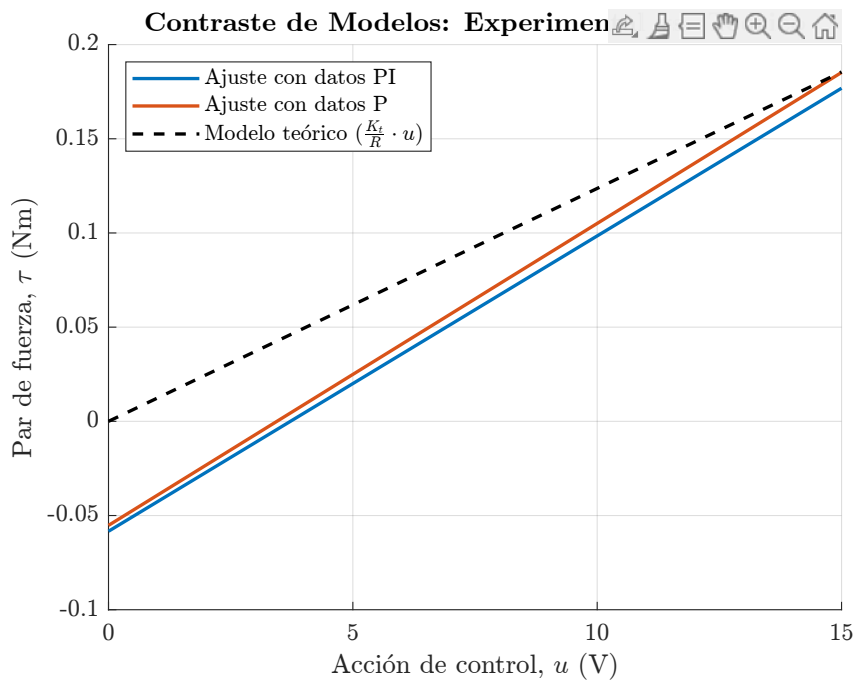


Figura 3.22: Comparación entre el modelo teórico y el modelo obtenido experimentalmente.

Finalmente, al comparar las funciones de transferencia estáticas obtenidas experimentalmente frente a la respuesta teórica con los datos del fabricante, se extraen dos conclusiones para la validación de la plataforma (Figura 3.22):

- **Efecto de la zona muerta:** Mientras que la curva teórica asume un comportamiento lineal ideal que cruza el origen matemático (0,0), las aproximaciones experimentales (tanto con control P como PI) registran un desplazamiento negativo sobre el eje de par de aproximadamente -0.05 Nm bajo una acción de control nula ($u = 0$ V). Este hecho evidencia que el actuador real requiere un umbral crítico de tensión de arranque (entre los 3.5 V y 4 V) destinado exclusivamente a vencer la fricción estática.
- **Ganancia del modelo y fidelidad experimental:** Se observa que la ganancia estática de conversión final a fuerza horizontal deducida en el modelo empírico (0.64 N/V) resulta ligeramente superior a la calculada teóricamente (0.4945 N/V ≈ 0.5 N/V). Esta discrepancia es un fenómeno habitual en la identificación de sistemas. Al ser una medición directa sobre el hardware real del proyecto, se adopta la ganancia obtenida de forma empírica.

3.4.2 Masa y rozamiento del carro

Debido a que el carro de la plataforma presenta un recorrido lineal acotado por los extremos físicos del raíl, la ejecución de ensayos de identificación en bucle abierto (idea inicial) conlleva el riesgo de colisionar con los finales de carrera. Por este motivo, se optó por un enfoque alternativo de **identificación en bucle cerrado** [24] para lograr el cometido.

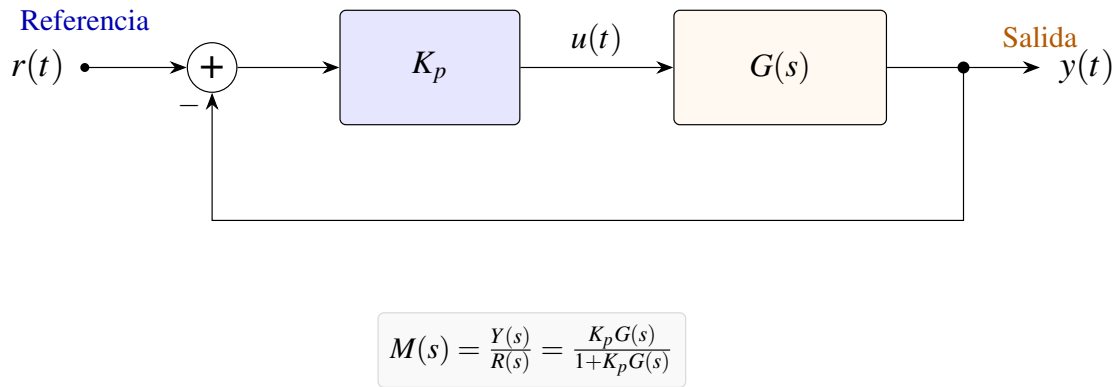


Figura 3.23: Control P para identificación en BC

El procedimiento consiste en inyectar una señal de referencia de posición $r(t)$. Mientras el controlador proporcional (K_p) actúa para estabilizar el sistema y confinar el carro dentro de los límites de seguridad del raíl, se registra la posición del carro sobre el riel $y(t)$.

A partir de estas dos señales ($r(t)$ e $y(t)$), las herramientas de identificación no proporcionan directamente el comportamiento físico de la planta ($G(s)$), sino la función de transferencia del bucle cerrado, denotada como $M(s)$ [24]:

$$M(s) = \frac{Y(s)}{R(s)} = \frac{K_p \cdot G(s)}{1 + K_p \cdot G(s)} \quad (3.10)$$

Una vez caracterizada la dinámica matemática de $M(s)$, se extrae la función de transferencia aislada de la planta, $G(s)$ [24]:

$$G(s) = \frac{M(s)}{K_p \cdot (1 - M(s))} \quad (3.11)$$

A partir de $G(s)$ se puede deducir el rozamiento viscoso y la masa del carro. Seguidamente se muestra la metodología seguida para la identificación en bucle cerrado, la dinámica obtenida experimentalmente y los parámetros finalmente caracterizados.

Diseño del experimento y adquisición de datos

Para materializar la metodología de identificación expuesta y excitar la dinámica del carro de forma segura, se implementó en el microcontrolador una rutina periódica en base a una máquina de estados temporizada. El ensayo práctico se estructuró bajo las siguientes directrices:

- **Entradas escalón:** La referencia de posición del raíl ($r(t)$) conmuta en magnitudes discretas de $\pm 10\text{ cm}$ ($\pm 0.1\text{ m}$) cada 10 s. Esta ventana temporal de diez segundos se seleccionó estratégicamente para garantizar que el sistema consiga llegar a la posición objetivo.
- **Instrumentación y almacenamiento:** Las variables críticas monitorizadas por el microcontrolador —la referencia, la posición real obtenida a través del encoder y la acción de control calculada— se estructuraron en un vector de telemetría síncrono con la rutina de control y en formato CSV. Esta trama se transmitió en tiempo real mediante el periférico UART-SCI hacia el ordenador con una tasa de muestreo fija de 100 Hz ($T = 10\text{ ms}$).
- **Procesamiento de los datos:** El archivo de datos históricos fue importado en la herramienta especializada `systemIdentification` de MATLAB. A través de este entorno se deduce de forma empírica la función de transferencia global del bucle cerrado ($M(s)$).

Fijada la constante del controlador de posición en un valor estricto de $K_p = 80$ durante el ensayo físico, el último paso metodológico consistió en aplicar la relación analítica 3.11 para aislar la dinámica de la planta $G(s)$.

Comportamiento del sistema en BC

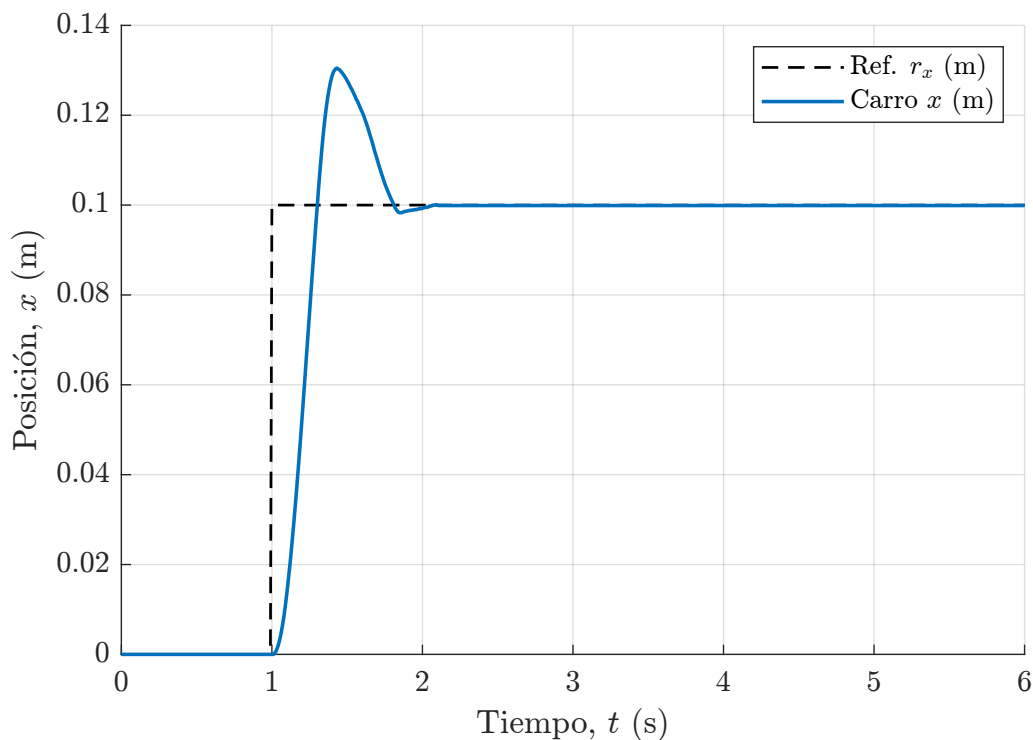


Figura 3.24: Dinámica $M(s)$ obtenida experimentalmente

Tras procesar los datos de la telemetría en MATLAB se obtiene la Figura 3.24, muestra que el control de posición del carro con $K_p = 80$ describe una **respuesta de segundo orden subamortiguada** ante cada escalón de la referencia, estabilizándose por completo antes de los 10 s programados.

Identificación de la planta

La dinámica en bucle cerrado obtenida mediante la herramienta `systemIdentification` de MATLAB es la siguiente:

$$M(s) = \frac{51.02}{s^2 + 5.714s + 51.02} \quad (3.12)$$

Si se emplea la expresión para aislar la dinámica en bucle abierto $G(s)$ (3.11). Se llega a obtener la siguiente función de transferencia:

$$G(s) = \frac{0.6378}{s(s + 5.714)} = \frac{0.11}{s(1 + 0.18s)} = \frac{K}{s(1 + \tau s)} \quad (3.13)$$

En primera instancia, la expresión abstracta de $G(s)$ en el dominio de *Laplace* describe el comportamiento dinámico global del sistema, pero no aporta información directa sobre las magnitudes físicas reales de la plataforma. Sin embargo, si se contrasta con el modelado derivado de las ecuaciones de Newton (el principio físico de masa-amortiguador del carro), es posible desglosar el sistema e identificar la masa y su coeficiente de rozamiento viscoso.

Aislamiento analítico de los parámetros físicos

La ecuación diferencial que gobierna el desplazamiento lineal del carro en el plano horizontal responde a la expresión de fuerzas (el desarrollo algebraico completo se detalla en el Apéndice A.3):

$$F(t) = M \cdot \ddot{x}(t) + c_x \cdot \dot{x}(t) \quad (3.14)$$

Si se considera la relación estática del actuador $F(s) = K_m \cdot U(s)$ (donde $K_m = 0.64 \text{ N/V}$), la función de transferencia de voltaje a posición es la siguiente:

$$G(s) = \frac{X(s)}{U(s)} = \frac{K_m/c_x}{s \left(1 + \frac{M}{c_x}s\right)} = \frac{K}{s(1 + \tau s)} \quad (3.15)$$

Conocida la ganancia del sistema ($K = 0.1116 \text{ m/(V} \cdot \text{s)}$) y su constante de tiempo ($\tau = 0.1750 \text{ s}$), se plantea el siguiente sistema de ecuaciones:

$$\begin{cases} \frac{K_m}{c_x} = K \\ \frac{M}{c_x} = \tau \end{cases} \Rightarrow \begin{cases} \frac{0.64}{c_x} = 0.1116 \\ \frac{M}{c_x} = 0.1750 \end{cases} \quad (3.16)$$

Resolviendo en primera instancia para el coeficiente de rozamiento viscoso (c_x) frente al avance, se obtiene:

$$c_x = \frac{0.64}{0.1116} \approx 5.7348 \text{ Ns/m} \quad (3.17)$$

Una vez determinado c_x , se sustituye dicho valor en la segunda relación para aislar y calcular la masa del carro (M):

$$M = \tau \cdot c_x = 0.1750 \cdot 5.7348 \approx 1.0036 \text{ Kg} \quad (3.18)$$

3.4.3 Rozamiento del péndulo

El rozamiento presente en el eje del péndulo es la fuerza que evita que oscile indefinidamente. Para caracterizar este fenómeno de forma empírica, se diseñó un ensayo de **oscilación libre**, manteniendo el motor del carro completamente desactivado ($u = 0\text{V}$) y bloqueado en una posición fija del raíl.

El firmware desarrollado para el microcontrolador TMS320F280049C (cuyo código fuente se encuentra en el repositorio del proyecto [18]) se encarga de leer el encoder del péndulo (e_{QEP2}) a una tasa de muestreo sincrónica de 100Hz ($T = 10\text{ms}$) y transmitir los valores angulares en radianes (θ) bajo formato CSV vía UART-SCI para su posterior análisis en MATLAB.

Diseño del experimento y decremento logarítmico

La metodología práctica ejecutada en el laboratorio constó de las siguientes fases operativas siguiendo las indicaciones del fabricante [19]:

- **Condiciones iniciales:** Se desplazó manualmente el péndulo hasta alcanzar una inclinación inicial aproximada de 20° ($\approx 0.35\text{rad}$) respecto de su posición estable ($\theta = 0\text{rad}$).
- **Registro temporal:** Se liberó el cuerpo de forma instantánea, permitiendo que oscilara libremente durante un intervalo extendido **entre uno y dos minutos**.
- **Procesamiento de los datos:** Tras capturar la telemetría en formato CSV. Se generó la Figura 3.25 para analizar el decremento de la amplitud de las oscilaciones en función del tiempo transcurrido.

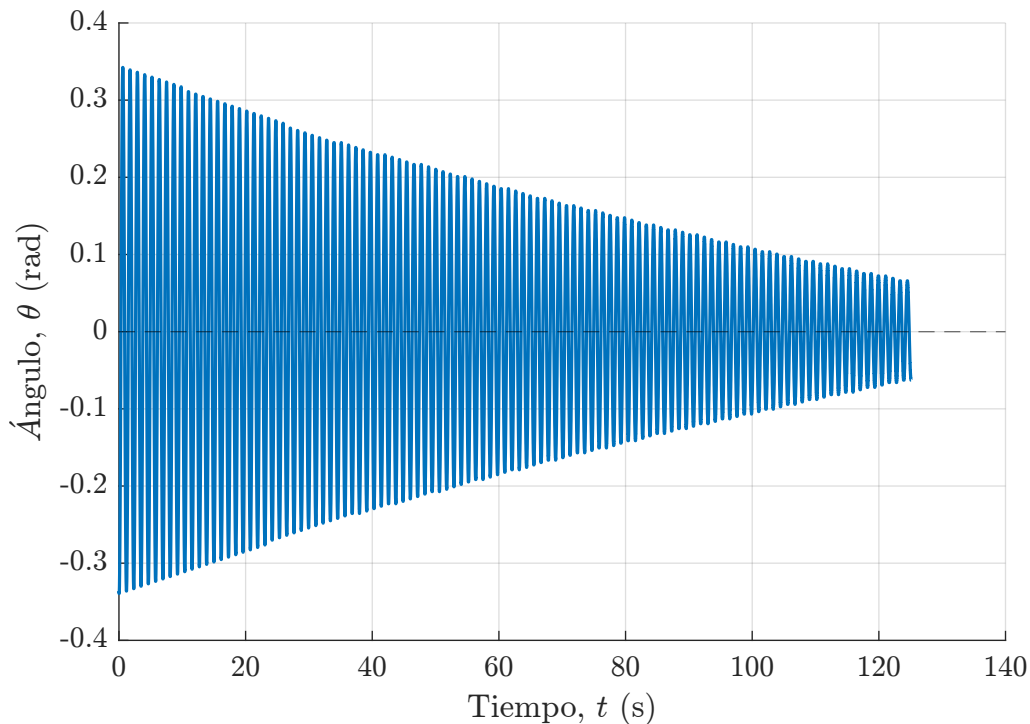


Figura 3.25: Ensayo de oscilación libre del péndulo y curva de decaimiento temporal.

Estimación del coeficiente de rozamiento

Los datos recopilados en este intervalo capturan la atenuación progresiva de la oscilación. Basándose en la naturaleza de este decaimiento exponencial, el coeficiente de rozamiento c_θ se estima con la expresión matemática [19]:

$$c_\theta = \frac{J_p}{n \cdot T} \log\left(\frac{A_1}{A_n}\right) \quad (3.19)$$

Donde cada uno de los parámetros que intervienen en el cálculo se definen a continuación:

- c_θ : Coeficiente de rozamiento viscoso equivalente del péndulo (Nms/rad).
- J_p : Momento de inercia total del péndulo ($\text{Kg} \cdot \text{m}^2$).
- T : Período de oscilación (s), correspondiente al intervalo de tiempo transcurrido entre dos picos de amplitud consecutivos en la señal temporal.
- n : Número entero de oscilaciones completas consideradas para el cálculo .
- A_1 : Amplitud angular inicial del primer pico seleccionado (rad o grados) al comienzo de la ventana de evaluación.
- A_n : Amplitud angular del pico situado n períodos después del pico inicial (rad o grados), la cual se ve atenuada debido a la pérdida de energía mecánica por fricción.

A partir del ensayo, se extrajeron las magnitudes numéricas correspondientes a una ventana de observación de dos minutos (120s), contabilizando un total de $n = 108$ oscilaciones completas ($n \approx 120/T$). Los valores experimentales junto con los datos del fabricante, quedan consolidados en la Tabla 3.6.

Cuadro 3.6: Parámetros para la identificación de c_θ

Parámetro del sistema	Símbolo	Valor
Período de oscilación promedio	T	1.12 s
Número de ciclos evaluados	n	108
Momento de inercia (fabricante)	J_p	0.0139 $\text{Kg} \cdot \text{m}^2$
Amplitud angular inicial	A_1	0.340 rad
Amplitud angular final	A_n	0.065 rad

Finalmente, sustituyendo este conjunto de valores en la expresión (3.19), se consigue la estimación definitiva del coeficiente de rozamiento viscoso:

$$c_\theta \approx 1.91 \cdot 10^{-4} \text{ Nms/rad} \quad (3.20)$$

3.4.4 Validación y contraste con los parámetros del fabricante

Para evaluar la desviación entre las aproximaciones teóricas y la realidad física del laboratorio, la Tabla 3.7 consolida una comparativa directa entre los datos proporcionados por el fabricante y los valores obtenidos empíricamente.

Cuadro 3.7: Valores del fabricante frente a resultados experimentales.

Parámetro	Símbolo	Valor Fabricante	Valor Identificado	Unidades
Masa del carro	M	1.12	1.0036	kg
Masa del péndulo	m	0.12	0.1200*	kg
Distancia al centro de masas	l	0.305	0.3050*	m
Inercia del péndulo	J_p	0.0136	0.0139†	kg · m ²
Fricción del carro	c_x	0.05	5.7348	N · s/m
Fricción del péndulo	c_θ	0.0005	0.00019	N · m · s/rad
Ganancia Voltaje-Fuerza	K_m	0.5000	0.6400	N/V

* Valores adoptados como invariantes en el modelo.

† Valor recalculado analíticamente a partir de las directrices del manual técnico.

El análisis comparativo de la Tabla 3.7 revela discrepancias de gran interés práctico, justificando plenamente la necesidad de realizar una identificación experimental antes de abordar el diseño del controlador:

- **Ajuste inercial del carro (M):** La masa efectiva obtenida experimentalmente (1.0036 kg) resulta muy próxima y del mismo orden de magnitud que la especificada por el fabricante (1.12 kg). Esta ligera reducción es habitual al haber estimado la dinámica en bucle cerrado, validando la consistencia del modelo.
- **Fenómenos disipativos (c_x y c_θ):** El coeficiente de fricción del carro (c_x) es notablemente superior al estimado en catálogo (5.7348 frente a 0.05 N · s/m). Esta diferencia radica en que el fabricante detalla únicamente el rozamiento lineal del carro deslizándose de forma aislada sobre el riel; por el contrario, el ensayo caracteriza el sistema completo, integrando la fricción interna del actuador y la resistencia de la correa dentada acoplada. Respecto a la fricción del péndulo (c_θ), el valor obtenido ($1.91 \cdot 10^{-4}$ N · m · s/rad) se mantiene dentro del mismo orden de magnitud que el valor nominal ($5 \cdot 10^{-4}$ N · m · s/rad).

La integración de estos parámetros validados en el entorno de simulación permite aproximar el modelo teórico a la realidad física de la plataforma. Este ajuste resulta indispensable para disponer de un banco de pruebas virtual preciso y representativo del comportamiento real del sistema en el laboratorio. Asimismo, con el propósito de garantizar la transparencia y la reproducibilidad de los resultados obtenidos, todos los scripts de procesamiento y ficheros de datos brutos empleados durante este proceso de análisis e identificación han sido debidamente estructurados y cargados en el **repositorio oficial del proyecto** [18].

3.5 Implementación del algoritmo de control

Para concluir con el capítulo, se presenta la implementación final de la rutina de control. Toda la estrategia algorítmica se ejecuta dentro de la rutina de interrupción periódica (ISR) asociada al `CpuTimer0` a una tasa de muestreo estricta de 100Hz ($T_s = 10\text{ms}$).

```

1  __interrupt void control(void)
2  {
3      CpuTimer0.InterruptCount++;
4
5      // --- BUCLE DE CONTROL ---
6      mide_encoder();
7      calcula_u();
8      aplica_u(u);
9
10     send_data = true; // Flag para envío por SCI
11     PieCtrlRegs.PIEACK.all = PIEACK_GROUP1;
12 }

```

Listing 3.5: Rutina de control principal (ISR).

A continuación, se detalla el diseño modular y la responsabilidad analítica de cada una de las tres subrutinas invocadas en el bucle principal:

- **mide_encoder:** Es la encargada de transformar las lecturas crudas del hardware en variables físicas con unidades del Sistema Internacional (m y rad). Para ello, lee directamente los registros de conteo de posición (QPOSCNT) de los módulos eQEP.
- **calcula_u:** Constituye el núcleo algorítmico del proyecto. Implementa la lógica de control híbrida que evalúa el error angular para conmutar de forma autónoma entre dos modos de operación: el algoritmo no lineal basado en energía (*swing-up*) para balancear el péndulo desde su posición de reposo, y la ley por realimentación del estado para estabilizar el sistema en la vertical superior.
- **aplica_u:** Recibe la acción de control calculada en voltios (u), compensa la zona muerta y convierte el valor resultante a unidades digitales para el módulo ePWM. A su vez, también asigna el sentido de giro con los GPIO encargados.

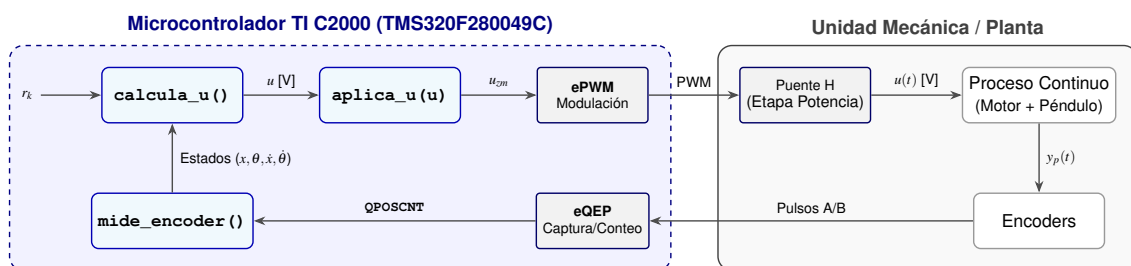


Figura 3.26: Funciones en el bucle de control

El código fuente completo se encuentra disponible para su consulta en el **repositorio oficial del proyecto** [18], concretamente en el archivo `control.c`. Este script se diseñó como una capa de abstracción de todo el hardware, aislando la complejidad de bajo nivel para permitir la experimentación y modificación de algoritmos de forma ágil y sencilla. A continuación, el análisis se focaliza únicamente en la función `calcula_u()`, dado que es la encargada de dictaminar la ley de control.

3.5.1 Control de posición en `calcula_u()`

Aunque el diseño original contemplaba un control de posición por realimentación del estado puro (Sección 2.3.5), los ensayos experimentales revelaron un error en estado estacionario en la posición del carro sobre el riel. Para corregirlo, se expandió el vector de estados original añadiendo un estado virtual (x_i) que acumula la integral del error de posición ($x_{ref} - x$).

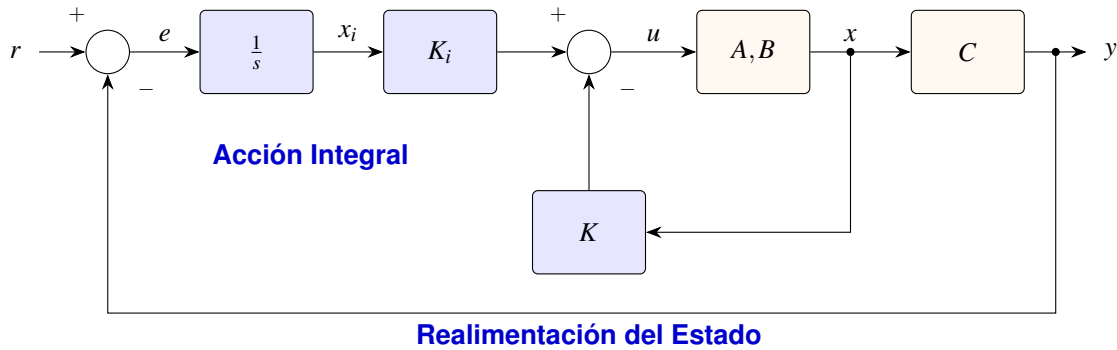


Figura 3.27: Control por realimentación del estado con integrador

Como se observa en la Figura 3.27, el error de posición se desvía hacia un integrador puro ($1/s$) para generar el nuevo estado x_i , cuya corrección se pondera mediante la ganancia K_i . De este modo, la acción integral elimina de forma autónoma el desfase del carro en el riel, mientras que la matriz original K se encarga del resto de estados.

Finalmente, en el Código 3.6 se detalla cómo se traduce esta estructura de control a código C para el microcontrolador.

```

1 // 1. Calculo de errores de posicion y velocidad del carro
2 float e1 = ref_x - x;           // Error de posicion
3 float e3 = 0.0f - x_dot;        // Error de velocidad
4
5 // 2. Errores asociados a la dinamica del pendulo
6 float e2 = ref_theta - theta;
7 float e4 = 0.0f - theta_dot;
8
9 // 3. Integracion numerica del error de posicion en x
10 Ie = Ie + T_sec * e1;
11
12 // 4. Ley de control realimentacion del estado con integrador
13 u = K1 * e1 + K2 * e2 + K3 * e3 + K4 * e4 + KI * Ie;

```

Listing 3.6: Código para el control de posición con término integral

3.5.2 Control de energía (Swing-up) en `calcula_u()`

Para balancear el péndulo desde su posición estable ($\theta = 0\text{rad}$) hasta la vertical superior inestable ($\theta = \pi\text{rad}$), se implementa una estrategia no lineal basada en el control de energía (Sección 2.3.3). A diferencia del control de posición, este algoritmo evalúa continuamente el estado energético del sistema para bombear energía mediante aceleraciones estratégicas del carro ($\ddot{x}_{des}$).

En el Código 3.7 se detalla la **transcripción de la ley de control** implementada en la función `calcula_u()` y descrita teóricamente la Sección 2.3.3.

```

1      // 1. Energia mecanica instantanea (Kin + Pot)
2      E_m = 0.5f * J * theta_dot * theta_dot - m * g * Lcm * cosf(theta);
3
4      // 2. Ley de bombeo de energia (Aceleracion virtual deseada del
5      // carro)
6      float x_ddot_r = Ke * theta_dot * cosf(theta) * (E_m - E_ref) - Kp
7      * x - Kd * x_dot;
8
9      // 3. PFL - Aceleracion angular inducida en el pendulo (lambda)
10     float theta_ddot = (1.0f / J) * (-m * g * Lcm * sinf(theta) - m *
11     Lcm * cosf(theta) * x_ddot_r - Cth * theta_dot);
12
13     // 4. PFL - Calculo de la Fuerza fisica "u" (N) para desacoplar no
14     // linealidades
15     u = (M + m) * x_ddot_r + m * Lcm * cosf(theta) * theta_ddot - m *
16     Lcm * sinf(theta) * (theta_dot * theta_dot) + Cx * x_dot;
17
18     u = u / 0.64f; //Conversion a voltios

```

Listing 3.7: Código para el control de energía con PFL

Con la integración de las leyes de control dentro de la rutina de interrupción periódica del microcontrolador, se completa la transición desde el modelo matemático abstracto y la simulación hasta una implementación física en tiempo real. La arquitectura modular diseñada garantiza que las tareas críticas de adquisición de datos, cálculo de la acción de control y modulación por ancho de pulsos (*PWM*) se ejecuten de manera determinista cada 10ms. Una vez consolidado el firmware y resueltas las particularidades del hardware, el sistema queda completamente preparado para la fase de validación experimental en la plataforma física, cuyos resultados se exponen en el siguiente capítulo.

4. Resultados

4.1 Puesta a punto e integración del hardware

Como paso previo fundamental a la ejecución de las leyes de control, y en consonancia con el objetivo específico **O4** de este proyecto, se llevó a cabo la recuperación mecánica y la sustitución íntegra del sistema electrónico obsoleto de la plataforma *Digital Pendulum 33-005*.

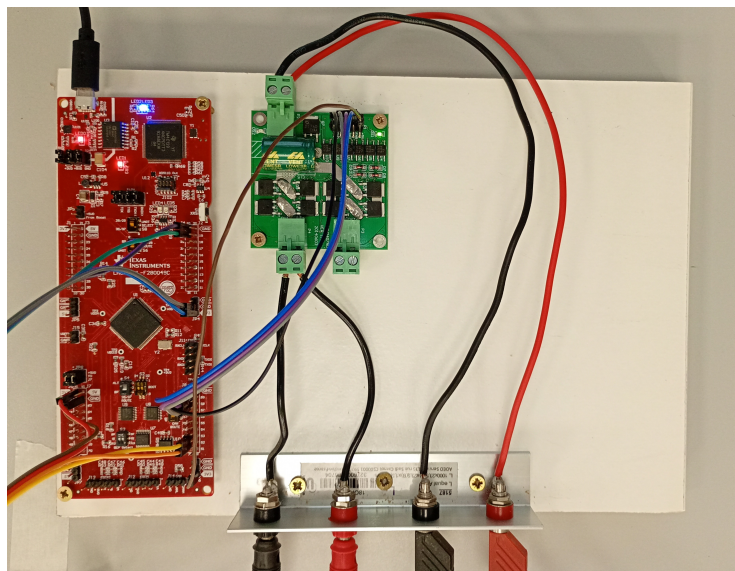


Figura 4.1: Nueva electrónica de control y etapa de potencia.

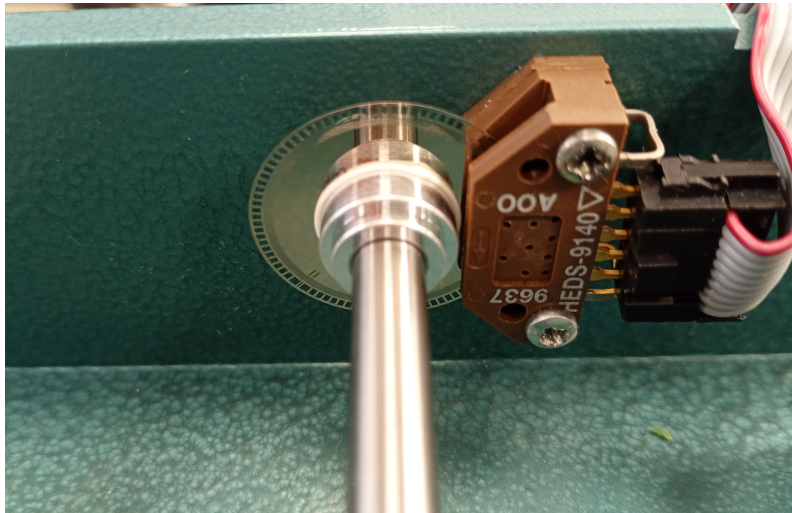
Esta transición desde un módulo comercial cerrado y obsoleto hacia una arquitectura abierta basada en el microcontrolador **TMS320F280049C** representa un salto cualitativo para los laboratorios y la actividad docente de la universidad. Al romper la dependencia con el hardware propietario y las licencias de software descatalogadas del fabricante original, la planta se transforma en una plataforma de **código abierto**.

Este nuevo enfoque fomenta el desarrollo de futuros trabajos universitarios por diversas razones:

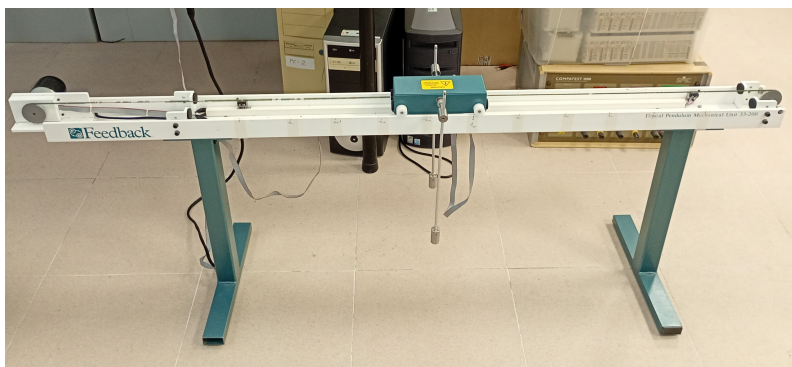
- **Sostenibilidad:** Uso de componentes electrónicos comerciales, económicos y fácilmente sustituibles en caso de avería.
- **Desarrollo y versatilidad científica:** La naturaleza abierta del sistema elimina las restricciones de diseño, fomentando la experimentación y la evolución de nuevas líneas de investigación. Al no depender de herramientas propietarias de terceros, el proyecto se convierte en una base donde futuros estudiantes pueden hacer crecer el ecosistema de desarrollo.

A la par de la renovación electrónica, la restauración de los componentes mecánicos constituyó el factor determinante para probar físicamente los controladores sintonizados en simulación. En primer lugar, el disco óptico deformado del encoder del péndulo se cambió por un nuevo. Esta reparación eliminó la pérdida de pulsos, garantizando una lectura de posición angular correcta.

En segundo lugar, se realizó un reacondicionamiento de la unidad mecánica. Este proceso incluyó la limpieza del riel, la limpieza de residuos de grasa reseca acumulada durante años, el posterior engrasado de los rodamientos del carro y de los ejes de la correa de transmisión y por último, el cambio de la correa dentada por una nueva.



(a) Renovación del disco óptico.



(b) Unidad mecánica reparada y lista para los ensayos.

Figura 4.2: Detalles de la puesta a punto mecánica y del sistema de captación.

4.2 Control de posición

En esta sección se evalúa el comportamiento de la ley de control por realimentación del estado en el sistema real. Las ganancias del controlador ($K = [K_1, K_2, K_3, K_4]$) se ajustaron y validaron en simulación antes de su implementación en el hardware real. El objetivo fundamental es doble:

- **Validación del modelo matemático identificado:** Las ganancias obtenidas en el modelo de simulación deben poder estabilizar el equipo del laboratorio sin necesidad de reajustes empíricos (prueba y error).
- **Análisis teórico en planta real:** Al ajustar las ganancias del controlador con el método de optimización *LQR* (Sección 2.3.5) es interesante analizar cómo afecta la ponderación de Q y R en el comportamiento del sistema real.

Para evaluar de forma homogénea las prestaciones de las distintas configuraciones del controlador, se diseñó un escenario de prueba que consiste en una secuencia de cambios de referencia en forma de escalón para la posición del carro (x_{ref}). La secuencia experimental se define como:

$$x_{ref}(t) = \begin{cases} 0.00 \text{ m} & \text{si } 0 \text{ s} \leq t < 5 \text{ s} \\ 0.15 \text{ m} & \text{si } 5 \text{ s} \leq t < 10 \text{ s} \\ -0.15 \text{ m} & \text{si } 10 \text{ s} \leq t < 15 \text{ s} \\ 0.00 \text{ m} & \text{si } t \geq 15 \text{ s} \end{cases} \quad (4.1)$$

El péndulo se inicializa en la vertical superior inestable ($\theta = \pi$ rad) mediante asistencia inicial, momento en el cual se activa el control de posición.

En el Cuadro 4.1 se muestran los seis controladores probados en la plataforma física, relacionando sus matrices de penalización (Q, R) con las ganancias asociadas y cargadas en el microcontrolador.

Cuadro 4.1: Configuraciones de controladores

Ref.	Matriz Q (Diagonal)	Escalar R	K_1 (x)	K_2 (θ)	K_3 ($\dot{x}$)	K_4 ($\dot{\theta}$)
C1	diag([1, 1, 1, 1])	0.100	-3.1623	70.4535	-15.5513	14.0179
C2	diag([1, 1, 1, 1])	0.010	-10.0000	106.0000	-24.4000	22.0500
C3	diag([1, 1, 1, 1])	0.001	-31.6228	246.8238	-56.5082	57.6621
C4	diag([2.5, 1, 1, 1])	0.001	-50.0000	271.5136	-66.5486	61.8432
C5	diag([10, 1, 1, 1])	0.001	-100.0000	333.2000	-92.7000	72.7000
C6	diag([5, 1, 0.7, 0.1])	0.001	-70.7107	227.8228	-67.6920	45.5221

Para la evaluación cuantitativa de cada controlador, se implementó un flujo continuo de adquisición de datos en tiempo real a través del puerto serie del microcontrolador. La telemetría recopilada de cada experimento, en formato *CSV*, fue posteriormente procesada en el entorno *MATLAB* para sintetizar las curvas de rendimiento sobre la planta física. Con el fin de garantizar la transparencia metodológica y permitir la réplica exacta de los resultados, tanto los registros de datos en bruto (*raw data*) como los scripts de postprocesado se encuentran completamente disponibles en el **repositorio oficial del proyecto** [18].

La Figura 4.3 revela la influencia del valor de los parámetros Q y R en el rendimiento del bucle de control, se observan diferencias en la dinámica en bucle cerrado dependiendo del vector de ganancias ($K = [K_1, K_2, K_3, K_4]$) obtenido.

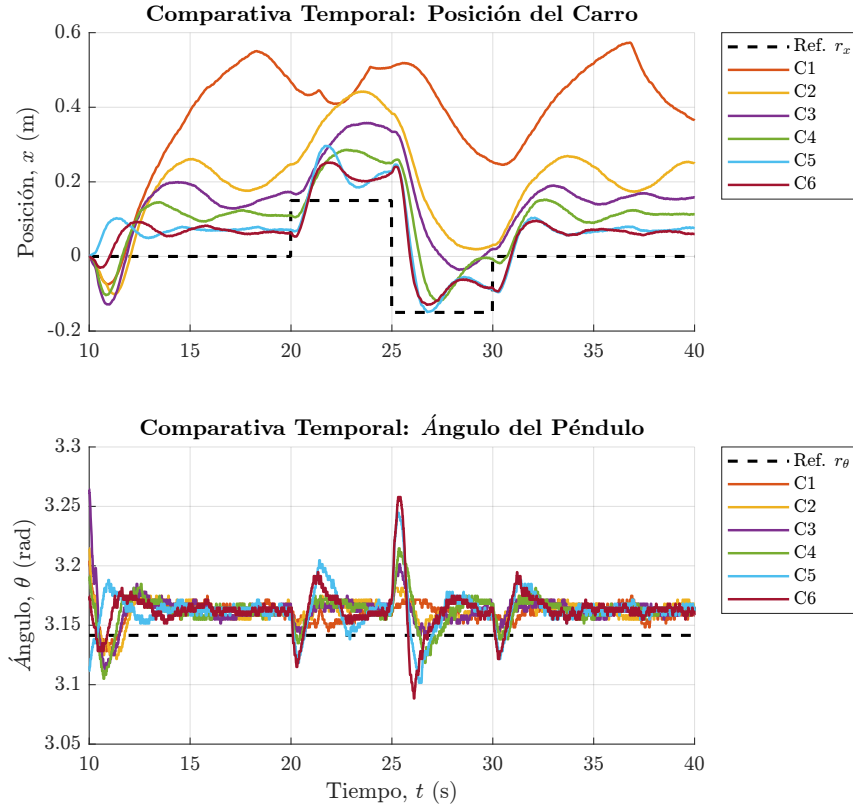


Figura 4.3: Desempeño de los controladores sobre la planta real

A continuación se desglosa el desempeño de cada controlador en función de sus parámetros de diseño (R, Q):

1. **Gasto energético (R):** Físicamente, disminuir R implica tolerar un mayor esfuerzo de actuación en voltios a cambio de minimizar el error más rápidamente. Experimentalmente, **C1** exhibe una respuesta lenta y perezosa ante los cambios de referencia en x . En cambio, **C3** intenta converger más rápidamente hacia x_{ref} a costa de una acción control más agresiva.
2. **Modulación del peso del estado de posición (Diagonal de Q):** Las configuraciones **C4** y **C5** incrementan selectivamente el primer elemento de la diagonal de Q (Q_{11}), el cual penaliza el error de posición del carro ($x_{ref} - x$), manteniendo $R = 0.001$. Al elevar este peso, la ganancia K_1 escala de -31.62 (**C3**) a -50.00 (**C4**) y finalmente a -100.00 (**C5**). Los resultados demuestran que el carro intenta converger hacia las referencias de ± 0.15 m con una velocidad superior. Sin embargo, este comportamiento introduce un compromiso de diseño (*trade-off*): la aceleración brusca del carro para corregir su posición lineal eleva las oscilaciones angulares del péndulo (θ), comprometiendo temporalmente la estabilidad vertical en cada escalón.
3. **Suavizado de la respuesta transitoria (C6):** El controlador **C6** introduce una ponderación estratégica adaptada al entorno real, reduciendo el peso a los errores en la velocidad angular y lineal ($Q_{33} = 0.7, Q_{44} = 0.1$), ya que ambas señales presentan ruido de medida.

El resultado experimental más relevante es que **los seis controladores obtenidos en simulación mantuvieron el péndulo equilibrado en su posición vertical inestable**. Este hecho constituye la prueba del éxito de la transferencia del modelo al sistema físico: certifica que el modelado dinámico, junto con los coeficientes de fricción y masa identificados en el laboratorio, describen la física real del sistema.

No obstante, en todas las gráficas, el error de posición en el estado x no es nulo ya que el carro se detiene a unos milímetros o centímetros de la referencia exacta ($x \neq x_{ref}$). Esta anomalía no responde a un fallo del diseño, sino a fenómenos físicos de la planta real que quedan fuera del alcance del modelo:

- **Zona muerta:** Aunque se hizo una compensación por software (Sección 3.3) eliminarla por completo es muy complicado ya que el rozamiento no es uniforme en todos los puntos del riel.
- **Transmisión:** La elasticidad de la correa es un efecto que juega en contra y es complicado de modelar.

Este comportamiento experimental fue el que motivó a añadir término integral en el control por realimentación del estado (Sección 3.5.1) para incrementar la robustez del controlador ante discrepancias entre el modelado teórico y la realidad física.

Como se aprecia en la Figura 4.4, la inclusión de la acción integral consigue llevar al carro a la posición de referencia. En el momento en que el carro se detiene con un error de posición residual ($x_{ref} - x$), comienza a acumularse de forma continua en el tiempo dentro del estado virtual I_e . Esta integración numérica genera un aumento progresivo y rampa en la señal de control u que, al cabo de unos instantes, logra romper el umbral de bloqueo mecánico y vencer la resistencia de la correa de transmisión.

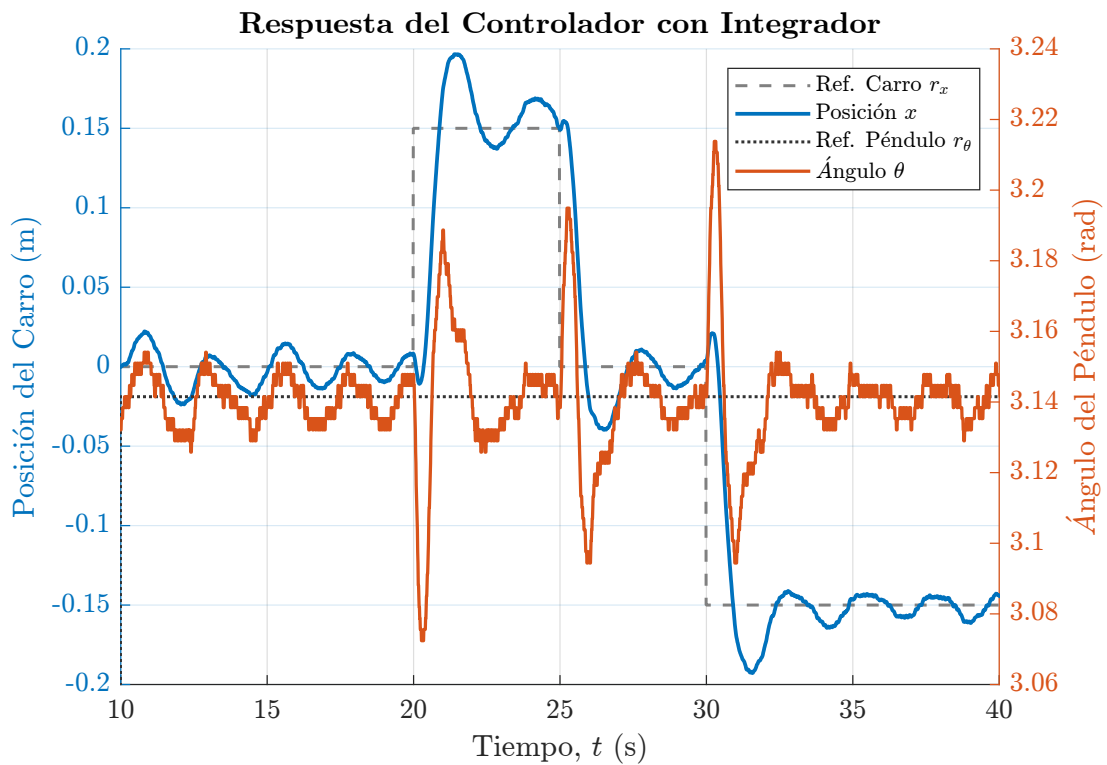


Figura 4.4: Control de posición con integrador sobre la planta real

4.3 Control de energía (Swing-up)

s

4.4 Conmutación entre controles

s

4.5 Comparativa entre los resultados alcanzados y los objetivos iniciales

Aquí hacer una tabla comparativa entre los objetivos propuestos y los resultados reales y los objetivos propuestos.

4.6 Evaluación de la planificación temporal y justificación de desviaciones

5. Conclusiones y posible trabajo futuro

Trabajo a futuro → Implementación en HAL, uso de algoritmos no basados en modelo (Reinforcement Learning)

Bibliografía

- [1] Feedback Instruments Ltd., *Digital Pendulum System: Getting Started (33-005-1M5)*, MATLAB 5 Version. Manual Part No. 1160-330051M5, Feedback Instruments Ltd., Crowborough, UK, 1998 (véase página 8).
- [2] Feedback Instruments Ltd., *Digital Pendulum System: External Interface (33-005-3M5)*, Explains the 16-bit DLL libraries and Real-Time Kernel for Windows 95, Feedback Instruments Ltd., Crowborough, UK, 1998 (véase página 8).
- [3] M. W. Spong, S. Hutchinson y M. Vidyasagar, *Robot modeling and control*. John Wiley & Sons, 2006 (véase página 18).
- [4] R. M. Murray, Z. Li y S. S. Sastry, *A mathematical introduction to robotic manipulation*. CRC press, 1994 (véase página 18).
- [5] R. Tedrake, *Underactuated Robotics: Algorithms for Planning and Control*. MIT Press, 2023, Disponible online en <http://underactuated.mit.edu/> (véanse páginas 19, 23).
- [6] K. J. Åström y K. Furuta, "Swinging up a pendulum by energy control," *Automatica*, volumen 36, páginas 287-295, 2000, Disponible online en <https://www.elsevier.com/locate/automatica> (véase página 21).
- [7] M. W. Spong, "Partial Feedback Linearization of Underactuated Mechanical Systems," en *Proceedings of the 1994 IEEE International Conference on Robotics and Automation*, Coordinated Science Laboratory, University of Illinois at Urbana-Champaign, IEEE, San Diego, CA, mayo de 1994, páginas 2356-2361 (véase página 22).
- [8] I. P. Alós, *Tema 7 - Análisis en Espacio de Estados del comportamiento dinámico de sistemas*, Apuntes de la asignatura IR2124 Sistemas Automáticos Avanzados, Grado en Inteligencia Robótica, Universitat Jaume I de Castelló, 2025 (véanse páginas 24, 25).
- [9] I. P. Alós, *Tema 8 - Control por realimentación del estado*, Apuntes de la asignatura IR2124 Sistemas Automáticos Avanzados, Grado en Inteligencia Robótica, Universitat Jaume I de Castelló, 2025 (véanse páginas 26, 27).

- [10] *TMS320F28004x Real-Time Microcontrollers Data Sheet*, manual, Literature Number: SPRSAD2J. Disponible en <https://www.ti.com/lit/ds/symlink/tms320f280049c.pdf>, Texas Instruments, 2024 (véase página 30).
- [11] T. Instruments, *Code Composer Studio (CCS) Integrated Development Environment*, misc, Disponible en <https://www.ti.com/tool/CCSTUDIO>, 2025 (véanse páginas 30, 44).
- [12] T. Instruments, *C2000Ware Software Development Kit*, misc, Disponible en <https://www.ti.com/tool/C2000WARE>, 2025 (véase página 30).
- [13] *7A-160W Dual Channel DC Motor Drive Module User Manual*, manual, Modelo: AQMH2407ND. Disponible en <https://www.handsontec.com/dataspecs/module/7A-160W%20motor%20control.pdf>, HandsOn Technology, 2024 (véanse páginas 31, 57).
- [14] *L298 Dual Full-Bridge Driver Data Sheet*, manual, Disponible en <https://www.st.com/resource/en/datasheet/l298.pdf>, STMicroelectronics, 2000 (véase página 31).
- [15] *EL357N-G Series: 4 PIN SOP Phototransistor Photocoupler*, manual, Literature Number: DPC-0000014 Rev. 6, Everlight Electronics Co., Ltd., 2014 (véase página 31).
- [16] *HEDS-9040/9140 Series Three Channel Optical Incremental Encoder Modules Data Sheet*, manual, Documento: HEDS-9140-DS100. Disponible en <https://www.broadcom.com/products/motion-control-encoders/incremental-encoders/optical-encoder-modules/heds-9140-a00>, Broadcom (Avago Technologies), 2024 (véase página 32).
- [17] *D.C. Direct Drive Motors: Type 82 760 0*, manual, Disponible en <https://www.crouzet.com/>, Crouzet Automatismes, 2024 (véase página 33).
- [18] M. Porcar, *TFG_Control_MiguelPorcar: Control de sistemas robóticos subactuados*, https://github.com/MPV247/TFG_Control_MiguelPorcar, Accedido: 19 de mayo de 2026, 2026 (véanse páginas 35, 41, 42, 44, 47, 64, 66, 67, 73, 89).
- [19] Feedback Instruments Ltd., *Digital Pendulum System: Teaching Manual (Model 33-005-4M5)*, Edition 01. Part No. 1160-330054M5, Feedback Instruments Ltd., Crowborough, East Sussex, UK, 1998 (véanse páginas 36, 64, 65).
- [20] MathWorks, *Linear-Quadratic Regulator (LQR) design - MATLAB lqr*, misc, Disponible online en la documentación oficial de Control System Toolbox, 2026. dirección: <https://es.mathworks.com/help/control/ref/lti.lqr.html> (véase página 37).
- [21] MathWorks, *Pole placement design - MATLAB place*, misc, Disponible online en la documentación oficial de Control System Toolbox, 2026. dirección: <https://es.mathworks.com/help/control/ref/place.html> (véase página 37).
- [22] Texas Instruments, *C2000Ware Peripheral Driver Library User's Guide*, 2024. dirección: https://software-dl.ti.com/C2000/docs/software_guide/c2000ware/drivers.html (véase página 44).
- [23] Texas Instruments, *TMS320F28004x Real-Time Microcontrollers Technical Reference Manual*, SPRUI33H, Disponible en línea., Texas Instruments, Dallas, Texas, 2024. dirección: <https://www.ti.com/lit/ug/sprui33h/sprui33h.pdf> (véanse páginas 47-49, 51, 52, 54, 90-101, 103).
- [24] I. P. Alós, *Tema 1- Análisis Entrada-Salida del comportamiento dinámico de sistemas*, Apuntes de la asignatura IR2124 Sistemas Automáticos Avanzados, Grado en Inteligencia Robótica, Universitat Jaume I de Castelló, 2025 (véase página 61).

6. Apéndices

A Desarrollo matemático y modelado dinámico complementario

A.1 Desarrollo del Lagrangiano

En este anexo se detalla el desarrollo matemático paso a paso del formalismo de Euler-Lagrange, partiendo de las energías del sistema hasta obtener la formulación matricial.

A.2 Análisis de la dinámica eléctrica del actuador

Si se quiere determinar la frecuencia óptima de operación, se debe analizar en primer lugar la constante de tiempo eléctrica (τ_e) del actuador a partir de su circuito equivalente R-L, si atendemos a los parámetros del fabricante (Tabla 2.2):

$$\tau_e = \frac{L}{R} = \frac{7.2 \text{ mH}}{5.5 \Omega} \approx 1.3 \text{ ms} \quad (6.1)$$

Considerando que se requiere un intervalo mínimo de $5\tau_e$ para que la variable eléctrica alcance el régimen permanente ante un escalón de tensión. El periodo mínimo teórico de la señal *PWM* (T_{PWM}) para despreciar la inductancia de la bobina del motor es:

$$\begin{cases} T_{\text{PWM}} & \geq 5\tau_e = 6.5 \text{ ms} \\ f_{\text{PWM, límite}} & = \frac{1}{T_{\text{PWM}}} = \frac{1}{6.5 \cdot 10^{-3} \text{ s}} \approx 153.84 \text{ Hz} \end{cases} \quad (6.2)$$

Desde el punto de vista del comportamiento estático del motor, trabajar a frecuencias bajas próximas a este límite ofrece la ventaja de reducir la zona muerta del actuador, ya que proporciona el tiempo necesario para que la corriente se establezca y venza la fricción estática.

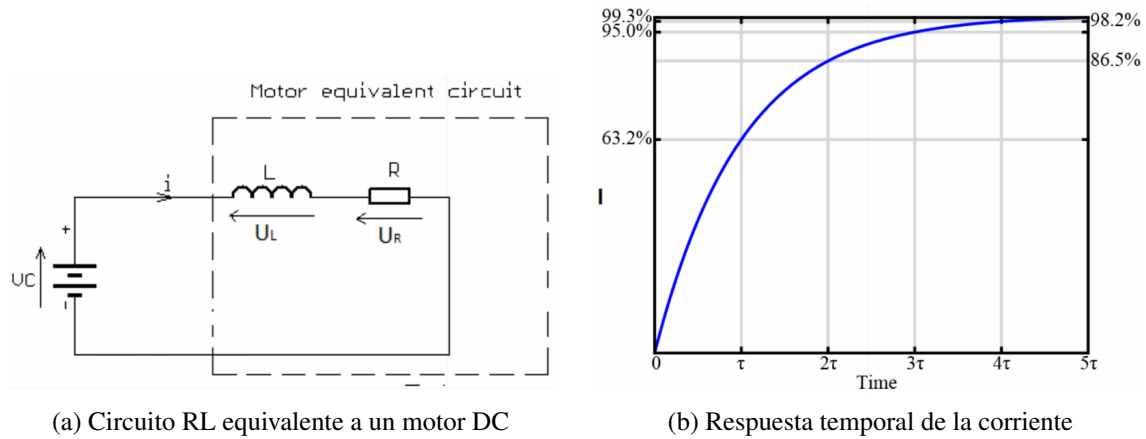


Figura 6.1: Circuito equivalente de un circuito motor DC y su dinámica ante entrada escalón

No obstante, operar el puente en H a frecuencias tan reducidas resulta complicado para el bucle de control debido a ciertas restricciones:

1. **Acoplamiento con la frecuencia de muestreo (f_{control}):** Para asegurar la estabilidad y el determinismo del bucle cerrado, la frecuencia del control debe ser significativamente menor que la frecuencia de conmutación del actuador. Estableciendo un periodo de muestreo para la rutina de control de $T_s = 10 \text{ ms}$ ($f_{\text{control}} = 100 \text{ Hz}$), una frecuencia de *PWM* cercana a 100 Hz se encuentra demasiado en el límite para interferir en el bucle de control.
2. **Sobrecalentamiento por rizado de corriente:** Las frecuencias bajas inducen un rizado de corriente severo incrementando las pérdidas óhmicas en el devanado ($\Delta P = I_{\text{rms}}^2 \cdot R$), derivando en una alta disipación térmica. Aunque para la plataforma experimental este factor no es destructivo (experimentos del orden de un par de minutos), degrada la eficiencia del sistema.
3. **Contaminación acústica:** Las frecuencias del espectro audible humano se sitúan entre 20 Hz y 20 kHz , con la banda de máxima sensibilidad entre 2 kHz y 4 kHz .

A.3 Función de transferencia del carro

En este apéndice se desglosa el procedimiento algebraico para la obtención de la función de transferencia entre voltaje y posición del carro. El sistema físico se modela simplificadaamente como un bloque de masa neta acoplado a un elemento disipativo (amortiguador) sobre el plano horizontal, tal y como se ilustra en la Figura 6.2.

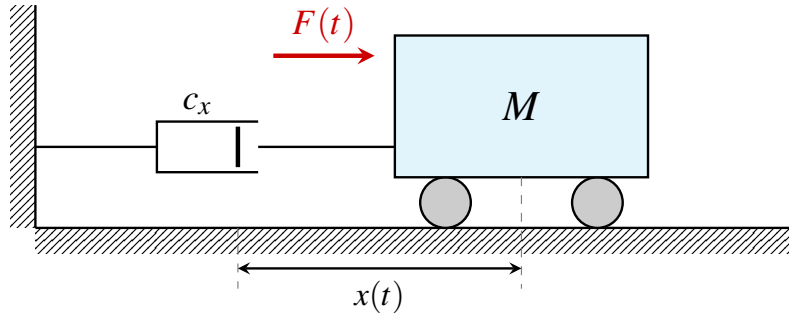


Figura 6.2: Modelo masa amortiguador

La ecuación diferencial que para este modelo simplificado es la siguiente:

$$M \cdot \ddot{x}(t) + c_x \cdot \dot{x}(t) = F(t) \quad (6.3)$$

Aplicando la transformada de Laplace bajo la hipótesis de condiciones iniciales nulas ($\dot{x}(0) = 0, x(0) = 0$), las derivadas temporales se traducen en el operador s :

$$M \cdot s^2 X(s) + c_x \cdot s X(s) = F(s) \quad (6.4)$$

Factorizando el término común del desplazamiento de la variable de estado horizontal $X(s)$, se obtiene:

$$s(M \cdot s + c_x) X(s) = F(s) \quad (6.5)$$

De este modo, la relación de la planta entre la posición de salida y la fuerza de entrada se expresa como:

$$G_{\text{física}}(s) = \frac{X(s)}{F(s)} = \frac{1}{s(M \cdot s + c_x)} \quad (6.6)$$

Para acoplar el modelo al entorno del microcontrolador, se introduce la ganancia estática de conversión del actuador, que vincula la tensión aplicada $U(s)$ con la fuerza horizontal en Newtons mediante el coeficiente K_m :

$$F(s) = K_m \cdot U(s) \quad (6.7)$$

Sustituyendo esta relación de potencia en la transferencia global del sistema, se deduce:

$$G(s) = \frac{X(s)}{U(s)} = \frac{X(s)}{\frac{1}{K_m} F(s)} = K_m \cdot G_{\text{física}}(s) = \frac{K_m}{s(M \cdot s + c_x)} \quad (6.8)$$

Finalmente, se manipula la expresión para obtener la función de transferencia en formato de **ganancia y constante de tiempo**:

$$G(s) = \frac{\frac{K_m}{c_x}}{s \left(\frac{M}{c_x} s + 1 \right)} = \frac{K}{s(1 + \tau s)} \quad (6.9)$$

Donde las agrupaciones de los componentes físicos quedan ligadas a la ganancia estática K y a la constante de tiempo del sistema τ .

B Soporte al trabajo de simulación

B.1 Dinámica en MATLAB Functions

Este apéndice agrupa los bloques de código implementados en Simulink mediante estructuras *MATLAB Function* para modelar la dinámica del sistema mecánico de dos grados de libertad.

MATLAB Function $M(q)^{-1}$

```
function Acel = Minv(q,par, J_p, M, m, l)
    M = [M + m,          +m*l*cos(q(2));
         +m*l*cos(q(2)), J_p];
    Acel = M\par; %Division de matrices.
end
```

Esta función calcula el vector de aceleraciones del sistema a partir de la inversión numérica de la matriz de inercia. Para optimizar el coste computacional, evita el cálculo explícito de la matriz inversa utilizando el operador de división a la izquierda.

Entradas: Vector de posiciones q , vector de fuerzas/pares generalizados remanentes par (derivado de restarle a las fuerzas de control los efectos de fricción, gravedad y Coriolis), inercia del péndulo J_p , masa del carro M , masa del péndulo m , y distancia al centro de masas l .

Salidas: Vector de aceleraciones generalizadas $\ddot{q} = [\ddot{x}, \ddot{\theta}]^T$.

Operación: Resuelve algebraicamente el sistema lineal para obtener $\ddot{q} = M(q)^{-1} \cdot \tau_{neto}$.

MATLAB Function $C(\dot{q}, q)$

```
function par_c = Cq(qdot, q, cx, ctheta, m, l)
    C = [cx,      -m*l*qdot(2)*sin(q(2))
         0,      ctheta      ];
    par_c = C*qdot;
end
```

Esta función evalúa los efectos dinámicos asociados a las fuerzas centrífugas, de Coriolis y a los coeficientes de amortiguamiento viscoso presentes en las articulaciones del mecanismo.

Entradas: Vector de velocidades $qdot = [\dot{x}, \dot{\theta}]^T$, vector de posiciones $q = [x, \theta]^T$, coeficientes de fricción viscosa (c_x, c_θ) , masa m y longitud del péndulo l .

Salidas: Vector de fuerzas generalizadas de Coriolis y amortiguamiento $par_c = C(\dot{q}, q)\dot{q}$.

Operación: Construye la matriz no lineal $C(\dot{q}, q)$ y realiza la multiplicación matricial por el vector de velocidades $(\dot{q})$.

MATLAB Function $G(q)$

```
function par_g = Gq(q, m, g, l)
    G = [0; m*g*l*sin(q(2))];
    par_g = G;
end
```

Esta función determina el par generado por el campo gravitatorio sobre el péndulo basándose en su posición angular.

Entradas: Vector de posiciones q , masa del péndulo m , constante de gravedad g , y longitud l .

Salidas: Vector de fuerzas gravitacionales generalizadas $par_g = G(q)$.

Operación: Evalúa el par gravitatorio que afecta exclusivamente al segundo grado de libertad (coordenada angular del péndulo $\theta = q(2)$).

MATLAB Function B

```
function tau_b = B(u)
    tau_b = [1; 0]*u;
end
```

Esta función mapea la acción de control hacia el espacio de coordenadas generalizadas del modelo dinámico (matriz de acoplamiento de la entrada).

Entradas: Señal de control escalar u (fuerza aplicada sobre el carro).

Salidas: Vector de fuerzas generalizadas de entrada $tau_b = B \cdot u$.

Operación: Proyecta la fuerza u sobre el primer grado de libertad (movimiento lineal del carro), asumiendo que el péndulo no tiene ningún actuador.

B.2 Script control de energía

MATLAB Function Swing-Up con PFL

```
function F = fcn(q, qdot, E_ref, m, l, ctheta, cx, J_p, M)
% --- PARAMETROS FISICOS Y GANANCIAS ---
g = 9.81;
Ke = 26;
kp = 6;
kd = 4;
% -- ESTADOS DEL SISTEMA --
x = q(1);           % Posicion del carro
theta = q(2);       % Angulo del pendulo
xdot = qdot(1);     % Velocidad del carro
thetadot = qdot(2); % Velocidad angular
% --- 1. CALCULO DE ENERGÍA ---
Ep = m * g * l * cos(theta);
Ek = 0.5 * J_p * thetadot^2;
E_total = Ep + Ek;
% --- 2. LEY DE CONTROL DE ENERGIA (Aceleracion 'u') ---
e = E_total - E_ref;
u = Ke * thetadot * cos(theta) * e - kp * x - kd * xdot;
% --- 3. LINEARIZACION POR RETROALIMENTACION PARCIAL (PFL)
---
% Convierte la aceleracion deseada 'u' en Fuerza 'F' para el
motor
thetadotdot = 1/J * (-m*l*cos(theta)*u ...
+ m*g*l*sin(theta) - ctheta*thetadot);
F = (M + m) * u ...
- (m * l * thetadot^2 * sin(theta)) ...
+ (m * l * cos(theta)) * thetadotdot ...
+ cc*xdot;
% --- 4. SATURACION ---
F_max = 2; % Newtons
if F > F_max
F = F_max;
elseif F < -F_max
F = -F_max;
end
end
```

Entradas: Vector de posiciones $q = [x, \theta]^T$, vector de velocidades $qdot = [\dot{x}, \dot{\theta}]^T$, energía mecánica de referencia del péndulo en posición vertical E_{ref} .

Salidas: Fuerza de empuje F (en Newtons).

Operación: El algoritmo ejecuta consecutivamente cuatro etapas fundamentales:

1. **Cálculo de Energía:** Evalúa la energía mecánica total instantánea del péndulo ($E_{total} = E_k + E_p$).
2. **Control de Energía:** Genera una aceleración de referencia u proporcional al error de energía (e).
3. **Linealización PFL:** Traduce analíticamente la aceleración virtual u en la fuerza real F requerida para compensar la dinámica no lineal del carro.
4. **Saturación:** Limita la salida a un umbral simétrico (± 2 N).

B.3 Configuración de la conexión MATLAB ROS

MATLAB Function conversión a mensaje de ROS

```
function msg = create_msg(blankMsg, q, time, start_time_epoch)
    msg = blankMsg;
    %Sincronización con el reloj interno de la máquina.
    total_time = start_time_epoch + time;
    s = floor(total_time);
    ns = (total_time - s) * 1e9;
    msg.header.stamp.sec = int32(s);
    msg.header.stamp.nanosec = uint32(ns);
    % 1. Configurar NOMBRES (coinciden con URDF)
    strArrayToAssign = {'cart_slider', 'pole_joint'};
    positionsToAssign = [q(1), q(2)];
    numStringsInArray = length(strArrayToAssign);
    msg.name_SL_Info.CurrentLength = uint32(numStringsInArray);
    for idx=1:numStringsInArray
        str = strArrayToAssign{idx};
        strLength = length(str);
        msg.name(idx).data(1:strLength) = uint8(str);
        msg.name(idx).data_SL_Info.CurrentLength = uint32(
strLength);
    end
    % 2. POSICIONES (float64 array)
    numPositions = length(positionsToAssign);
    msg.position_SL_Info.CurrentLength = uint32(numPositions);
    msg.position(1:numPositions) = positionsToAssign;
    % 3. VELOCIDAD y ESFUERZO (vacíos)
    msg.velocity_SL_Info.CurrentLength = uint32(0);
    msg.effort_SL_Info.CurrentLength = uint32(0);
end
```

Esta función se encarga de mapear la información matemática simulada hacia un mensaje de tipo *sensor_msgs/JointState*, permitiendo que el visualizador gráfico de ROS (RViz) interprete la telemetría en tiempo real de acuerdo a las articulaciones definidas en el archivo URDF del robot.

Entradas: Estructura base vacía del mensaje de ROS (*blankMsg*), vector de posiciones generalizadas $q = [x, \theta]^T$, tiempo acumulado de la simulación (*time*) y la marca de tiempo absoluta en formato Unix Epoch obtenida al iniciar el proceso (*start_time_epoch*).

Salidas: Mensaje estructurado *msg* (*sensor_msgs/JointState*) listo para ser publicado en ROS.

Operación: El algoritmo procesa los datos siguiendo el protocolo estricto de mensajería de ROS:

1. **Sincronización Temporal:** Combina el tiempo base Unix con el reloj local del solucionador de Simulink, descomponiendo el tiempo total en segundos enteros (*sec*) y nanosegundos fraccionarios (*nanosec*) para rellenar el campo *header.stamp*.
2. **Asignación de Identificadores (Names):** Configura un arreglo de caracteres con los nombres de las articulaciones virtuales descritas en el URDF ('*cart_slider*' y '*pole_joint*').
3. **Inyección de Telemetría (Positions):** Almacena de forma indexada el vector de estados continuos *q* dentro del arreglo de tipo *float64* reservado para las posiciones.
4. **Vaciado de Campos Auxiliares:** Configura explícitamente a cero la longitud de los arreglos de velocidad y esfuerzo (*velocity* y *effort*), solo se requiere el envío de datos posicionales para la sincronización de la cinemática directa en RViz.

En la Figura 6.3 se detalla la parametrización del *solver* numérico. Se ha seleccionado un paso de tiempo fijo (*Fixed-step*) con el algoritmo de integración *ode14x*, estableciendo un tamaño de paso de $\Delta t = 0.01$ s (10 ms).

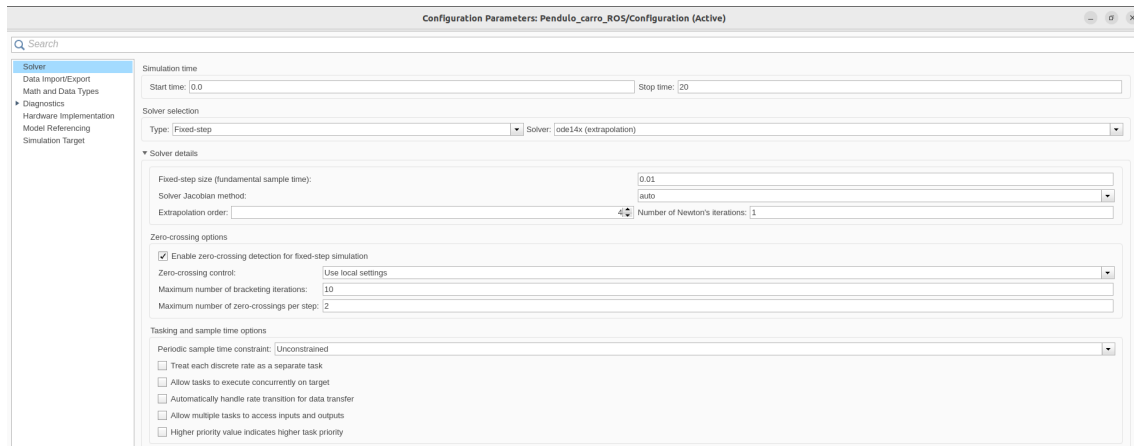


Figura 6.3: Configuración del *solver* con paso de tiempo fijo

Por otro lado, como se ilustra en la Figura 6.4, dentro del menú *Hardware Implementation* se ha establecido como objetivo la plataforma *Robot Operating System 2 (ROS 2)*.

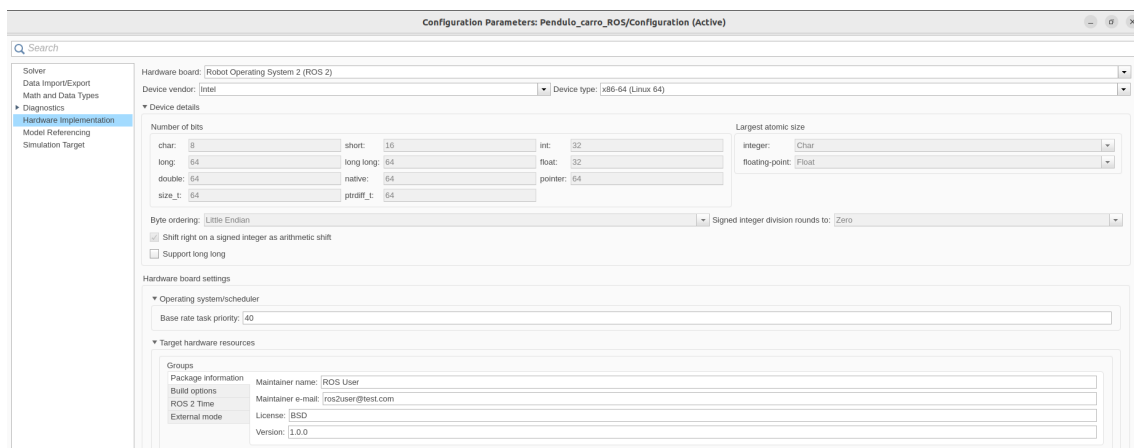


Figura 6.4: *Hardware Implementation*

B.4 Desarrollo de simulaciones recursivas

Para conectar la teoría matemática con la implementación en código, se parte de la definición $\dot{\mathbf{x}}(t) = f(\mathbf{x}(t), u(t))$. Al aproximar la derivada temporal mediante el método de Euler de primer orden para un periodo de muestreo constante T , se obtiene la siguiente relación discreta:

$$\dot{\mathbf{x}}_k \approx \frac{\mathbf{x}_{k+1} - \mathbf{x}_k}{T} \implies \mathbf{x}_{k+1} = \mathbf{x}_k + T \cdot f(\mathbf{x}_k, u_k) \quad (6.10)$$

Donde el vector de derivadas temporales $\mathbf{f} = [f_1, f_2, f_3, f_4]^T$ representa las velocidades y aceleraciones instantáneas del sistema en cada iteración k . Las velocidades se obtienen de forma directa como:

$$f_1 = \dot{x}_k = x_3 \quad (6.11)$$

$$f_2 = \dot{\theta}_k = x_4 \quad (6.12)$$

Sin embargo, para obtener las aceleraciones ($f_3 = \ddot{x}_k$, $f_4 = \ddot{\theta}_k$), se debe aislar el vector de aceleraciones generalizadas del modelo dinámico:

$$\mathbf{M}(\mathbf{q}_k) \begin{bmatrix} \ddot{x}_k \\ \ddot{\theta}_k \end{bmatrix} + \mathbf{C}(\mathbf{q}_k, \dot{\mathbf{q}}_k) \dot{\mathbf{q}}_k + \mathbf{G}(\mathbf{q}_k) = \begin{bmatrix} 1 \\ 0 \end{bmatrix} u_k \quad (6.13)$$

$$\begin{bmatrix} f_3 \\ f_4 \end{bmatrix} = \begin{bmatrix} \ddot{x}_k \\ \ddot{\theta}_k \end{bmatrix} = \mathbf{M}(\mathbf{q}_k)^{-1} \left(\begin{bmatrix} u_k \\ 0 \end{bmatrix} - \mathbf{C}(\mathbf{q}_k, \dot{\mathbf{q}}_k) \dot{\mathbf{q}}_k - \mathbf{G}(\mathbf{q}_k) \right) \quad (6.14)$$

La traducción de este modelo matemático al entorno algorítmico se realiza estructurando las ecuaciones anteriores dentro de un bucle `for`. El código fuente desarrollado se presenta a continuación:

Implementación en MATLAB

```
for k=1:N-1
    % --- 1. CALCULO DE LA ACCION DE CONTROL ---
    u(k) = control(x(:,k), ref);

    % --- 2. DINAMICA (NO LINEAL) DEL SISTEMA ---
    M_t = [M,          +mp*lcm*cos(x(2,k));
           +mp*lcm*cos(x(2,k)),      J];

    C_t = [cc,      -mp*lcm*x(4,k)*sin(x(2,k))
           0,          cp          ];

    G_t = [0; +mp*g*lcm*sin(x(2,k))];

    % --- 3. SIMULACION DE LA SIGUIENTE ITERACION ---
    f1=x(3,k);
    f2=x(4,k);
    f34=M_t \ ([u(k);0]-C_t*x(3:4,k)-G_t);

    x(:,k+1)=x(:,k)+T*[f1;f2;f34];
end
```

El script completo de la simulación, junto con los parámetros reales del sistema y los controladores funcionales detallados, se encuentra disponible para su consulta y descarga en el repositorio del proyecto [18].

C Configuración y registros del microcontrolador (TMS320F280049C)

C.1 Tabla de Multiplexación Completa del TMS320F280049C

0, 4, 8, 12	1	2	3	5	6	7	9	10	11	13	14	15
GPIO0	EPWM1_A				I2CA_SDA							
GPIO1	EPWM1_B				I2CA_SCL							
GPIO2	EPWM2_A			OUTPUT XBAR1	PMBUSA_ SDA		SCIA_TX	FSIRXA_D1				
GPIO3	EPWM2_B	OUTPUT XBAR2		OUTPUT XBAR2	PMBUSA_ SCL	SPIA_CLK	SCIA_RX	FSIRXA_D0				
GPIO4	EPWM3_A			OUTPUT XBAR3	CANA_TX			FSIRXA_ CLK				
GPIO5	EPWM3_B		OUTPUT XBAR3		CANA_RX	SPIA_STE	FSITXA_D1					
GPIO6	EPWM4_A	OUTPUT XBAR4	SYNCOU	EQEP1_A	CANB_TX	SPIB_SOMI	FSITXA_D0					
GPIO7	EPWM4_B		OUTPUT XBAR5	EQEP1_B	CANB_RX	SPIB_SIMO	FSITXA_ CLK					
GPIO8	EPWM5_A	CANB_TX	ADCSOCAO	EQEP1_ STROBE	SCIA_TX	SPIA_SIMO	I2CA_SCL	FSITXA_D1				
GPIO9	EPWM5_B	SCIB_TX	OUTPUT XBAR6	EQEP1_ INDEX	SCIA_RX	SPIA_CLK		FSITXA_D0				
GPIO10	EPWM6_A	CANB_RX	ADCSOCBO	EQEP1_A	SCIB_TX	SPIA_SOMI	I2CA_SDA	FSITXA_ CLK				
GPIO11	EPWM6_B	SCIB_RX	OUTPUT XBAR7	EQEP1_B	SCIB_RX	SPIA_STE	FSIRXA_D1					
GPIO12	EPWM7_A	CANB_TX		EQEP1_ STROBE	SCIB_TX	PMBUSA_ CTL	FSIRXA_D0					
GPIO13	EPWM7_B	CANB_RX		EQEP1_ INDEX	SCIB_RX	PMBUSA_ ALERT	FSIRXA_ CLK					
GPIO14	EPWM8_A	SCIB_TX			OUTPUT XBAR3	PMBUSA_ SDA	SPIB_CLK	EQEP2_A				
GPIO15	EPWM8_B	SCIB_RX			OUTPUT XBAR4	PMBUSA_ SCL	SPIB_STE	EQEP2_B				

Figura 6.5: *GPIO Muxed Pins*, Parte I: GPIO0 a GPIO15 [23]

0, 4, 8, 12	1	2	3	5	6	7	9	10	11	13	14	15
GPIO16	SPIA_SIMO	CANB_TX	OUTPUT XBAR7	EPWM5_A	SCIA_TX	SD1_D1	EQEP1_ STROBE	PMBUSA_ SCL	XCLKOUT			
GPIO17	SPIA_SOMI	CANB_RX	OUTPUT XBAR8	EPWM5_B	SCIA_RX	SD1_C1	EQEP1_ INDEX	PMBUSA_ SDA				
GPIO18_X2	SPIA_CLK	SCIB_TX	CANA_RX	EPWM6_A	I2CA_SCL	SD1_D2	EQEP2_A	PMBUSA_ CTL	XCLKOUT			
GPIO20												
GPIO21												
GPIO22_VFBSW	EQEP1_ STROBE		SCIB_TX		SPIB_CLK	SD1_D4	LINA_TX					
GPIO23_VSW												
GPIO24	OUTPUT XBAR1	EQEP2_A		EPWM8_A	SPIB_SIMO	SD1_D1		PMBUSA_ SCL	SCIA_TX	ERRORSTS		
GPIO25	OUTPUT XBAR2	EQEP2_B			SPIB_SOMI	SD1_C1	FSITXA_D1	PMBUSA_ SDA	SCIA_RX			
GPIO26	OUTPUT XBAR3	EQEP2_ INDEX		OUTPUT XBAR3	SPIB_CLK	SD1_D2	FSITXA_D0	PMBUSA_ CTL	I2CA_SDA			
GPIO27	OUTPUT XBAR4	EQEP2_ STROBE		OUTPUT XBAR4	SPIB_STE	SD1_C2	FSITXA_ CLK	PMBUSA_ ALERT	I2CA_SCL			
GPIO28	SCIA_RX		EPWM7_A	OUTPUT XBAR5	EQEP1_A	SD1_D3	EQEP2_ STROBE	LINA_TX	SPIB_CLK	ERRORSTS		
GPIO29	SCIA_TX		EPWM7_B	OUTPUT XBAR6	EQEP1_B	SD1_C3	EQEP2_ INDEX	LINA_RX	SPIB_STE	ERRORSTS		
GPIO30	CANA_RX		SPIB_SIMO	OUTPUT XBAR7	EQEP1_ STROBE	SD1_D4						
GPIO31	CANA_TX		SPIB_SOMI	OUTPUT XBAR8	EQEP1_ INDEX	SD1_C4	FSIRXA_D1					
GPIO32	I2CA_SDA		SPIB_CLK	EPWM8_B	LINA_TX	SD1_D3	FSIRXA_D0	CANA_TX				
GPIO33	I2CA_SCL		SPIB_STE	OUTPUT XBAR4	LINA_RX	SD1_C3	FSIRXA_ CLK	CANA_RX				
GPIO34	OUTPUT XBAR1				PMBUSA_ SDA							
GPIO35	SCIA_RX		I2CA_SDA	CANA_RX	PMBUSA_ SCL	LINA_RX	EQEP1_A	PMBUSA_ CTL				TDI
GPIO37	OUTPUT XBAR2		I2CA_SCL	SCIA_TX	CANA_TX	LINA_TX	EQEP1_B	PMBUSA_ ALERT				TDO

Figura 6.5: *GPIO Muxed Pins*, Parte II: GPIO16 a GPIO37 (continuación) [23]

0, 4, 8, 12	1	2	3	5	6	7	9	10	11	13	14	15
GPIO39					CANB_RX	FSIRXA_CLK						
GPIO40					PMBUSA_SDA	FSIRXA_D0	SCIB_TX	EQEP1_A				
GPIO41												
GPIO42												
GPIO43												
GPIO44												
GPIO45												
GPIO46												
GPIO47												
GPIO48												
GPIO49												
GPIO50												
GPIO51												
GPIO52												
GPIO53												
GPIO54												
GPIO55												
GPIO56	SPIA_CLK			EQEP2_STROBE	SCIB_TX	SD1_D3	SPIB_SIMO		EQEP1_A			
GPIO57	SPIA_STE			EQEP2_INDEX	SCIB_RX	SD1_C3	SPIB_SOMI		EQEP1_B			
GPIO58				OUTPUT_XBAR1	SPIB_CLK	SD1_D4	LINA_TX	CANB_TX	EQEP1_STROBE			
GPIO59				OUTPUT_XBAR2	SPIB_STE	SD1_C4	LINA_RX	CANB_RX	EQEP1_INDEX			

Figura 6.5: *GPIO Muxed Pins*, Parte III: GPIO39 a GPIO59 (continuación) [23]

C.2 Registros del módulo eQEP

En primer lugar, se proporciona el mapa de memoria global del módulo de hardware para la lectura de encoders (*eQEP*), el cual delimita las direcciones base del periférico y sus registros de control asociados.

Offset	Acronym	Register Name	Write Protection	Section
0h	QPOSCNT	Position Counter		Go
2h	QPOSINIT	Position Counter Init		Go
4h	QPOSMAX	Maximum Position Count		Go
6h	QPOSCMP	Position Compare		Go
8h	QPOSILAT	Index Position Latch		Go
Ah	QPOSSLAT	Strobe Position Latch		Go
Ch	QPOSLAT	Position Latch		Go
Eh	QUTMR	QEP Unit Timer		Go
10h	QUPRD	QEP Unit Period		Go
12h	QWDTMR	QEP Watchdog Timer		Go
13h	QWDPRD	QEP Watchdog Period		Go
14h	QDECCTL	Quadrature Decoder Control		Go
15h	QEPCTL	QEP Control		Go
16h	QCAPCTL	Quadrature Capture Control		Go
17h	QPOSCTL	Position Compare Control		Go
18h	QEINT	QEP Interrupt Control		Go
19h	QFLG	QEP Interrupt Flag		Go
1Ah	QCLR	QEP Interrupt Clear		Go
1Bh	QFRC	QEP Interrupt Force		Go
1Ch	QEPSTS	QEP Status		Go
1Dh	QCTMR	QEP Capture Timer		Go
1Eh	QCPRD	QEP Capture Period		Go
1Fh	QCTMRLAT	QEP Capture Latch		Go
20h	QCPRLAT	QEP Capture Period Latch		Go
30h	REV	QEP Revision Number		Go
32h	QEPSTROBESEL	QEP Strobe select register		Go
34h	QMACTRL	QMA Control register		Go

Figura 6.6: Mapa de memoria del periférico *eQEP* [23]

Debido a la gran densidad de funciones integradas en el periférico y con el propósito de optimizar la lectura técnica de este documento, en las secciones siguientes se desglosa la estructura interna de bits únicamente de aquellos registros de control modificados de forma explícita en la rutina de inicialización. Este análisis selectivo justifica el impacto funcional de cada campo binario configurado sobre el comportamiento dinámico de la planta.

El primer bloque crítico modificado en el código fuente corresponde al registro `QDECCTL`. En la Figura 6.8 se detalla la distribución y propósito de sus campos binarios según la documentación oficial del fabricante.

15	14	13	12	11	10	9	8
QSRC	SOEN	SPSEL	XCR	SWAP	IGATE	QAP	
R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h
7	6	5	4	3	2	1	0
QBP	QIP	QSP	RESERVED				
R/W-0h	R/W-0h	R/W-0h	R-0h				

Figura 6.7: Registro `QDECCTL` [23].

Bit	Field	Type	Reset	Description
15-14	QSRC	R/W	0h	Position-counter source selection Reset type: SYSRSn 0h (R/W) = Quadrature count mode (QCLK = iCLK, QDIR = iDIR) 1h (R/W) = Direction-count mode (QCLK = xCLK, QDIR = xDIR) 2h (R/W) = UP count mode for frequency measurement (QCLK = xCLK, QDIR = 1) 3h (R/W) = DOWN count mode for frequency measurement (QCLK = xCLK, QDIR = 0)
13	SOEN	R/W	0h	Sync output-enable Reset type: SYSRSn 0h (R/W) = Disable position-compare sync output 1h (R/W) = Enable position-compare sync output
12	SPSEL	R/W	0h	Sync output pin selection Reset type: SYSRSn 0h (R/W) = Index pin is used for sync output 1h (R/W) = Strobe pin is used for sync output
11	XCR	R/W	0h	External Clock Rate Reset type: SYSRSn 0h (R/W) = 2x resolution: Count the rising/falling edge 1h (R/W) = 1x resolution: Count the rising edge only
10	SWAP	R/W	0h	CLK/DIR Signal Source for Position Counter Reset type: SYSRSn 0h (R/W) = Quadrature-clock inputs are not swapped 1h (R/W) = Quadrature-clock inputs are swapped
9	IGATE	R/W	0h	Index pulse gating option Reset type: SYSRSn 0h (R/W) = Disable gating of Index pulse 1h (R/W) = Gate the index pin with strobe
8	QAP	R/W	0h	QEPA input polarity Reset type: SYSRSn 0h (R/W) = No effect 1h (R/W) = Negates QEPA input
7	QBP	R/W	0h	QEPB input polarity Reset type: SYSRSn 0h (R/W) = No effect 1h (R/W) = Negates QEPB input
6	QIP	R/W	0h	QEPI input polarity Reset type: SYSRSn 0h (R/W) = No effect 1h (R/W) = Negates QEPI input

Figura 6.8: Distribución de campos y bits de configuración del registro QDECCTL [23].

Bit	Field	Type	Reset	Description
5	QSP	R/W	0h	QEPS input polarity Reset type: SYSRSn 0h (R/W) = No effect 1h (R/W) = Negates QEPS input
4-0	RESERVED	R	0h	Reserved

Figura 6.8: Distribución de campos y bits de configuración del registro QDECCTL [23] (continuación).

A partir de la distribución expuesta en la Figura 6.8, las asignaciones lógicas realizadas en el código se justifican técnicamente de la siguiente manera:

- **QDECCTL.bit.QSRC = 0h:** Configura el modo de conteo en cuadratura pura (*Quadrature count mode*).
- **QDECCTL.bit.SWAP = 0h:** Determina que las entradas de reloj y dirección no se intercambian por hardware (*Not swapped*). Esto asegura la concordancia directa entre el sentido de giro real del eje y el incremento del contador de posición.

Finalmente, para el registro QEPCTL se detalla la distribución y propósito de sus campos binarios según la documentación oficial del fabricante.

15	14	13	12	11	10	9	8
FREE_SOFT		PCRM		SEI		IEI	
R/W-0h		R/W-0h		R/W-0h		R/W-0h	
7	6	5	4	3	2	1	0
SWI	SEL	IEL		QPEN	QCLM	UTE	WDE
R/W-0h	R/W-0h	R/W-0h		R/W-0h	R/W-0h	R/W-0h	R/W-0h

Figura 6.9: Registro QEPCTL [23].

Bit	Field	Type	Reset	Description
15-14	FREE_SOFT	R/W	0h	Emulation mode Reset type: SYSRSn 0h (R/W) = QPOSCNT behavior Position counter stops immediately on emulation suspend 0h (R/W) = QWDTMR behavior Watchdog counter stops immediately 0h (R/W) = QUTMR behavior Unit timer stops immediately 0h (R/W) = QCTMR behavior Capture Timer stops immediately 1h (R/W) = QPOSCNT behavior Position counter continues to count until the rollover 1h (R/W) = QWDTMR behavior Watchdog counter counts until WD period match roll over 1h (R/W) = QUTMR behavior Unit timer counts until period rollover 1h (R/W) = QCTMR behavior Capture Timer counts until next unit period event 2h (R/W) = QPOSCNT behavior Position counter is unaffected by emulation suspend 2h (R/W) = QWDTMR behavior Watchdog counter is unaffected by emulation suspend 2h (R/W) = QUTMR behavior Unit timer is unaffected by emulation suspend 2h (R/W) = QCTMR behavior Capture Timer is unaffected by emulation suspend 3h (R/W) = Same as FREE_SOFT_2
13-12	PCRM	R/W	0h	Position counter reset Reset type: SYSRSn 0h (R/W) = Position counter reset on an index event 1h (R/W) = Position counter reset on the maximum position 2h (R/W) = Position counter reset on the first index event 3h (R/W) = Position counter reset on a unit time event
11-10	SEI	R/W	0h	Strobe event initialization of position counter Reset type: SYSRSn 0h (R/W) = Does nothing (action disabled) 1h (R/W) = Does nothing (action disabled) 2h (R/W) = Initializes the position counter on rising edge of the QEPS signal 3h (R/W) = Clockwise Direction: Initializes the position counter on the rising edge of QEPS strobe Counter Clockwise Direction: Initializes the position counter on the falling edge of QEPS strobe

Figura 6.10: Distribución de campos y bits de configuración del registro QEPCTL [23].

Bit	Field	Type	Reset	Description
9-8	IEI	R/W	0h	Index event init of position count Reset type: SYSRSn 0h (R/W) = Do nothing (action disabled) 1h (R/W) = Do nothing (action disabled) 2h (R/W) = Initializes the position counter on the rising edge of the QEPI signal (QPOSCNT = QPOSINIT) 3h (R/W) = Initializes the position counter on the falling edge of QEPI signal (QPOSCNT = QPOSINIT)
7	SWI	R/W	0h	Software init position counter Reset type: SYSRSn 0h (R/W) = Do nothing (action disabled) 1h (R/W) = Initialize position counter (QPOSCNT=QPOSINIT). This bit is not cleared automatically
6	SEL	R/W	0h	Strobe event latch of position counter Reset type: SYSRSn 0h (R/W) = The position counter is latched on the rising edge of QEPS strobe (QPOSSLAT = POSCCNT). Latching on the falling edge can be done by inverting the strobe input using the QSP bit in the QDECCTL register 1h (R/W) = Clockwise Direction: Position counter is latched on rising edge of QEPS strobe Counter Clockwise Direction: Position counter is latched on falling edge of QEPS strobe
5-4	IEL	R/W	0h	Index event latch of position counter (software index marker) Reset type: SYSRSn 0h (R/W) = Reserved 1h (R/W) = Latches position counter on rising edge of the index signal 2h (R/W) = Latches position counter on falling edge of the index signal 3h (R/W) = Software index marker. Latches the position counter and quadrature direction flag on index event marker. The position counter is latched to the QPOSILAT register and the direction flag is latched in the QEPSTS[QDLF] bit. This mode is useful for software index marking.
3	QPEN	R/W	0h	Quadrature position counter enable/software reset Reset type: SYSRSn 0h (R/W) = Reset the eQEP peripheral internal operating flags/read-only registers. Control/configuration registers are not disturbed by a software reset. When QPEN is disabled, some flags in the QFLG register do not get reset or cleared and show the actual state of that flag. 1h (R/W) = eQEP position counter is enabled
2	QCLM	R/W	0h	QEP capture latch mode Reset type: SYSRSn 0h (R/W) = Latch on position counter read by CPU. Capture timer and capture period values are latched into QCTMRLAT and QCPRDLAT registers when CPU reads the QPOSCNT register. 1h (R/W) = Latch on unit time out. Position counter, capture timer and capture period values are latched into QPOSILAT, QCTMRLAT and QCPRDLAT registers on unit time out.
1	UTE	R/W	0h	QEP unit timer enable Reset type: SYSRSn 0h (R/W) = Disable eQEP unit timer 1h (R/W) = Enable unit timer
0	WDE	R/W	0h	QEP watchdog enable Reset type: SYSRSn 0h (R/W) = Disable the eQEP watchdog timer 1h (R/W) = Enable the eQEP watchdog timer

Figura 6.10: Distribución de campos y bits de configuración del registro QEPCTL [23] (continuación).

A partir de la distribución expuesta en la Figura 6.10, las configuraciones aplicadas en el firmware se justifican técnicamente de la siguiente manera:

- **QEPCTL.bit.FREE_SOFT = 2h:** Impide que el contador de posición se detenga debido a un punto de ruptura en la depuración.
- **QEPCTL.bit.QPEN = 1h:** Habilita formalmente el contador de posición del eQEP.

C.3 Registros del módulo ePWM

Se proporciona el mapa de memoria global del módulo de hardware para la modulación por ancho de pulso (*ePWM*), el cual delimita las direcciones base del periférico y sus registros de control asociados.

Offset	Acronym	Register Name	Write Protection	Section
0h	TBCTL	Time Base Control Register		Go
1h	TBCTL2	Time Base Control Register 2		Go
4h	TBCTR	Time Base Counter Register		Go
5h	TBSTS	Time Base Status Register		Go
8h	CMPCTL	Counter Compare Control Register		Go
9h	CMPCTL2	Counter Compare Control Register 2		Go
Ch	DBCTL	Dead-Band Generator Control Register		Go
Dh	DBCTL2	Dead-Band Generator Control Register 2		Go
10h	AQCTL	Action Qualifier Control Register		Go
11h	AQTSRCSEL	Action Qualifier Trigger Event Source Select Register		Go
14h	PCCTL	PWM Chopper Control Register		Go
18h	VCAPCTL	Valley Capture Control Register		Go
19h	VCNTCFG	Valley Counter Config Register		Go
20h	HRCNFG	HRPWM Configuration Register	EALLOW	Go
21h	HRPWR	HRPWM Power Register	EALLOW	Go
26h	HRMSTEP	HRPWM MEP Step Register	EALLOW	Go
27h	HRCNFG2	HRPWM Configuration 2 Register	EALLOW	Go
2Dh	HRPCTL	High Resolution Period Control Register	EALLOW	Go
2Eh	TRREM	HRPWM High Resolution Remainder Register	EALLOW	Go
34h	GLDCTL	Global PWM Load Control Register	EALLOW	Go
35h	GLDCFG	Global PWM Load Config Register	EALLOW	Go
38h	EPWMXLINK	EPWMx Link Register		Go
40h	AQCTLA	Action Qualifier Control Register For Output A		Go
41h	AQCTLA2	Additional Action Qualifier Control Register For Output A		Go
42h	AQCTLB	Action Qualifier Control Register For Output B		Go
43h	AQCTLB2	Additional Action Qualifier Control Register For Output B		Go
47h	AQSFR	Action Qualifier Software Force Register		Go
49h	AQCSFR	Action Qualifier Continuous S/W Force Register		Go
50h	DBREDHR	Dead-Band Generator Rising Edge Delay High Resolution Mirror Register		Go
51h	DBRED	Dead-Band Generator Rising Edge Delay High Resolution Mirror Register		Go
52h	DBFEDHR	Dead-Band Generator Falling Edge Delay High Resolution Register		Go
53h	DBFED	Dead-Band Generator Falling Edge Delay Count Register		Go
60h	TBPHS	Time Base Phase High		Go
62h	TBPRDHR	Time Base Period High Resolution Register		Go
63h	TBPRD	Time Base Period Register		Go
6Ah	CMPSA	Counter Compare A Register		Go

Figura 6.11: Mapa de memoria del periférico *ePWM* [23]

Se desglosa la estructura interna de bits únicamente de aquellos registros de control modificados de forma explícita en la rutina de inicialización. Para el caso concreto de este periférico se abordan los registros: TBCTL y AQCTLA.

El primer bloque crítico modificado en el código fuente corresponde al registro TBCTL. En la Figura 6.13 se detalla la distribución y propósito de sus campos binarios según la documentación oficial del fabricante.

15	14	13	12	11	10	9	8
FREE_SOFT		PHSDIR	CLKDIV		HSPCLKDIV		
R/W-0h		R/W-0h	R/W-0h		R/W-1h		
7	6	5	4	3	2	1	0
HSPCLKDIV	SWFSYNC	SYNCOSSEL		PRDL	PHSEN	CTRMODE	
R/W-1h	R-0/W1S-0h	R/W-0h		R/W-0h	R/W-0h	R/W-3h	

Figura 6.12: Registro TBCTL [23].

Bit	Field	Type	Reset	Description
15-14	FREE_SOFT	R/W	0h	Emulation Mode Bits. These bits select the behavior of the ePWM time-base counter during emulation events 00: Stop after the next time-base counter increment or decrement 01: Stop when counter completes a whole cycle: - Up-count mode: stop when the time-base counter = period (TBCTR = TBPRD) - Down-count mode: stop when the time-base counter = 0x00 (TBCTR = 0x00) - Up-down-count mode: stop when the time-base counter = 0x00 (TBCTR = 0x00) 1x: Free run Reset type: SYSRSn
13	PHSDIR	R/W	0h	Phase Direction Bit This bit is only used when the time-base counter is configured in the up-down-count mode. The PHSDIR bit indicates the direction the time-base counter (TBCTR) will count after a synchronization event occurs and a new phase value is loaded from the phase (TBPHS) register. This is irrespective of the direction of the counter before the synchronization event.. In the up-count and down-count modes this bit is ignored. 0: Count down after the synchronization event. 1: Count up after the synchronization event. Reset type: SYSRSn
12-10	CLKDIV	R/W	0h	Time Base Clock Pre-Scale Bits These bits select the time base clock pre-scale value (TBCLK = EPWMCLK/(HSPCLKDIV * CLKDIV): 000: /1 (default on reset) 001: /2 010: /4 011: /8 100: /16 101: /32 110: /64 111: /128 Reset type: SYSRSn

Figura 6.13: Distribución de campos y bits de configuración del registro TBCTL [23]

Bit	Field	Type	Reset	Description
9-7	HSPCLKDIV	R/W	1h	High Speed Time Base Clock Pre-Scale Bits These bits determine part of the time-base clock prescale value. $TBCLK = EPWMCLK / (HSPCLKDIV \times CLKDIV)$. This divisor emulates the HSPCLK in the TMS320x281x system as used on the Event Manager (EV) peripheral. 000: /1 001: /2 (default on reset) 010: /4 011: /6 100: /8 101: /10 110: /12 111: /14 Reset type: SYSRSn
6	SWFSYNC	R-0/W1S	0h	Software Forced Sync Pulse 0: Writing a 0 has no effect and reads always return a 0. 1: Writing a 1 forces a one-time synchronization pulse to be generated. SWFSYNC affects EPWMxSYNCO only when SYNCOSSEL = 00. Reset type: SYSRSn
5-4	SYNCOSSEL	R/W	0h	Sync Output Select 00: EPWMxSYNCO / SWFSYNC 01: CTR = zero: Time-base counter equal to zero (TBCTR = 0x00) 10: CTR = CMPB: Time-base counter equal to counter-compare B (TBCTR = CMPB) 11: EPWMxSYNCO is defined by TBCTL2[SYNCOSSELX] Reset type: SYSRSn
3	PRDL	R/W	0h	Active Period Reg Load from Shadow Select 0: The period register (TBPRD) is loaded from its shadow register when the time-base counter, TBCTR, is equal to zero and/or a sync event as determined by the TBCTL2[PRDLDSYNC] bit. A write/read to the TBPRD register accesses the shadow register. 1: Immediate Mode (Shadow register bypassed): A write or read to the TBPRD register accesses the active register. Reset type: SYSRSn
2	PHSEN	R/W	0h	Counter Reg Load from Phase Reg Enable 0: Do not load the time-base counter (TBCTR) from the time-base phase register (TBPHS). 1: Allow Counter to be loaded from the Phase register (TBPHS) and shadow to active load events when an EPWMxSYNCO input signal occurs or a software-forced sync signal, see bit 6. Reset type: SYSRSn
1-0	CTRMODE	R/W	3h	Counter Mode The time-base counter mode is normally configured once and not changed during normal operation. If you change the mode of the counter, the change will take effect at the next TBCLK edge and the current counter value shall increment or decrement from the value before the mode change. These bits set the time-base counter mode of operation as follows: 00: Up-count mode 01: Down-count mode 10: Up-down count mode 11: Freeze counter operation (default on reset) Reset type: SYSRSn

Figura 6.13: Distribución de campos y bits de configuración del registro TBCTL [23] (continuación)

Valores modificados en la función de inicialización para obtener un comportamiento de señal determinado:

- **TBCTL.bit.CTRMODE = 2h (TB_COUNT_UPDOWN):** Establece el modo de conteo triangular simétrico (ascendente-descendente).
- **TBCTL.bit.HSPCLKDIV = 5h:** El valor numérico 5h actúa como un índice de configuración por hardware que le indica al microcontrolador que debe aplicar un divisor de frecuencia real entre 10 sobre el reloj principal del sistema.
- **TBCTL.bit.CLKDIV = 1h:** El valor numérico 1h indica un divisor de 2 para la frecuencia base. (CORREGIR ESTO)

Finalmente, para el registro AQCTLA se detalla la distribución y propósito de sus campos binarios según la documentación oficial del fabricante.

15	14	13	12	11	10	9	8
RESERVED				CBD		CBU	
R-0-0h				R/W-0h		R/W-0h	
7	6	5	4	3	2	1	0
CAD		CAU		PRD		ZRO	
R/W-0h		R/W-0h		R/W-0h		R/W-0h	

Figura 6.14: Registro AQCTLA [23].

Bit	Field	Type	Reset	Description
15-12	RESERVED	R-0	0h	Reserved
11-10	CBD	R/W	0h	Action When TBCTR = CMPB on Down Count Note: By definition, in count up-down mode when the counter equals 0 the direction is defined as 1 or counting up. 00: Do nothing (action disabled) 01: Clear: force EPWMxA output low. 10: Set: force EPWMxA output high. 11: Toggle EPWMxA output: low output signal will be forced high, and a high signal will be forced low. Reset type: SYSRSn
9-8	CBU	R/W	0h	Action When TBCTR = CMPB on Up Count Note: By definition, in count up-down mode when the counter equals 0 the direction is defined as 1 or counting up. 00: Do nothing (action disabled) 01: Clear: force EPWMxA output low. 10: Set: force EPWMxA output high. 11: Toggle EPWMxA output: low output signal will be forced high, and a high signal will be forced low. Reset type: SYSRSn
7-6	CAD	R/W	0h	Action When TBCTR = CMPA on Down Count Note: By definition, in count up-down mode when the counter equals 0 the direction is defined as 1 or counting up. 00: Do nothing (action disabled) 01: Clear: force EPWMxA output low. 10: Set: force EPWMxA output high. 11: Toggle EPWMxA output: low output signal will be forced high, and a high signal will be forced low. Reset type: SYSRSn
5-4	CAU	R/W	0h	Action When TBCTR = CMPA on Up Count Note: By definition, in count up-down mode when the counter equals 0 the direction is defined as 1 or counting up. 00: Do nothing (action disabled) 01: Clear: force EPWMxA output low. 10: Set: force EPWMxA output high. 11: Toggle EPWMxA output: low output signal will be forced high, and a high signal will be forced low. Reset type: SYSRSn

Figura 6.15: Distribución de campos y bits de configuración del registro AQCTLA [23]

Bit	Field	Type	Reset	Description
3-2	PRD	R/W	0h	Action When TBCTR = TBPRD Note: By definition, in count up-down mode when the counter equals 0 the direction is defined as 1 or counting up. 00: Do nothing (action disabled) 01: Clear: force EPWMxA output low. 10: Set: force EPWMxA output high. 11: Toggle EPWMxA output: low output signal will be forced high, and a high signal will be forced low. Reset type: SYSRSn
1-0	ZRO	R/W	0h	Action When TBCTR = 0 Note: By definition, in count up-down mode when the counter equals 0 the direction is defined as 1 or counting up. 00: Do nothing (action disabled) 01: Clear: force EPWMxA output low. 10: Set: force EPWMxA output high. 11: Toggle EPWMxA output: low output signal will be forced high, and a high signal will be forced low. Reset type: SYSRSn

Figura 6.15: Distribución de campos y bits de configuración del registro AQCTLA [23] (continuación)

A partir de la distribución expuesta en la Figura 6.15, los eventos lógicos configurados en el firmware se justifican técnicamente de la siguiente manera:

- **AQCTLA.bit.CAU = 1h (AQ_CLEAR):** Fuerza de forma síncrona el pin físico a un nivel lógico bajo (0 V) cuando el contador de tiempo base (TBCTR) se encuentra en su fase ascendente (*Up-count*) e intersecta exactamente el valor numérico umbral almacenado en el registro comparador CMPA.
- **AQCTLA.bit.CAD = 2h (AQ_SET):** Fuerza el pin físico a un nivel lógico alto (3.3 V) cuando el contador de tiempo base (TBCTR) cambia a su fase descendente (*Down-count*) e intersecta nuevamente el valor guardado en el comparador CMPA.

C.4 Tasa de baudios del SCI

En este anexo se detalla el cálculo y la configuración de los registros SCIHBAUD y SCILBAUD del periférico *SCI* para configurar la tasa de baudios en la comunicación serie. Antes de desarrollar el cálculo dado el valor de ambos registros se dispone de la descripciones de cada registro proporcionadas por *Texas Instruments*.

Bit	Field	Type	Reset	Description
15-8	RESERVED	R	0h	Reserved
7-0	BAUD	R/W	0h	SCI 16-bit baud selection Registers SCIHBAUD (MSbyte). The internally-generated serial clock is determined by the low speed peripheral clock (LSPCLK) signal and the two baud-select registers. The SCI uses the 16-bit value of these registers to select one of 64K serial clock rates for the communication modes. $BRR = (SCIHBAUD \ll 8) + (SCILBAUD)$ The SCI baud rate is calculated using the following equation: SCI Asynchronous Baud = $LSPCLK / ((BRR + 1) * 8)$ Alternatively, $BRR = LSPCLK / (SCI Asynchronous Baud * 8) - 1$ Note that the above formulas are applicable only when $0 < BRR < 65536$. If $BRR = 0$, then SCI Asynchronous Baud = $LSPCLK / 16$ Where: BRR = the 16-bit value (in decimal) in the baud-select registers Reset type: SYSRSn

Figura 6.16: Descripción del registro SCIHBAUD [23]

Bit	Field	Type	Reset	Description
15-8	RESERVED	R	0h	Reserved
7-0	BAUD	R/W	0h	See SCIHBAUD Detailed Description Reset type: SYSRSn

Figura 6.17: Descripción del registro SCILBAUD [23]

A partir de las especificaciones de los registros ilustradas en la Figura 6.16 y la Figura 6.17, se observa que ambos bloques de 8 bits se combinan internamente para formar una única palabra de 16 bits encargada de almacenar el divisor de velocidad, denominado operativamente como *Baud Rate Register (BRR)* [23].

Tomando como base las asignaciones hexadecimales implementadas en el firmware de inicialización de este proyecto:

```

1 SciaRegs.SCIHBAUD.all = 0x0000; // Configura el MSByte del divisor
2 SciaRegs.SCILBAUD.all = 0x001A; // Configura el LSByte del divisor

```

La tasa de baudios a partir del valor almacenado en ambos registros se obtiene del siguiente modo [23]:

$$BRR = (SCIHBAUD \ll 8) + SCILBAUD \quad (6.15)$$

Sustituyendo los valores cargados en el código:

$$BRR = (0x00 \ll 8) + 0x1A = 0x001A = 26 \text{ (en base decimal)} \quad (6.16)$$

Según la Figura 6.16 la tasa de baudios en función de BRR se define con la siguiente operación matemática [23]:

$$\text{Baud} = \frac{f_{LSPCLK}}{(BRR + 1) \times 8} \quad (6.17)$$

Finalmente, considerando el reloj del periférico SCI en $f_{LSPCLK} = 25\text{ MHz}$, y aplicando el valor decimal calculado para el divisor ($BRR = 26$), se determina la tasa de baudios real:

$$\text{Baud Real} = \frac{25000000}{(26+1) \times 8} = \frac{25000000}{27 \times 8} = \frac{25000000}{216} \approx 115740.74\text{ bps} \quad (6.18)$$

Notemos que el valor obtenido no es exactamente de 115200 bps, luego error cometido respecto a la tasa teórica deseada de es:

$$\text{Error (\%)} = \left| \frac{\text{Baud Teórico} - \text{Baud Real}}{\text{Baud Teórico}} \right| \times 100 \quad (6.19)$$

$$\text{Error (\%)} = \left| \frac{115200 - 115740.74}{115200} \right| \times 100 = \left| \frac{-540.74}{115200} \right| \times 100 \approx 0.469\% \quad (6.20)$$

Este porcentaje de error es mínimo y no representa un problema de sincronismo entre emisión y recepción en la comunicación UART.

C.5 Configuración de la ISR en detalle

En este anexo se desglosa el funcionamiento interno del bloque de temporizadores de la CPU del **TMS320F280049C** y se detalla el firmware de bajo nivel desarrollado para establecer el determinismo temporal del sistema.

Arquitectura interna y registros del CPU-Timer

El microcontrolador dispone de tres temporizadores de 32 bits de propósito general (TIMER0/1/2), cuyo esquema lógico de bloques de Texas Instruments se detalla en la Figura 6.18.

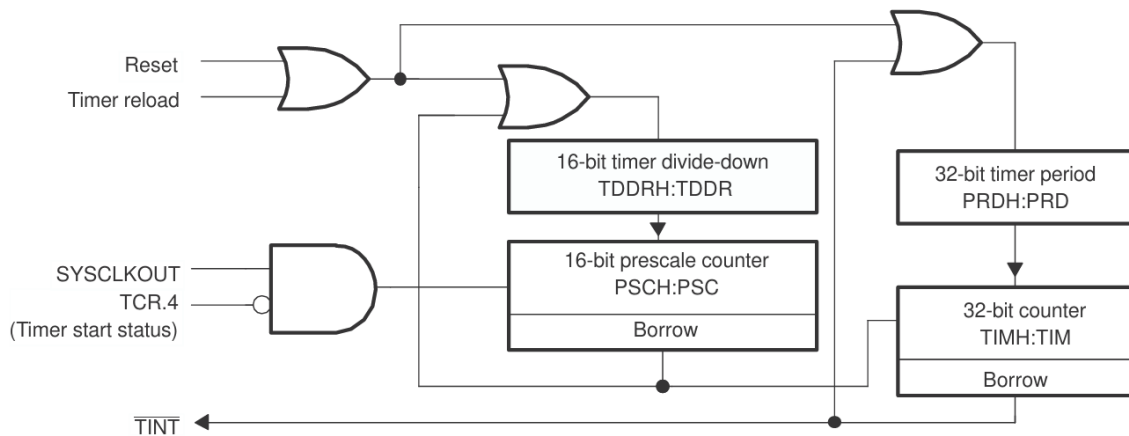


Figura 6.18: Diagrama lógico de bloques y registros asociados de los CPU Timers [23]

Como se observa en el diagrama de la Figura 6.18, la base de tiempo se rige por un contador decreciente de 32 bits compuesto por los registros `TIMH : TIM` [23]. El funcionamiento del hardware se describe bajo los siguientes parámetros de control:

- **Registro de Periodo (`PRDH : PRD`):** Almacena el valor máximo de la cuenta atrás que equivale al periodo deseado (10ms). Cada vez que el contador principal llega a cero, los registros de periodo vuelven a cargar el valor original de manera automática si se activa la señal de *Timer reload*.
- **Pre-escalador (`TDDRH : TDDR` y `PSCH : PSC`):** Permiten dividir la frecuencia del reloj principal del sistema (`SYSCLKOUT`) para ralentizar la velocidad de la cuenta si la aplicación requiriese periodos extremadamente largos. En este diseño, se configura para que el temporizador decrezca a la frecuencia nativa del procesador (100MHz).
- **Generación de Alerta (`TINT`):** Cuando el contador decrementa y alcanza el valor cero, se produce un evento de desbordamiento por préstamo (*Borrow*). Este flanco lógico es el encargado de disparar la señal de interrupción física hacia el procesador.

Iniciación a bajo nivel

```

1  void setup_TimeInterrupt(void (*isr_ptr)(void), uint32_t T)
2  {
3      EALLOW;
4      // 1. Redirección en la Tabla de Vectores del PIE
5      PieVectTable.TIMER0_INT = isr_ptr;
6      EDIS;
7
8      // 2. Inicialización y carga del periodo
9      InitCpuTimers();
10     // Reloj a 100 MHz, Periodo T en us.
11     ConfigCpuTimer(&CpuTimer0, 100, T);
12
13     // 3. Configuración del registro TCR
14     CpuTimer0Regs.TCR.all = 0x4000;
15
16     // 4. Apertura de compuertas en el modulo PIE y la CPU
17     IER |= M_INT1;
18     PieCtrlRegs.PIEIER1.bit.INTx7 = 1;
19
20     // 5. Activación global del tiempo real
21     EINT;
22     ERTM;
23 }

```

Listing 6.1: Rutina de bajo nivel para la inicialización del Timer 0

Análisis de las instrucciones críticas de registro

1. **Protección de Memoria (EALLOW/EDIS):** La tabla de vectores que contiene las direcciones de las interrupciones críticas está protegida por hardware contra escrituras accidentales. La instrucción en ensamblador EALLOW deshabilita temporalmente esta protección, permitiendo sobrescribir la posición de memoria PieVectTable.TIMER0_INT con el puntero de nuestra función control, cerrando el acceso seguro inmediatamente después con EDIS.
2. **Habilitación en el registro de control (CpuTimer0Regs.TCR.all = 0x4000):** Al cargar el valor hexadecimal 0x4000, se establece a 1 exclusivamente el bit número 14 del registro, correspondiente al bit TIE (*Timer Interrupt Enable*). Este bit le da permiso explícito al temporizador para generar un pulso de hardware en el pin interno TINT cada vez que la cuenta regresiva toque el cero.
3. **Configuración de Máscaras Jerárquicas (IER y PIEIER1):** El pulso de interrupción debe atravesar la arquitectura en cascada. La instrucción PieCtrlRegs.PIEIER1.bit.INTx7 = 1 abre la compuerta específica para el Timer 0 dentro del Grupo 1 del multiplexor de interrupciones. Posteriormente, la línea de código IER |= M_INT1 habilita el paso del Grupo 1 completo directamente hacia el núcleo de procesamiento de la CPU.
4. **Líneas de Ejecución (EINT / ERTM):** La macro EINT activa de forma global todas las interrupciones del microcontrolador. Por su parte, ERTM (*Enable Real-Time Mode*) es una directiva fundamental para las herramientas de depuración en tiempo real; permite que, incluso si el microcontrolador está conectado al ordenador enviando datos de telemetría o monitorizando variables, la interrupción del temporizador mantenga su máxima prioridad física de ejecución.